

M&A OPERATING SYSTEM

Product Design

AI Analytics Platform

Draft v1.0 · May 2026

Andrew Bush (www.maoperatingsystem.com/bio-andrew-bush)

Contents

- About This Document..... 4
 - About M&A Operating System..... 5
 - Find Out More 6
- 1. Executive Overview: Governed AI-Enabled Analytics and Data Mining..... 7
 - Introduction 7
 - Table of Contents..... 8
 - The Proposed Approach 9
 - Accuracy, Traceability, and Repeatability 10
 - Why Text-to-SQL Is Not the Long-Term Foundation.....11
 - Proposed Architecture at a Glance 12
 - How the Platform Works 13
 - Examples by Persona 14
 - Supporting Roles and Activities..... 15
 - Design Principles..... 16
 - Implementation, Success, and Development..... 17
 - The Executive Decision..... 18
- 2. Proposed Architecture and Capabilities..... 19
 - Purpose and Boundaries 19
 - Platform Roles 20
 - Architecture and Request Flow 21
 - AI Consumers 24
 - MCP Capability Layer (MCP)..... 25
 - Intent Resolution Agent (IRA) 27
 - Semantic Metrics Repository (SMR)..... 29
 - Role-Aware Projection Layer (RAPL) 32
 - Semantic Validation Layer (SVL)..... 35
 - Semantic Controls Layer (SCL)..... 37
 - Physical Query Planner (PQP) 39
 - Federated Query Engine (FQE) 41
 - Data Visualization Language (DVL) 45
 - Narrative Synthesis Agent (NSA) 47
 - Analytical Lineage Store (ALS) 49
 - Provenance Artifact Service (PAS)..... 52
 - Structured Response..... 54
 - Governed Data-Mining Example 56

External Components.....	59
Design Decisions Requiring Evaluation.....	61
3. Glossary.....	62
Appendix A. Text-to-SQL: Appropriate Use and Governance Boundary	66
Purpose	66
Where Text-to-SQL Adds Value	67
Why It Is Not the Governed Execution Layer	68
A Complementary Operating Model.....	70
Evaluation Implication	71
Conclusion.....	72
Appendix B. Illustrative Reference Implementation and Evaluation Plan	73
Reference Build Summary.....	74
Architecture Overview	79
Layer-by-Layer Stack Decisions.....	80
Infrastructure	128
Evaluation Plan and Decision Gates.....	145

About This Document

This document is part of the **M&A Operating System** product design specification series. It describes the intended design, architecture, and behavior of the platform within. It is intended for internal distribution across product, engineering, and design review functions.

This is a living specification. It reflects design intent at the time of publication and will be updated as the product evolves.

About M&A Operating System

M&A Operating System is a boutique advisory firm focused on helping organizations modernize the operational foundations required for scalable analytics, trusted enterprise information, and practical AI adoption.

Our work sits at the intersection of enterprise data strategy, information architecture, operating model modernization, and transformation execution. We specialize in helping organizations design practical, business-aligned approaches to enterprise information management that support operational scalability, analytics enablement, governance, and modern AI capabilities.

A core part of our approach is the recognition that successful data and AI initiatives are not driven solely by technology platforms. Long-term success depends on how well organizations structure, model, govern, operationalize, and align information with real business processes and decision-making.

Our Product Design document series reflects this philosophy. These documents are intended to provide practical frameworks, methodologies, architectural concepts, and implementation guidance that help organizations move beyond theoretical strategy into executable operational design.

M&A Operating System's approach combines: - Enterprise data and AI strategy - Subject-based data modeling and information architecture - Semantic and operational data design - Governance and organizational alignment - Operating model modernization - Transformation execution and delivery leadership - Practical AI operationalization and readiness

Our goal is simple: help organizations build practical, scalable, and trusted information foundations that improve operational effectiveness, accelerate analytics and AI adoption, and support better business outcomes.

Find Out More

To learn more about M&A Operating System, explore the platform, or speak with our team: maoperatingsystem.com

1. Executive Overview: Governed AI-Enabled Analytics and Data Mining

Introduction

About This Document

This technical white paper proposes an alternative approach to AI-enabled analytics and data mining in highly regulated and governed environments, where accuracy, traceability, and repeatability are critical. It is written for executives, architects, analytics leaders, and governance teams evaluating how to extend AI access to enterprise data.

The proposal combines AI interpretation with approved semantic definitions, controlled computation, and recorded evidence. The remaining sections explain the architecture, illustrate a technology stack, propose evaluation measures, examine Text-to-SQL risks, and discuss possible extensions. Examples are illustrative; this paper does not report deployment results or establish regulatory compliance.

Table of Contents

- [The Proposed Approach](#)
- [Accuracy, Traceability, and Repeatability](#)
- [Why Text-to-SQL Is Not the Long-Term Foundation](#)
- [Proposed Architecture at a Glance](#)
- [How the Platform Works](#)
- [Examples by Persona](#)
- [Supporting Roles and Activities](#)
- [Design Principles](#)
- [Implementation, Success, and Development](#)
- [The Executive Decision](#)

The Proposed Approach

We propose a governed semantic execution layer that gives people and AI applications access to approved calculations and datasets.

Business users need answers expressed in business terms: portfolio performance, risk exposure, liquidity, or operating outcomes. Those answers depend on agreed definitions, reliable source data, and permission to use it. When teams reconstruct those decisions for each report or request, they repeat work and risk producing different answers to the same business question.

The paper uses the name AI Analytics Platform for this proposed architecture. It makes approved analytical definitions reusable across conversations, applications, and automated analysis. A user asks a question through an existing interface. The platform identifies the approved calculation, checks access, executes it against registered sources, and returns the result with an explanation of how it was produced.

The platform sits above existing warehouses, data services, and specialist analytical engines. It requires supported connections and accurate mappings to those sources. Organizations can retain their existing investments while making governed analytics available to more consumers.

The proposed scope includes metric queries, drilldown, and governed bulk dataset retrieval for data-mining workflows. Responses include results, governed chart or table specifications, optional narrative summaries, and audit evidence. Consumer applications provide the user experience. The platform governs the dataset supplied to a data-mining workflow; it does not, by itself, validate downstream feature engineering, model training, or research conclusions. Section 2 demonstrates this boundary with a governed data-mining example.

Accuracy, Traceability, and Repeatability

The proposal addresses three distinct requirements. Each needs its own evidence before adoption.

Requirement	Meaning and Proposed Basis
Accuracy	A result answers the intended question using the correct business definition and suitable source data. Approved formulas, request validation, and comparison with independently verified results support this requirement. Consistent execution alone does not establish correctness.
Traceability	An authorized reviewer can follow a result back to the request, definition versions, permissions, source references, and execution decisions. The proposed lineage record preserves those links under an agreed retention policy.
Repeatability	An authorized rerun using the same request, data snapshot, definitions, permissions, and calculation settings produces the same result within a declared numerical tolerance. Version and snapshot preservation make this test possible.

For data mining, these requirements apply to the extracted dataset as well as to calculated metrics: which records and fields were supplied, under which selection rules, and from which source state. Subsequent model training or analysis requires its own validation.

Why Text-to-SQL Is Not the Long-Term Foundation

We do not believe Text-to-SQL alone is a sustainable long-term foundation for AI-enabled analytics in a highly regulated organization.

Text-to-SQL uses a large language model (LLM) to turn a question into a database query. These implementations are easy to stand up and quick to demonstrate: a model receives database schemas, generates a query, and returns an answer through a conversational interface. This makes them an attractive starting point for AI-enabled analytics and exploration.

The challenge is moving from a successful demonstration to recurring business metrics, data-mining workflows, automated decision inputs, and regulatory submissions. A convincing answer in a demonstration does not show that the calculation is correct, that a reviewer can trace it, or that an authorized rerun can repeat it. This paper proposes governed semantic execution as the alternative foundation for those workloads.

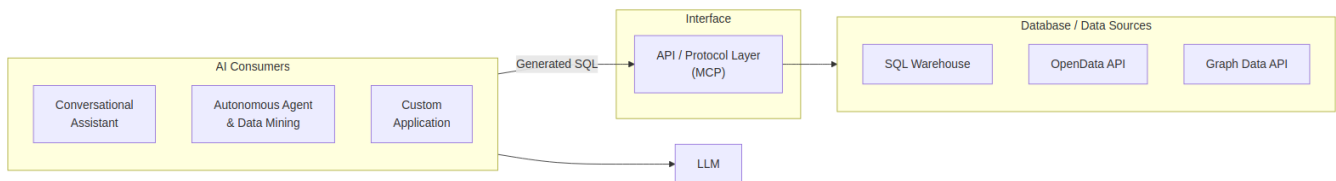
A generated query does not, by itself, establish which business definition was approved, which version applied, or whether the result can be reproduced. Database permissions and query validation can restrict execution, but they do not decide what a regulated metric means. Organizations still need accountable owners, approved formulas, controlled changes, and evidence linking each result to those decisions.

The proposed platform makes those requirements part of the execution process. AI selects from approved analytical definitions; deterministic software constructs and executes the calculation. This reduces reliance on the model inferring business meaning from physical schemas on each request.

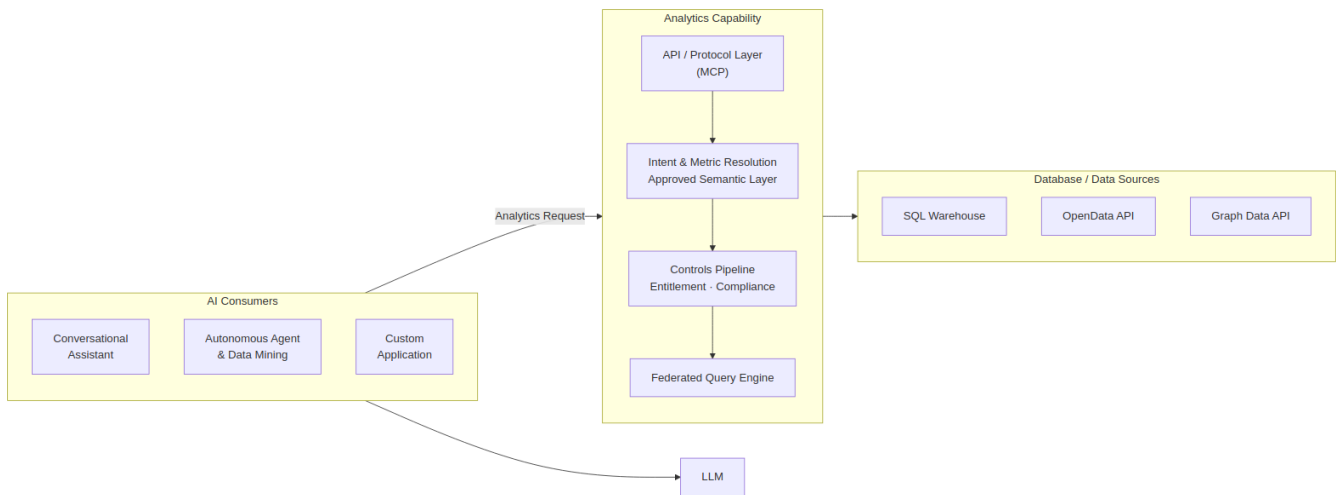
Proposed Architecture at a Glance

In both approaches, AI clients and LLMs are present. The difference is what happens next. In Text-to-SQL, the AI generates queries directly against raw database schemas. In the AI Analytics Platform, it submits structured requests to a governed semantic layer, and the platform handles all computation, governance, and execution.

Text-to-SQL Approach



AI Analytics Platform



In the platform approach, every request routes through the API layer, traverses the invariant controls sequence, and produces an audit record. No path to execution backends, physical schemas, or raw data exists outside that pipeline.

The Text-to-SQL appendix explains the appropriate role of Text-to-SQL and the boundary between controlled exploration and governed execution.

Text-to-SQL remains useful alongside the platform for controlled exploration and discovering new analytical needs. A shared conversational interface can support both paths, provided users can distinguish exploratory output from governed results. Useful exploratory calculations can become candidates for review and registration. AI may help draft those definitions; accountable people approve them before governed use.

How the Platform Works

In a governed AI-enabled analytics situation, the Semantic Metrics Repository (SMR) provides a catalog of approved metrics, dimensions, operations, and dataset contracts. Each definition states what the analysis means, how it is calculated, and which data it uses. Versioning preserves the definition associated with a historical result.

A request follows a common sequence:

1. **Understand the question.** AI matches a natural-language request to registered operations and asks for clarification when needed. Applications that already know the required operation can submit it directly.
2. **Check permission and controls.** The platform checks identity, permitted metrics and data, query size, complexity, and compliance purpose before execution.
3. **Calculate the result.** The platform builds a query from approved definitions and executes it across supported sources. AI does not write the executable query or calculate the returned values.
4. **Present and record the answer.** The platform returns data with a governed presentation specification and an optional AI summary. It records the definitions, access decisions, and execution details needed to explain the result.

AI consumers connect through the Model Context Protocol (MCP), a standard interface for tools and data. The same controls apply to people using conversational assistants and to autonomous applications.

For a compliance-relevant metric requested for a compliance purpose, the design adds a signed evidence package and an export gate. Export remains blocked until the required evidence is sealed. This supports review; it does not replace the organization's approval process or establish regulatory compliance by itself.

Reproducibility depends on preserving the relevant data snapshot, definition versions, access policies, and calculation settings. A consistent formula cannot produce an identical historical result from changed source data.

Examples by Persona

User Need	Platform Response
A portfolio manager compares returns with a benchmark.	The platform applies the approved return definitions to authorized portfolios and returns a comparison with its calculation record.
A risk officer investigates a limit breach.	The platform combines registered risk and limit data, identifies breaches, and supports further investigation within the officer's permitted scope.
A research application retrieves historical positions.	The platform supplies an approved dataset, limits fields and rows to the application's permissions, and records the retrieval.
A treasury analyst prepares liquidity figures for a submission.	The platform calculates the approved liquidity coverage ratio and produces the additional evidence required by its compliance-purpose controls.

These scenarios use the same governed service. Their differences are the approved operation, the caller's permissions, the requested output, and the evidence required.

Supporting Roles and Activities

The platform depends on accountable ownership of definitions, permissions, and source data.

Data Modellers describe the source data and its business meaning. Metrics Modellers define calculations and dataset contracts. Analytics Governance approves definitions and oversees analytical quality. The Entitlements Manager maintains access policies, Integration Engineers connect sources, and Platform Admins operate the service.

This work determines what the platform can answer. An unregistered business concept requires a reviewed definition before it becomes available through the governed path. Catalog coverage therefore grows through a maintained approval process.

The platform cannot correct an unsuitable formula or poor source data merely by executing consistently. Owners must establish data quality, freshness, retention, and acceptable uses. Security and compliance teams must validate the controls for the intended deployment.

Design Principles

The following principles retain the identifiers used throughout the specification.

Principle	Executive Meaning
P1 - Semantic abstraction	Users work with approved business concepts. Access to physical execution details is separately controlled.
P2 - Controls before execution	Requests must pass the required checks before data-source execution begins.
P3 - Deterministic metric resolution	Each metric resolves to an approved, versioned definition.
P4 - Complete analytical lineage	Accepted requests produce evidence of the definitions, decisions, and execution or failure involved.
P5 - Role-aware by default	Access requires explicit permission and is restricted to the caller's authorized data.
P6 - Governed narrative	AI summaries are checked against computed results and withheld when validation fails.
P7 - Deterministic visualization	Registered rules select the chart or table format for the result.
P8 - Explainability at every layer	Authorized users can inspect the interpretation, calculation, and supporting evidence.
P9 - Administrator sovereignty within governance bounds	Administrators configure the service within mandatory governance constraints.
P10 - Deterministic computation, not generation	Approved software computes values from registered definitions; AI interprets requests and summarizes results.

Implementation, Success, and Development

The architecture separates required behavior from technology choices. The proposed reference stack uses Python services, Starburst for federated query execution, PostgreSQL and object storage for policies and evidence, Redis for caching and concurrency, and language models for interpretation and summaries. These choices require validation against the organization's sources, scale, and operating requirements.

Success means that users obtain the analysis they need with approved definitions, appropriate access, and usable evidence. The organization should measure adoption, definition coverage, data freshness, response time, failures, cost, and the integrity of the audit record. Proposed targets require workload testing and a baseline; they are not demonstrated results.

Possible future extensions include scheduled monitoring, alerts, saved queries, and collaboration. They are outside the proposed governed core and are not commitments about current availability or delivery dates.

Supporting Section	What It Adds
2. Proposed Architecture and Capabilities	Defines roles, component responsibilities, interfaces, and controls.
3. Glossary	Defines the named components and technical terms.
Appendix A. Text-to-SQL	Defines the boundary between controlled exploration and governed execution.
Appendix B. Illustrative Reference Implementation and Evaluation Plan	Applies the logical design to one proposed technology stack and defines how that implementation should be evaluated.

The Executive Decision

Adoption requires commitment to a governed analytical service and the people who maintain its definitions and controls.

A bounded evaluation should establish whether the proposed platform can serve a meaningful set of business questions across the required sources, enforce existing permissions, and produce evidence that reviewers can use. Leadership should name the accountable owners and agree the measures of success before that evaluation begins.

The decision to expand should follow evidence that users can obtain reliable analysis with less repeated definition and reconciliation work, while the organization retains control over access, calculation changes, and audit records.

2. Proposed Architecture and Capabilities

Purpose and Boundaries

This section defines the proposed logical architecture for governed AI-enabled analytics and data mining. It explains each component's responsibility and the contracts between components. It is implementation-neutral: it names required behavior, inputs, outputs, and evidence without selecting products, programming languages, frameworks, storage services, or deployment patterns. Appendix B maps these responsibilities to one illustrative technology stack.

The design separates AI-assisted interpretation from governed computation. A language model may match a question to an approved operation and summarize a completed result. Registered definitions, deterministic software, and controlled data services perform the calculation. A structured caller may bypass natural-language interpretation, but it may not bypass entitlements, validation, controls, execution evidence, or other mandatory stages.

Component names identify logical responsibilities. They do not require one independently deployed service per component. Required behaviors describe the proposal, not verified features of a released product.

Platform Roles

The operating model separates analytical use, definition ownership, access policy, governance, integration, and platform operation. One person may hold several roles, but the organization must review combinations that create conflicts of interest.

Role	Primary Responsibility
Analytical End User	Requests governed analysis and views results within assigned permissions.
Power Analyst	Performs deeper exploration, drilldown, and authorized evidence export.
Data Modeller	Maintains business and physical data metadata in the Semantic Data Repository (SDR).
Metrics Modeller	Authors metric, dimension, operation, and dataset definitions for review.
Entitlements Manager	Maintains analytical access, row-scope, and masking policies.
Analytics Governance	Approves definitions and oversees analytical quality and catalog integrity.
Integration Engineer	Registers supported source connections and physical mappings.
Platform Admin	Operates the service within governance policy. This role does not confer analytical data access.

Production access must come from approved entitlement policy, not from an authoring or administrative role alone.

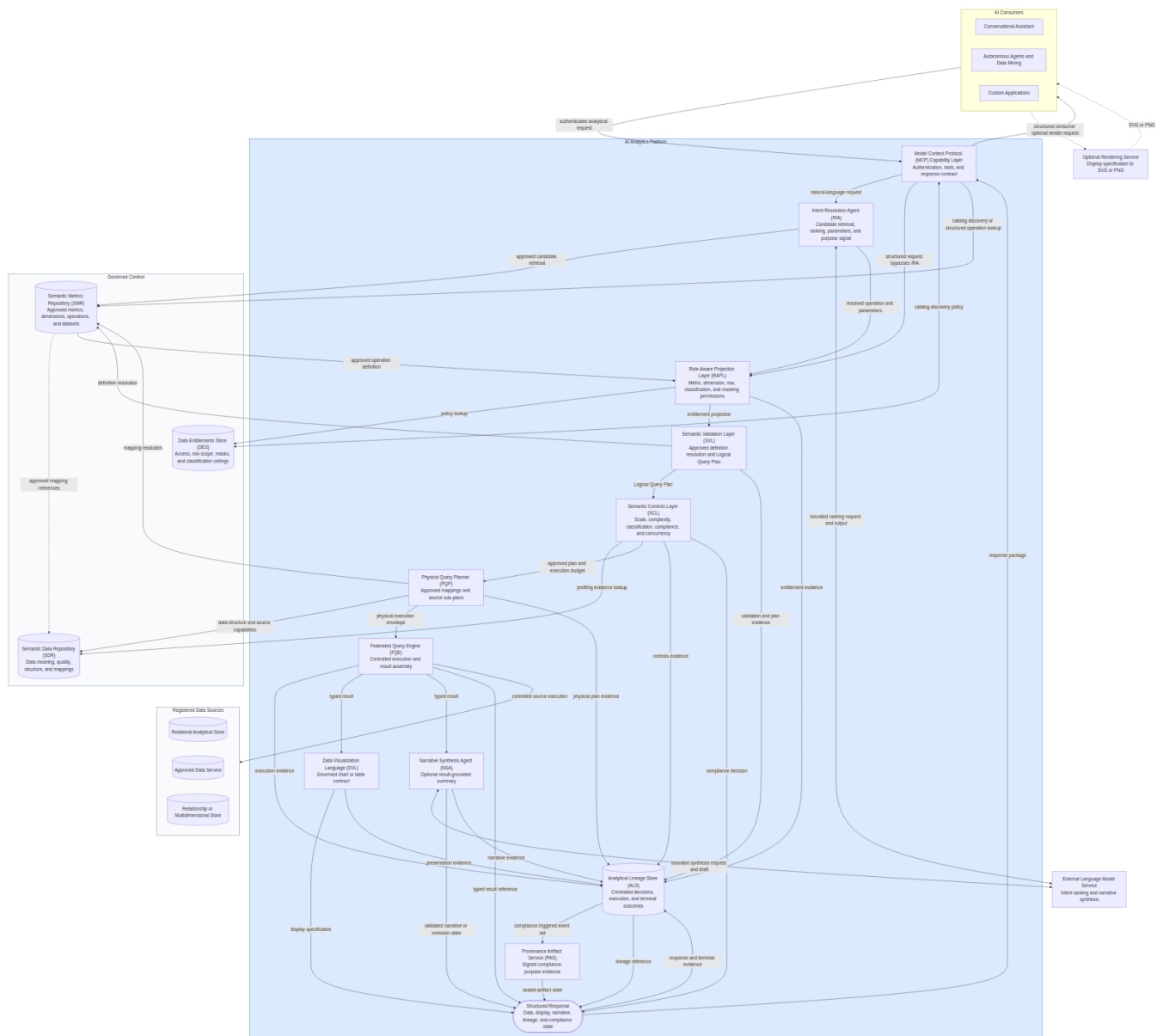
Role and Capability Boundaries

Activity	Accountable Role	Required Separation
Use approved analytics	Analytical End User or Power Analyst	Access remains limited by Data Entitlements Store (DES) policy.
Author data metadata and mappings	Data Modeller	Production approval remains with the designated governance process.
Author metrics and operations	Metrics Modeller	The author cannot make a definition available merely by submitting it.
Approve analytical definitions	Analytics Governance	Approval is recorded against a versioned definition.
Maintain entitlement policy	Entitlements Manager	Platform administration does not imply authority to grant data access.
Register source connections	Integration Engineer	Credentials remain inside the controlled execution boundary.
Operate platform services	Platform Admin	Administrative access does not imply permission to run analytical queries.

The implementation must record who proposed, reviewed, approved, deprecated, and used each governed definition. Organizations may assign these responsibilities differently, but they must preserve accountable approval and prevent infrastructure privileges from becoming analytical entitlements.

Architecture and Request Flow

The governed path applies the same controls to conversational users, autonomous agents, and custom applications.



Component Interaction Map

The architecture is not a strictly linear sequence. Some components provide lookups, several write evidence during the request, structured requests bypass IRA, and presentation components run in parallel. The component contracts below therefore associate inputs and outputs with the component that owns them rather than with a numbered step.

Component	Business Role	Receives	Provides
AI Consumers	Start governed analysis and present the answer.	Business question or approved operation; structured response	Authenticated analytical request; optional render request

Component	Business Role	Receives	Provides
MCP Capability Layer	Govern the platform boundary and public contract.	Authenticated request; DES catalog-discovery policy; assembled response package	Entitlement-scoped catalog view; correlated natural-language request to IRA; structured request with approved SMR operation reference to RAPL; structured response to consumer
Intent Resolution Agent (IRA)	Translate business language into an approved analytical request.	Natural-language request; approved candidates from SMR; bounded model output	Resolved operation and parameters; confidence and purpose evidence; clarification request when needed
Semantic Metrics Repository (SMR)	Govern reusable analytical meaning and versions.	Authoring changes; discovery and definition lookups; mapping lookups	Approved operations, metrics, dimensions, datasets, presentation metadata, and mapping references
Semantic Data Repository (SDR)	Govern data meaning, structure, quality, and mapping context.	Approved data metadata and lineage updates	Versioned data context and mapping targets referenced by SMR and PQP
Data Entitlements Store (DES)	Govern business-level analytical access policy independently.	Approved role and access-policy changes	Versioned metric, dimension, row-scope, mask, and classification policies
Role-Aware Projection Layer (RAPL)	Determine the caller's permitted analytical scope.	Resolved request; authenticated identity; DES policies	Entitlement projection to SVL; entitlement evidence to ALS
Semantic Validation Layer (SVL)	Turn an entitled request into a valid logical calculation.	Resolved request; entitlement projection; approved SMR definitions	Logical Query Plan to SCL; validation and plan evidence to ALS; structured rejection
Semantic Controls Layer (SCL)	Decide whether a valid plan may execute under current policy and conditions.	Logical Query Plan; versioned controls; profiling evidence; operating state	Approved plan and budget to PQP; controls evidence to ALS; compliance decision to response assembly; structured rejection
Physical Query Planner (PQP)	Translate logical calculations into bounded source instructions.	Approved Logical Query Plan; execution budget; SMR and SDR mapping context	Physical execution envelope to FQE; plan integrity evidence to ALS
Federated Query Engine (FQE)	Execute approved source instructions and assemble controlled results.	Physical execution envelope; registered source connections	Typed result to DVL, NSA, and response assembly; execution evidence to ALS; explicit terminal outcome
Registered Data Sources	Execute bounded operations over governed business data.	Authorized source instructions from FQE	Typed source data or explicit failure to FQE
Data Visualization Language (DVL)	Define a consistent governed presentation.	Typed result; resolved intent; approved labels and units	Display specification to response assembly; presentation evidence to ALS
Narrative Synthesis Agent (NSA)	Produce an optional result-grounded explanation.	Bounded result content; bounded model draft	Validated narrative or omission state to response assembly; narrative evidence to ALS

Component	Business Role	Receives	Provides
External Language Model Service	Support bounded language ranking and drafting.	Candidate-ranking request from IRA; synthesis request from NSA	Ranking or draft text back to the requesting component
Analytical Lineage Store (ALS)	Preserve the correlated evidence chain across the request.	Intent, entitlement, validation, controls, plan, execution, and presentation events	Lineage reference to response assembly; event set to PAS and audit export
Provenance Artifact Service (PAS)	Seal additional evidence for compliance-purpose results.	Active compliance trigger; correlated ALS event set	Sealed artifact-state reference or failure state to response assembly; export state
Structured Response	Assemble the governed delivery package.	Typed result; display; narrative; lineage; compliance decision; optional artifact state; warnings and errors	Response package to MCP Capability Layer; response and terminal evidence to ALS
Optional Rendering Service	Produce a static visual without joining analytical computation.	Self-contained display specification from a consumer	SVG or PNG to the consumer

The running example uses one request throughout. Each primary flow component shows both its input and output as technology-neutral JSON. When a downstream contract requires a request, definition, projection, plan, result, or evidence reference, it reuses the same field name and value so the handoff is visible. Identifiers and values are illustrative, not implementation defaults.

A result is repeatable only when the request, source snapshot, definition versions, effective permissions, configuration, and execution software are preserved. Deterministic planning does not compensate for changed data or definitions.

AI Consumers

Business definition. AI consumers are the channels through which people and automated processes request and use governed analysis.

Why this role exists. The architecture separates the user experience from analytical governance so that several interfaces can use the same approved definitions, permissions, controls, and evidence.

Input and output. A consumer starts with a business question or a known approved operation and the caller's authenticated context. It outputs a governed analytical request and later renders the structured response without recalculating or reinterpreting its values.

AI consumers include conversational assistants, autonomous agents, data-mining workflows, and custom applications. They provide the user experience; they do not own analytical definitions, permissions, query planning, or execution.

A consumer may submit natural language or an approved operation identifier with typed parameters. It passes the caller's authenticated context, preserves the returned result identifier for follow-up actions, and distinguishes governed results from exploratory output produced outside this platform.

Input and Output Example

The running example asks the platform to compare the caller's equity portfolios with their benchmarks for the current quarter:

```
{
  "input": {
    "business_request": "Compare my equity portfolios with their benchmarks this quarter"
  },
  "output": {
    "analytical_request_id": "request-draft-001",
    "question": "Compare my equity portfolios with their benchmarks this quarter"
  }
}
```

The authentication and identity context travels through the trusted transport. It is not embedded in the analytical request body.

The consumer request becomes the MCP Capability Layer input, beginning the governed pipeline.

MCP Capability Layer (MCP)

Governing principles: P1 - Semantic abstraction, P2 - Controls before execution, and P5 - Role-aware by default

Business definition. The MCP Capability Layer is the governed front door to the analytical service.

Why it exists. It gives every consumer one consistent contract and prevents conversational, automated, or custom interfaces from creating alternative paths around authentication and governance.

Input and output. It receives an authenticated natural-language or structured request. It outputs a correlated request to the correct pipeline stage and, after processing, a structured response to the consumer.

The MCP Capability Layer is the single governed analytical entry point. The transport authenticates the caller before tool routing and passes identity through trusted request context. Bearer tokens are not analytical tool arguments. A structured request bypasses only language interpretation: the supplied operation identifier is resolved to an approved SMR definition before the request reaches RAPL.

Tool Catalog

Tool	Purpose
run_analytics	Accepts either a natural-language question or an approved operation with typed parameters. Natural language is resolved by the IRA before governed execution begins.
list_operations	Lists operations and parameters available to the caller.
drilldown	Creates a governed follow-up request from a prior result and hierarchy selection.

A drilldown inherits analytical context but receives a new request identifier and passes the current entitlement and controls pipeline again. Prior approval never authorizes a later request automatically.

Execution Profiles

Profile	Result
data_retrieval	Returns a typed, paginated dataset governed by an approved dataset contract.
metric_query	Returns calculated data and a DVL display specification.
full_analytical	Adds an optional governed narrative to the metric response.

Execution profiles control optional response assembly, not governance. Every profile uses the complete entitlement, validation, controls, planning, execution, and lineage path. PAS invocation is compliance-driven and independent of presentation profile.

Intent Confirmation

When the IRA cannot select one operation confidently, the MCP layer returns ranked confirmation cards containing the proposed operation, bound parameters, material filters, and presentation preview. Refinement does not execute a backend query. The selected request still passes all downstream controls.

Confidence thresholds, candidate counts, and refinement limits require evaluation. They are not accuracy guarantees.

A confirmation card contains the candidate operation, resolved parameter values, time period, material filters, confidence, and a presentation preview. It must not expose physical schemas or executable queries. The session binds the candidate set to the authenticated caller and expires after a short configured interval.

The consumer may select a candidate, refine the question, or cancel. Selection resumes the original governed request; refinement produces a new ranked candidate set without querying a backend. Operations marked as requiring confirmation always present a card, regardless of confidence.

Capability Governance

The capability layer exposes only registered tools and approved operation metadata. Tool and catalog discovery apply a bounded DES policy view before IRA candidate retrieval. This view limits which metadata the caller may discover; it does not authorize execution. RAPL independently evaluates the complete request and current entitlement policy before validation or planning. Request limits, session limits, and allowed response sizes are versioned platform controls. Changes to tool contracts follow the same review and compatibility process as other public interfaces.

Input and Output Example

The MCP layer accepts the natural-language request and creates a correlation identifier before routing it to the IRA:

```
{
  "input": {
    "authenticated_context_ref": "caller-context",
    "analytical_request_id": "request-draft-001",
    "question": "Compare my equity portfolios with their benchmarks this quarter"
  },
  "output": {
    "request_id": "req-20260518-093241",
    "route": "natural_language",
    "analytical_request_id": "request-draft-001",
    "question": "Compare my equity portfolios with their benchmarks this quarter",
    "discoverable_catalog_ref": "approved-operations-for-caller"
  }
}
```

For natural-language requests, the correlated request becomes the IRA input. A structured request bypasses IRA, resolves its operation identifier against the SMR, and then proceeds to RAPL with the approved operation reference.

Intent Resolution Agent (IRA)

Governing principles: P1 - Semantic abstraction and P10 - Deterministic computation, not generation

Business definition. The IRA translates a user's business language into a registered analytical operation and its typed parameters.

Why it exists. Users should not need to know catalog identifiers, but the platform must not let a language model invent calculations or executable queries. The IRA bridges that gap and asks for confirmation when meaning is uncertain.

Input and output. It receives a natural-language request and the approved definitions discoverable by the caller. It outputs a resolved operation, parameters, confidence evidence, and purpose signal, or a clarification request.

The IRA is the only AI step before computation. It retrieves candidate operations from the SMR, ranks them against the user's request, binds parameters, and returns either a resolved request or a confirmation choice. It does not receive credentials, execute queries, or generate executable SQL.

The IRA uses registered descriptions rather than physical schemas. Caller-dependent phrases such as "my portfolios" remain unresolved until RAPL evaluates the authenticated identity. The output contains an approved operation identifier, typed parameters, candidate evidence, model details, and a confidence score.

The IRA also estimates whether the stated purpose is compliance-related. Ambiguous purpose requires clarification. The score is an input to a deterministic control; the model does not make the final control decision. Structured callers bypass the IRA and declare purpose through a validated field.

Resolution Pipeline

Stage	Required Behavior	Evidence Retained
Candidate retrieval	Search approved SMR operation and metric descriptions within the caller's discoverable scope.	Query embedding version, retrieved identifiers, and retrieval scores.
Candidate ranking	Rank candidates and bind parameters using the natural-language request and registered descriptions.	Model and prompt-template versions, ranked identifiers, confidence, and bound parameters.
Ambiguity check	Compare the leading candidates and apply operation-specific confirmation rules.	Thresholds, score difference, and confirmation decision.
Purpose classification	Estimate whether the stated purpose is compliance-related.	Purpose score and any clarification outcome.
Output validation	Validate the selected operation identifier and typed parameters against the SMR contract.	Validation result and rejection reason when applicable.

Retrieval narrows the model's choice set; it does not approve a definition or establish that the chosen intent is correct. Evaluation must measure top-candidate accuracy, correct-candidate coverage, clarification quality, and false certainty across representative language and user roles.

Input and Output Example

The IRA resolves the question to an approved comparison operation. The symbolic caller-dependent scope remains unresolved:

```
{
  "input": {
    "request_id": "req-20260518-093241",
    "question": "Compare my equity portfolios with their benchmarks this quarter",
    "discoverable_catalog_ref": "approved-operations-for-caller"
  },
  "output": {
    "resolved_request_id": "resolved-20260518-093242",
    "request_id": "req-20260518-093241",
    "operation_id": "compare_portfolio_to_benchmark",
    "parameters": {
      "portfolio_scope": "caller_authorized_portfolios",
      "asset_class": "EQUITY",
      "time_period": "current_quarter"
    },
    "confidence": 0.93,
    "compliance_purpose_score": 0.08,
    "confirmation_required": false
  }
}
```

The IRA uses the SMR to limit interpretation to approved analytical choices. Its resolved operation then becomes part of the RAPL input.

Semantic Metrics Repository (SMR)

Governing principles: P1 - Semantic abstraction, P3 - Deterministic metric resolution, and P10 - Deterministic computation, not generation

Business definition. The SMR is the organization's governed rulebook for analytical meaning: what a metric, dimension, operation, or dataset means and which approved version applies.

Why it exists. Without a shared catalog, each report, application, or AI interaction can reconstruct the same business concept differently. The SMR makes approved definitions reusable and versioned.

Input and output. Authors submit proposed definitions through governance. At runtime, the IRA, SVL, and PQP submit approved identifiers or discovery queries. The SMR outputs discoverable operation metadata, pinned semantic definitions, and approved mapping references.

In a governed AI-enabled analytics situation, the Semantic Metrics Repository (SMR) provides a catalog of approved metrics, dimensions, operations, and dataset contracts. It is the authoritative source for concepts that the governed path can resolve.

Definition Type	Required Content
Metric	Formula or governed measure, aggregation, units, dimensions, classification, source affinity, owner, version, and approval state.
Dimension	Business meaning, hierarchy, compatible metrics, and mapping information.
Operation	Parameter contract, supported metrics and dimensions, execution profile, and confirmation requirements.
Dataset	Approved fields, classifications, selection rules, mapping, refresh expectations, and pagination policy.

Only versions in the Approved state are normally resolvable for new governed requests. A Deprecated version may remain temporarily resolvable only when an explicit transition policy permits it. Historical versions remain available to authorized reviewers for lineage reconstruction.

Definition Lifecycle and Versioning

State	Meaning
Proposed	An author has submitted a draft that cannot be used for governed execution.
In Review	Named reviewers are assessing meaning, sources, tests, ownership, and access implications.
Approved	The exact version may be resolved by governed requests.
Deprecated	Existing evidence remains interpretable, but new use is discouraged or blocked according to policy.
Retired	The version is unavailable for new execution and retained only for authorized historical reconstruction.

Approving a new version does not rewrite earlier lineage. Each request pins the versions it resolved. Compatibility rules determine whether consumers must change when parameters, dimensions, units, formulas, mappings, or classifications change.

Minimum Definition Contract

Every definition carries a stable identifier, semantic version, owner, approval state, effective period, classification, and change history. Metrics additionally declare their formula or governed measure, aggregation behavior, units, compatible dimensions, source affinity, physical mapping reference, and tests. Operations declare typed parameters, output expectations, execution profile, permitted drilldowns, and whether confirmation is mandatory. Dataset contracts declare approved fields, classifications, selection rules, ordering, pagination, and freshness expectations.

The SMR and Semantic Data Repository (SDR) are separate stores within the Data Context Store (DCS). The SMR defines analytical meaning; the SDR describes organizational data and its physical structure. Approved mappings link the two without exposing physical schema details through the normal AI interface.

Formula Language

The formula language expresses metric logic using registered metric identifiers, logical fields, arithmetic, conditions, safe division, time windows, filtered aggregation, and other explicitly supported operations. It does not accept arbitrary SQL or backend-specific expressions.

A formula is human-readable and audit-visible, but registration does not prove correctness. Approval requires accountable ownership, test cases, source validation, and comparison with independently verified results.

Authoring and Discovery

Metrics Modellers submit definitions through an authenticated workflow. Analytics Governance reviews and approves them. Consumers discover only approved operations available under their entitlements. Search indexes and embeddings may improve discovery; they do not change approval state or access rights.

Input and Output Example

The resolved operation references two approved metrics without exposing their physical implementation:

```

{
  "input": {
    "operation_id": "compare_portfolio_to_benchmark",
    "required_state": "approved"
  },
  "output": {
    "operation_definition_ref": "smr:operation:compare_portfolio_to_benchmark@1.3.0",
    "operation_id": "compare_portfolio_to_benchmark",
    "version": "1.3.0",
    "status": "approved",
    "metrics": [
      {
        "metric_id": "portfolio_return",
        "version": "2.1.0"
      },
      {
        "metric_id": "benchmark_return",
        "version": "1.4.2"
      }
    ],
    "dimensions": [
      "portfolio"
    ],
    "required_parameters": [
      "portfolio_scope",
      "asset_class",
      "time_period"
    ],
    "intent_pattern": "comparison",
    "presentation_metadata_ref": "smr:presentation:portfolio-benchmark-comparison@2.0",
    "execution_profile": "full_analytical"
  }
}

```

The approved operation definition and resolved request now move to RAPL, where the caller's permitted analytical scope is determined.

Role-Aware Projection Layer (RAPL)

Governing principle: P5 - Role-aware by default

Business definition. RAPL converts organizational access policy into the exact analytical scope permitted for the current caller and request.

Why it exists. Permission to reach a data service does not explain which business metrics, dimensions, populations, or fields a person may use. RAPL makes those business-level restrictions explicit before planning begins.

Input and output. It receives the resolved request, authenticated identity context, and versioned DES policies. It outputs an entitlement projection containing approved concepts, row scope, masks, classification ceiling, and decision evidence.

RAPL converts authenticated identity claims and policies from the Data Entitlements Store (DES) into an effective entitlement projection. It validates identity context, retrieves role definitions, merges them according to policy, resolves claim-based row scopes, and records the decision.

Restriction	Proposed Treatment
Data and metric access	Deny any required domain, classification, or metric outside the effective access set.
Dimension access	Reject a required unauthorized dimension; do not silently change the question.
Row scope	Resolve typed predicates from trusted claims and attach them to the plan.
Column masking	Carry each required mask and its policy basis into planning and execution.
Classification ceiling	Preserve the effective ceiling for validation and controls.

The proposed multi-role rule uses union for granted data, metrics, and dimensions; strict intersection for row scopes; and union for masks, so any applicable mask remains in force. Explicit-deny and missing-claim behavior must be defined consistently in DES policy and tested independently of role order.

Row scopes are typed predicates, not interpolated query fragments. Masks are enforced before protected values leave the controlled execution boundary. Each projection records the roles, policy versions, resolved scopes, masks, and decision basis in the ALS.

Multi-Role Merge and Deny Behavior

Entitlement	Proposed Merge Rule
Data domains, metrics, and dimensions	Union of grants, subject to explicit-deny policy and classification ceiling.
Row scopes	Strict intersection. Every applicable population restriction remains in force.
Column masks	Union. A mask required by any applicable role remains active.
Classification ceiling	Most restrictive applicable ceiling unless policy explicitly defines another reviewed rule.

The merge must be independent of role order and must fail closed when a required role definition or claim cannot be resolved. The design decision about whether an explicit deny overrides every grant must be uniform across DES, RAPL, tests, and audit explanations.

Column Masking Modes

Mode	Result Treatment
null replacement	Replace the protected value with null while preserving the field.
redacted label	Replace the value with a fixed approved label.
excluded	Remove the field from the returned schema.
hash replacement	Replace the value with a deterministic one-way representation when approved grouping behavior is required.

Masking changes what downstream consumers can observe. The selected mode, protected field, policy version, and enforcement location therefore form part of the analytical evidence.

Input and Output Example

RAPL resolves the symbolic portfolio scope from the authenticated caller's policy and claims:

```

{
  "input": {
    "resolved_request_id": "resolved-20260518-093242",
    "request_id": "req-20260518-093241",
    "operation_definition_ref": "smr:operation:compare_portfolio_to_benchmark@1.3.0",
    "authenticated_context_ref": "caller-context",
    "entitlement_policy_ref": "current-policy-snapshot"
  },
  "output": {
    "entitlement_projection_ref": "projection:req-20260518-093241",
    "request_id": "req-20260518-093241",
    "policy_version": "portfolio-access-7.2",
    "approved_metrics": [
      "portfolio_return",
      "benchmark_return"
    ],
    "approved_dimensions": [
      "portfolio"
    ],
    "row_scope": {
      "field": "portfolio_id",
      "operator": "in",
      "values": [
        "GLOB_EQ_OPP",
        "UK_CORE_INC",
        "ASIA_PAC_GRW",
        "EUR_BAL_INC"
      ]
    },
    "column_masks": [],
    "classification_ceiling": "INTERNAL",
    "decision": "approved"
  }
}

```

The entitlement projection becomes an SVL input alongside the resolved request and pinned SMR definitions.

Semantic Validation Layer (SVL)

Governing principles: P2 - Controls before execution and P10 - Deterministic computation, not generation

Business definition. The SVL is the semantic quality gate that turns an entitled business request into a precise, technology-neutral calculation plan.

Why it exists. A recognized intent can still be incomplete, incompatible, unapproved, or inconsistent with the caller's permissions. The SVL stops those requests before physical planning or source access.

Input and output. It receives the resolved request, entitlement projection, and pinned SMR definitions. It outputs a backend-independent Logical Query Plan or a structured rejection.

The SVL validates a fully qualified request and compiles it into a platform-independent LQP. No AI model runs in this component.

The SVL validates structure and parameter types; resolves operations, metrics, dimensions, and datasets to approved versions; checks semantic compatibility; applies the RAPL projection; attaches validated compliance inputs; and creates a directed plan containing logical concepts rather than SQL, connection details, or physical identifiers.

Unknown, unapproved, incompatible, or unauthorized required concepts cause a structured rejection. The SVL does not return a partial calculation that changes the requested meaning. Its validation shows conformity to registered definitions; it does not establish that the definitions or source data are correct.

Validation Stages

Stage	Checks	Output
Request validation	Required fields, types, parameter ranges, and operation contract.	A well-formed request or structured rejection.
Semantic resolution	Approval state, pinned versions, metric-dimension compatibility, and dataset contract.	Fully resolved logical identifiers.
Entitlement application	Permitted concepts, row predicates, masks, and classification ceiling.	An entitlement-constrained request.
Plan generation	Logical scans, filters, calculations, joins, sorts, limits, and output shape.	A backend-independent LQP.

The LQP also carries definition versions, data-affinity hints, mask directives, compliance inputs, and a preliminary impact indicator. It contains no credentials, connection endpoints, physical table names, or executable SQL.

Input and Output Example

The SVL converts the qualified request into a backend-independent plan:

```

{
  "input": {
    "resolved_request_id": "resolved-20260518-093242",
    "operation_definition_ref": "smr:operation:compare_portfolio_to_benchmark@1.3.0",
    "entitlement_projection_ref": "projection:req-20260518-093241"
  },
  "output": {
    "lqp_id": "lqp-20260518-093243",
    "request_id": "req-20260518-093241",
    "operation_id": "compare_portfolio_to_benchmark",
    "execution_profile": "full_analytical",
    "intent_pattern": "comparison",
    "presentation_metadata_ref": "smr:presentation:portfolio-benchmark-comparison@2.0",
    "entitlement_projection_ref": "projection:req-20260518-093241",
    "metric_versions": {
      "portfolio_return": "2.1.0",
      "benchmark_return": "1.4.2"
    }
  },
  "steps": [
    {
      "type": "metric_scan",
      "metrics": [
        "portfolio_return",
        "benchmark_return"
      ]
    },
    {
      "type": "filter",
      "field": "asset_class",
      "operator": "eq",
      "value": "EQUITY"
    },
    {
      "type": "row_scope",
      "field": "portfolio_id",
      "operator": "in",
      "value_ref": "projection:req-20260518-093241"
    },
    {
      "type": "sort",
      "field": "portfolio_return",
      "direction": "descending"
    }
  ],
  "time_period": {
    "type": "relative",
    "value": "current_quarter"
  },
  "compliance_inputs": {
    "metric_signal": false,
    "purpose_score": 0.08
  },
  "column_masks": []
}

```

The completed Logical Query Plan becomes the SCL input for the mandatory release decision.

Semantic Controls Layer (SCL)

Governing principles: P2 - Controls before execution, P8 - Explainability at every layer, and P9 - Administrator sovereignty within governance bounds

Business definition. The SCL is the operational and governance release gate for an otherwise valid analytical plan.

Why it exists. Entitlement and semantic validity do not address excessive scale, unsafe complexity, operating capacity, classification boundaries, or compliance-purpose handling. The SCL decides whether the plan may proceed under current policy and conditions.

Input and output. It receives the Logical Query Plan, versioned control policy, profiling evidence, and operating state. It outputs an approved plan with an execution budget or a structured rejection, together with a recorded decision.

The SCL is the mandatory release gate between logical validation and physical planning. It evaluates every LQP against versioned controls and records the decision before execution.

Control	Decision
Data scale	Compare an evidence-based scan estimate with the configured limit.
Complexity	Check plan nodes, join depth, source count, and other bounded measures.
Classification	Compare the highest required classification with the permitted ceiling.
Compliance	Evaluate approved metric metadata and declared or classified purpose.
Concurrency	Check active work against the operating limit.
Timeout	Assign a bounded execution budget after blocking checks pass.

Estimates are not exact measurements. The record identifies the source and freshness of the statistics used. Rejections return structured reasons and safe ways to narrow the request.

The proposed compliance trigger requires both a compliance-relevant metric and a compliance purpose. For natural-language requests, the SCL evaluates the IRA score against a configured threshold; structured requests declare purpose explicitly. The SCL records both signals. When both are active, it marks the provenance artifact as required and blocks export until PAS seals the completed evidence set after execution. Whether this rule is sufficient for a jurisdiction or activity remains a deployment-specific compliance decision.

Control Decision Contract

Each control records its configured limit, observed or estimated value, data source, evaluation version, outcome, and reason. The SCL evaluates all controls needed for a useful explanation, even when one failure is sufficient to block execution. Sensitive platform state, such as other tenants' workload details, is not returned to the caller.

The data-scale estimate uses current SDR profiling statistics and the resolved row scope. The complexity control considers structural cost separately from volume. The classification control applies both platform boundaries and the caller's effective ceiling. The concurrency control protects shared execution resources without weakening tenant isolation.

After all blocking controls pass, the SCL assigns a timeout and writes the signed or integrity-protected decision record before releasing the plan.

No interactive user, automated agent, administrator, cache path, or internal retry bypasses this release decision. A retrieved or modified request receives its own decision record.

Input and Output Example

The SCL records each decision and assigns the execution budget:

```
{
  "input": {
    "lqp_id": "lqp-20260518-093243",
    "controls_policy_ref": "current-controls",
    "profiling_evidence_ref": "sdr-profile:portfolio-performance-source@2026-05-18T09:30:00Z",
    "operating_state_ref": "current-capacity"
  },
  "output": {
    "controls_decision_ref": "controls:lqp-20260518-093243",
    "compliance_decision_ref": "compliance-decision:req-20260518-093241",
    "lqp_id": "lqp-20260518-093243",
    "decision": "approved",
    "checks": {
      "data_scale": {
        "estimate": 412000,
        "limit": 50000000,
        "result": "pass"
      },
      "complexity": {
        "score": 4,
        "limit": 50,
        "result": "pass"
      },
      "classification": {
        "required": "INTERNAL",
        "ceiling": "INTERNAL",
        "result": "pass"
      },
      "compliance": {
        "metric_signal": false,
        "purpose_signal": false,
        "result": "standard"
      },
      "concurrency": {
        "active": 3,
        "limit": 20,
        "result": "pass"
      }
    },
    "timeout_seconds": 30
  }
}
```

An approved controls decision releases the Logical Query Plan and execution budget to the PQP.

Physical Query Planner (PQP)

Governing principles: P1 - Semantic abstraction and P10 - Deterministic computation, not generation

Business definition. The PQP translates an approved business calculation into bounded instructions that registered data sources can execute.

Why it exists. Business definitions should remain stable when physical schemas or source technologies change. The PQP contains that translation boundary and keeps physical details away from AI consumers and language models.

Input and output. It receives the approved Logical Query Plan, execution budget, pinned mappings, and source capabilities. It outputs a physical execution envelope containing source sub-plans, protection directives, and integrity evidence.

The PQP is the boundary between logical concepts and physical execution. It resolves the approved physical mapping associated with each pinned definition version, expands logical time expressions, groups work by source affinity, and compiles source-specific sub-plans.

The PQP applies row predicates and masking requirements to each relevant sub-plan. Where a source can enforce masking, the generated query performs it before values leave that source. Otherwise, an approved controlled execution stage applies the mask before returning data outside the FQE boundary.

A repeatable physical plan requires the same LQP, definition and mapping versions, planner version, configuration, and source capabilities. The PQP records those dependencies. It does not hold source credentials or execute plans.

Compilation Stages

Stage	Responsibility
Mapping resolution	Resolve each logical scan and measure against the mapping pinned to the approved definition version.
Predicate binding	Bind typed dimension filters and RAPL row scopes using backend-safe parameters.
Time resolution	Expand relative periods against the recorded evaluation timestamp and calendar.
Source grouping	Group compatible work by data affinity and identify required cross-source assembly.
Source instruction compilation	Produce a bounded sub-plan in the registered instruction format for each source group.
Plan sealing	Create an integrity digest and record planner, instruction-format, mapping, and configuration versions.

An implementation may delegate federation to one execution service or create several source sub-plans with an explicit assembly contract. Both approaches implement the same logical responsibility; Appendix B proposes a specific choice.

Input and Output Example

The PQP resolves the approved mappings and creates a technology-neutral execution envelope:

```

{
  "input": {
    "lqp_id": "lqp-20260518-093243",
    "controls_decision_ref": "controls:lqp-20260518-093243",
    "mapping_set_ref": "approved-mappings",
    "source_capabilities_ref": "registered-source-capabilities"
  },
  "output": {
    "plan_id": "plan-20260518-093244",
    "lqp_id": "lqp-20260518-093243",
    "controls_decision_ref": "controls:lqp-20260518-093243",
    "entitlement_projection_ref": "projection:req-20260518-093241",
    "mapping_versions": [
      "portfolio-mapping-4.6"
    ],
    "plan_digest": "digest:<illustrative-plan-value>",
    "sub_plans": [
      {
        "source_id": "portfolio-performance-source",
        "metrics": [
          "portfolio_return",
          "benchmark_return"
        ],
        "filters": [
          "asset_class",
          "authorized_portfolios",
          "resolved_date_range"
        ]
      }
    ],
    "resolved_date_range": {
      "from": "2026-04-01",
      "to": "2026-05-18"
    },
    "column_masks": [],
    "timeout_seconds": 30
  }
}

```

The physical execution envelope becomes the FQE input; no upstream component executes against a data source.

Federated Query Engine (FQE)

Governing principles: P1 - Semantic abstraction, P4 - Complete analytical lineage, and P10 - Deterministic computation, not generation

Business definition. The FQE is the controlled execution boundary that runs approved source instructions and assembles their results.

Why it exists. Source credentials, timeouts, federation, caching, masking, and failure handling require one accountable boundary. Concentrating them here prevents upstream components and AI consumers from gaining direct backend access.

Input and output. It receives an approved physical execution envelope. It outputs a typed result or an explicit terminal outcome, plus execution evidence for the ALS.

The FQE is the only logical component with access to registered backend connections. It validates source availability, executes approved sub-plans within the assigned timeout, joins compatible results on governed dimensions, enforces any remaining masks within its controlled boundary, and records execution outcomes.

Caching uses a canonical key that includes the effective plan, tenant, entitlement projection, definition versions, and relevant configuration. A cache hit still passes current authorization and produces a new lineage event. Compliance-purpose queries bypass the cache unless approved policy provides equivalent evidence and freshness guarantees.

Source failure is visible. The response identifies incomplete sources and affected metrics. A partial result may be returned only when the approved operation contract permits partial semantics; otherwise, the request fails rather than representing missing values as a complete answer.

The FQE records source identifiers, plan hashes, timing, row counts, cache status, errors, and mask application. It does not expose credentials or unrestricted physical schema details to consumers or language models.

Execution Pipeline

Stage	Required Behavior
Plan admission	Verify plan integrity, source registration, timeout, tenant, and control approval.
Cache evaluation	Check an entitlement-safe canonical key and current policy before reuse.
Source execution	Run sub-plans within their time and resource budgets.
Assembly	Join or combine results only on approved compatible dimensions.
Final protection	Apply any remaining masks and verify the returned schema before release.
Evidence write	Record the outcome before returning the result or terminal failure.

Supported source categories may include relational analytical stores, governed semantic services, approved data APIs, relationship stores, and multidimensional analytical engines. Technical connectivity does not make a source usable

automatically. Each connection requires approved mappings, credentials, classifications, timeout behavior, and failure semantics.

Result-size limits determine whether the engine returns inline data, paginates an approved dataset, streams within a controlled protocol, or rejects the request. These choices are part of the operation contract and response evidence.

Input and Output Example

The FQE executes the approved plan and returns typed result rows:

```

{
  "input": {
    "plan_id": "plan-20260518-093244",
    "controls_decision_ref": "controls:lqp-20260518-093243"
  },
  "output": {
    "result_id": "res-20260518-093247",
    "data_ref": "result:res-20260518-093247",
    "request_id": "req-20260518-093241",
    "plan_id": "plan-20260518-093244",
    "status": "complete",
    "execution_profile": "full_analytical",
    "sources_used": [
      "portfolio-performance-source"
    ],
    "cache_status": "miss",
    "schema": [
      {
        "field": "portfolio",
        "type": "string",
        "label": "Portfolio"
      },
      {
        "field": "portfolio_return",
        "type": "decimal",
        "label": "Portfolio Return",
        "unit": "ratio"
      },
      {
        "field": "benchmark_return",
        "type": "decimal",
        "label": "Benchmark Return",
        "unit": "ratio"
      }
    ],
    "rows": [
      {
        "portfolio_id": "GLOB_EQ_OPP",
        "portfolio": "Global Equity Opportunities",
        "portfolio_return": 0.0421,
        "benchmark_return": 0.0385
      },
      {
        "portfolio_id": "ASIA_PAC_GRW",
        "portfolio": "Asia Pacific Growth",
        "portfolio_return": 0.0367,
        "benchmark_return": 0.039
      },
      {
        "portfolio_id": "UK_CORE_INC",
        "portfolio": "UK Core Income",
        "portfolio_return": 0.0287,
        "benchmark_return": 0.0254
      },
      {
        "portfolio_id": "EUR_BAL_INC",
        "portfolio": "EUR Balanced Income",
        "portfolio_return": 0.0193,
        "benchmark_return": 0.0231
      }
    ]
  }
}

```

```
]
}
}
```

The typed result now branches to presentation, response assembly, and evidence recording. DVL determines how consumers should display it.

Data Visualization Language (DVL)

Governing principle: P7 - Deterministic visualization

Business definition. The DVL turns a governed result into a consistent chart or table contract that consumers can render.

Why it exists. Presentation choices can change interpretation. A governed display contract prevents each consumer or language model from independently choosing labels, units, ordering, thresholds, and chart form.

Input and output. It receives the approved intent, typed result schema, values, and presentation metadata. It outputs a versioned display specification or the governed table fallback.

The DVL produces a structured chart or table specification from the approved intent and result schema. A registry maps comparison, trend, distribution, threshold, attribution, relationship, and composition patterns to compatible presentation contracts. A governed table is the fallback.

Contract selection is deterministic for the same intent, result schema, registry version, and configuration. Labels and units come from approved definitions, not physical fields. An authorized analyst may request an allowed compatible override; the platform rejects incompatible choices and records accepted overrides.

The DVL returns a specification, not a rendered image. Consumer applications render it directly or pass it to an independently governed rendering service.

Intent Pattern Taxonomy

Pattern	Purpose	Typical Contract
Comparison	Compare measures across discrete categories.	Grouped bar chart or governed table.
Trend	Show change over an ordered time dimension.	Line chart.
Distribution	Show spread, frequency, or concentration.	Histogram or governed table.
Threshold	Compare observations with a limit or benchmark.	Threshold matrix, bar chart, or table.
Attribution	Decompose a total into contributing factors.	Waterfall chart.
Relationship	Show association between two measures.	Scatter plot.
Composition	Show part-to-whole structure.	Treemap or governed table.

Chart Contract Selection

Each registered contract declares compatible patterns, required field types, cardinality limits, axis or column bindings, sorting, interaction behavior, accessibility requirements, and fallback behavior. The selector evaluates the most specific compatible contract first. If none matches, it returns the governed table contract rather than asking a language model to invent a display.

Themes may change approved colors and typography without changing the analytical contract. Threshold colors, units, labels, ordering, and benchmark semantics come from governed metadata and must remain interpretable in accessible and monochrome presentations.

Input and Output Example

The DVL selects the registered comparison contract from the intent and result shape:

```
{
  "input": {
    "result_id": "res-20260518-093247",
    "data_ref": "result:res-20260518-093247",
    "intent_pattern": "comparison",
    "presentation_metadata_ref": "smr:presentation:portfolio-benchmark-comparison@2.0"
  },
  "output": {
    "display_spec_ref": "display:res-20260518-093247",
    "result_id": "res-20260518-093247",
    "type": "chart",
    "contract": "multi_series_comparison",
    "contract_version": "2.0",
    "mark": "bar",
    "category": {
      "field": "portfolio",
      "label": "Portfolio"
    },
    "measures": [
      {
        "field": "portfolio_return",
        "label": "Portfolio Return",
        "format": "percentage"
      },
      {
        "field": "benchmark_return",
        "label": "Benchmark Return",
        "format": "percentage"
      }
    ],
    "sort": {
      "field": "portfolio_return",
      "direction": "descending"
    }
  }
}
```

In parallel with DVL, the NSA receives the bounded result content needed for an optional narrative.

Narrative Synthesis Agent (NSA)

Governing principles: P6 - Governed narrative and P10 - Deterministic computation, not generation

Business definition. The NSA provides an optional plain-language explanation of a completed result.

Why it exists. A concise explanation makes analysis easier to consume, but narrative generation must remain separate from calculation and must not introduce unsupported values or conclusions.

Input and output. It receives only approved result labels, values, units, dimensions, and permitted derivations. It outputs a validated narrative or an explicit omission state.

The NSA optionally produces a concise plain-language summary after computation. Its input is limited to approved result labels, values, units, and dimensions. It does not calculate metrics or change the result.

A validator checks every stated number against the returned data or an independently recomputed permitted derivation. If validation fails, the service may retry once and then omits the narrative while recording the failure. Quantitative matching reduces unsupported claims but does not prove that wording, emphasis, or causal interpretation is correct. Evaluation therefore includes semantic review, not numeric matching alone.

Disabling the NSA has no effect on calculation, display specification, lineage, or compliance evidence.

Anchoring and Validation

The narrative contract separates a short lead statement, supporting detail, and the result identifiers to which each claim is anchored. Validation covers numbers, units, dimension labels, comparison direction, and permitted derived counts. It rejects invented causes, recommendations, forecasts, or significance claims unless the approved operation explicitly provides those outputs.

The lineage record captures the model, prompt-template version, validation outcome, retry count, and omission reason. Model selection may vary by result complexity, but changing models must not alter computed data or the DVL contract.

Input and Output Example

The NSA summarizes only statements supported by the result rows:

```

{
  "input": {
    "result_id": "res-20260518-093247",
    "data_ref": "result:res-20260518-093247",
    "allowed_content": [
      "approved labels",
      "result values",
      "units",
      "permitted derived counts"
    ]
  },
  "output": {
    "narrative_ref": "narrative:res-20260518-093247",
    "result_id": "res-20260518-093247",
    "narrative": {
      "lead": "Two of four equity portfolios outperformed their benchmarks this quarter.",
      "detail": "Global Equity Opportunities returned 4.21% compared with 3.85% for its benchmark. UK Core Income returned 2.87% compared with 2.54%.",
      "anchored_to": [
        "GLOB_EQ_OPP",
        "UK_CORE_INC",
        "ASIA_PAC_GRW",
        "EUR_BAL_INC"
      ]
    },
    "validation": {
      "numbers": "passed",
      "units": "passed",
      "dimension_labels": "passed"
    }
  }
}

```

DVL and NSA write their presentation evidence to the ALS, where it joins the request, decision, plan, and execution events already recorded for the request.

Analytical Lineage Store (ALS)

Governing principles: P4 - Complete analytical lineage and P8 - Explainability at every layer

Business definition. The ALS is the evidence ledger for how each governed request was interpreted, authorized, controlled, planned, executed, and presented.

Why it exists. A result cannot be traced or assessed for repeatability if its definitions, permissions, decisions, source references, and terminal outcome are scattered or overwritten.

Input and output. It receives correlated events from each stage. It outputs an authorized lineage chain, searchable references, integrity evidence, and the event set used by PAS and audit export.

The ALS preserves correlated events for every accepted request, including entitlement decisions, controls decisions, execution, presentation, and terminal failure. Records use a common request identifier so an authorized reviewer can reconstruct the sequence without treating one final payload as the complete history.

By default, lineage stores hashes, definition and policy versions, bounded summaries, source references, plan identifiers, decisions, and timing rather than full raw requests or results. An approved policy may require additional content. The design minimizes sensitive data while preserving evidence for its intended review.

Claims of immutability depend on storage enforcement. Write-once retention, versioning, signatures, encryption, tenant-scoped authorization, key management, reconciliation, and tested recovery provide the required integrity properties. Application convention alone is insufficient. Corrections create linked amendment records rather than altering prior evidence.

Retention varies by record class, jurisdiction, and business purpose. Definition versions and references needed to interpret retained results remain available for at least the corresponding evidence period.

Stored Event Types

Event	Minimum Evidence
Request accepted	Tenant, caller reference, tool, operation, parameters or bounded request representation, and timestamp.
Intent resolved	Candidate set, selected operation, model evidence, confidence, and confirmation history.
Entitlement projected	Policy versions, active roles, resolved row scopes, masks, ceiling, and decision.
Semantic request validated	Pinned definitions, entitlement projection, compliance inputs, and logical plan digest.
Controls evaluated	Each control input, limit, outcome, reason, and assigned timeout.
Plan compiled	Logical and physical plan digests plus definition, mapping, planner, and instruction-format versions.
Execution completed	Sources, timings, row counts, cache status, failures, and result digest.
Presentation prepared	DVL contract, narrative validation, and any omission reason.
Provenance artifact sealed	Artifact identifier, schema version, verification metadata, and export state when PAS is required.

Event	Minimum Evidence
Response assembled	Response status, included references, warnings, errors, and delivery outcome.
Request terminated	Rejection, cancellation, timeout, or failure stage and structured reason.

Isolation, Retention, and Audit Export

Every index entry and object key is tenant-scoped. Authorization applies to both search metadata and stored objects. Audit export is a separately authorized capability; possession of a result identifier does not grant access to its evidence.

Retention policy defines periods for request events, controls decisions, execution evidence, result artifacts, blocked requests, and definition versions. A shorter result-data period may be appropriate, but retained evidence must state when source data or result payloads are no longer available for exact reconstruction. Legal hold, deletion, amendment, and key-rotation behavior require explicit design and testing.

An audit export includes the selected event chain, referenced definition and policy versions, verification material, export scope, and exporter's identity. The platform signs the export package and records the export as a new auditable event.

Input and Output Example

The ALS correlates stage records without storing the full result by default:

```

{
  "input": {
    "request_id": "req-20260518-093241",
    "resolved_request_id": "resolved-20260518-093242",
    "entitlement_projection_ref": "projection:req-20260518-093241",
    "lqp_id": "lqp-20260518-093243",
    "controls_decision_ref": "controls:lqp-20260518-093243",
    "compliance_decision_ref": "compliance-decision:req-20260518-093241",
    "plan_id": "plan-20260518-093244",
    "result_id": "res-20260518-093247",
    "display_spec_ref": "display:res-20260518-093247",
    "narrative_ref": "narrative:res-20260518-093247"
  },
  "output": {
    "lineage_ref": "lineage:req-20260518-093241",
    "request_id": "req-20260518-093241",
    "result_id": "res-20260518-093247",
    "events": [
      {
        "type": "request_accepted",
        "record_id": "evt-request-001"
      },
      {
        "type": "intent_resolved",
        "record_id": "evt-intent-001"
      },
      {
        "type": "entitlement_projected",
        "record_id": "evt-access-001"
      },
      {
        "type": "semantic_validated",
        "record_id": "evt-validation-001"
      },
      {
        "type": "controls_evaluated",
        "record_id": "evt-controls-001"
      },
      {
        "type": "plan_compiled",
        "record_id": "evt-plan-001"
      },
      {
        "type": "execution_completed",
        "record_id": "evt-execution-001"
      },
      {
        "type": "presentation_prepared",
        "record_id": "evt-presentation-001"
      }
    ],
    "result_digest": "digest:<illustrative-value>",
    "retention_policy": "analytical-evidence-standard"
  }
}

```

When both compliance signals are active, the correlated ALS event set becomes the PAS input.

Provenance Artifact Service (PAS)

Governing principles: P2 - Controls before execution, P4 - Complete analytical lineage, and P9 - Administrator sovereignty within governance bounds

Business definition. PAS packages the evidence required for a compliance-purpose result into a sealed, independently verifiable artifact.

Why it exists. Standard lineage supports operational review, while regulated uses may require a fixed schema, named framework references, a digital signature, and an export gate.

Input and output. It receives the active compliance trigger and the correlated ALS event set. It outputs an `artifact_state_ref` for a sealed artifact with verification metadata and an explicit export state, or a failure state that keeps export blocked.

PAS assembles additional evidence for a request whose two compliance signals are active. It reads the relevant ALS events, adds the metric, framework, intent, policy, plan, execution, and result references required by the approved artifact schema, and signs the canonical artifact bytes.

The platform enforces the export gate. A compliance-purpose result cannot be exported until PAS confirms that the artifact is complete and sealed. The response identifies the artifact, schema version, signing key identifier, algorithm, and verification status.

A valid signature shows that the signed bytes have not changed since sealing. It does not prove that the calculation is correct or that the artifact satisfies a legal requirement. The organization validates the schema, retention, signing controls, and approval process for each intended framework.

Compliance Artifact Contract

Field	Purpose
compliance purpose and intent score	Records the purpose input and threshold decision.
triggered metrics and frameworks	Identifies the approved metadata that activated the additional evidence path.
regulatory trace identifier	Correlates the artifact with the retained event chain.
artifact schema version	Identifies the structure used by reviewers and verification software.
definition, policy, plan, and result references	Binds the artifact to the governed computation.
signing key identifier, algorithm, and signature	Supports independent integrity verification without exposing signing-key material.
export status	Shows whether sealing completed and export is permitted.

If assembly or signing fails, the result remains non-exportable and the ALS records the terminal artifact state. Amendments create a new signed artifact that references the original; they do not replace sealed evidence.

Running Example: No PAS Invocation

The portfolio comparison reaches the compliance decision with both signals inactive. PAS is not invoked. The orchestration path records that routing outcome, and response assembly uses the SCL decision to produce the explicit standard compliance state:

```
{
  "input": {
    "request_id": "req-20260518-093241",
    "compliance_decision_ref": "compliance-decision:req-20260518-093241",
    "compliance_trigger": {
      "metric_signal": false,
      "purpose_signal": false
    }
  },
  "output": {
    "request_id": "req-20260518-093241",
    "pas_invoked": false,
    "artifact_required": false,
    "artifact_id": null,
    "compliance_route": "standard",
    "reason": "two_signal_trigger_not_active"
  }
}
```

The result, presentation outputs, lineage reference, compliance decision, and any sealed artifact state now become the response-assembly input.

Structured Response

Business definition. The structured response is the governed delivery contract between the analytical platform and its consumers.

Why it exists. Consumers need one unambiguous way to distinguish data, presentation, narrative, evidence, warnings, compliance state, and terminal status without reconstructing analytical meaning.

Input and output. Response assembly receives the typed result or terminal outcome plus presentation and evidence references. It outputs one structured envelope that the consumer can render, retain, or pass to an authorized follow-up action.

The platform is headless. It returns a structured response that lets different consumers present the same governed result without reinterpreting its meaning.

Response Element	Purpose
request_id and result_id	Correlate the response, follow-up actions, and evidence.
status	Distinguishes complete, incomplete, rejected, failed, and timed-out outcomes.
data	Contains typed, paginated rows or an approved dataset extract.
display_spec	Contains the DVL chart or table contract when required.
narrative	Contains the optional validated NSA summary.
lineage	References the authorized analytical evidence.
compliance	Reports artifact and export-gate state when applicable.
warnings and errors	Identify limitations, affected sources, and remediation.

DVL Specification Format

A DVL specification is a discriminated chart or table envelope. It contains approved labels, units, formatting hints, encodings or columns, interaction rules, and the contract version. It may contain result data or a controlled reference, depending on response size and consumer capability. It never substitutes physical field names for governed labels.

Response Status and Failure Semantics

Complete, incomplete, rejected, failed, and timed-out are distinct states. An incomplete response identifies missing sources or metrics and appears only when the operation contract permits partial semantics. A rejection means a governance or validation rule prevented execution. A failure or timeout means approved execution did not complete. Consumers must present these states explicitly and must not render an incomplete or failed response as a complete governed answer.

Input and Output Example

The final response combines the computed result with its governed presentation and evidence references:

```

{
  "input": {
    "request_id": "req-20260518-093241",
    "result_id": "res-20260518-093247",
    "data_ref": "result:res-20260518-093247",
    "display_spec_ref": "display:res-20260518-093247",
    "narrative_ref": "narrative:res-20260518-093247",
    "lineage_ref": "lineage:req-20260518-093241",
    "compliance_decision_ref": "compliance-decision:req-20260518-093241",
    "artifact_state_ref": null
  },
  "output": {
    "request_id": "req-20260518-093241",
    "result_id": "res-20260518-093247",
    "status": "complete",
    "data_ref": "result:res-20260518-093247",
    "display_spec_ref": "display:res-20260518-093247",
    "narrative_ref": "narrative:res-20260518-093247",
    "lineage": {
      "record_ref": "lineage:req-20260518-093241",
      "available_to_caller": true
    },
    "compliance": {
      "state_ref": "compliance:req-20260518-093241",
      "triggered": false,
      "export_status": "permitted"
    },
    "warnings": [],
    "errors": []
  }
}

```

The MCP Capability Layer returns this response to the consumer, completing the governed request flow.

Governed Data-Mining Example

The portfolio-comparison example demonstrates governed metric calculation. Data mining uses the same platform controls but resolves an approved dataset contract rather than a metric formula. The output is a bounded, typed extract whose selection rules, permitted fields, source state, membership, pagination, and export decision are recorded.

In this variation, an automated research application already knows the approved operation and submits a structured request. It bypasses IRA language interpretation, but it does not bypass SMR resolution, RAPL, SVL, SCL, PQP, FQE, or ALS. The approved operation uses the `data_retrieval` execution profile, so DVL and NSA are not invoked. PAS remains independent of the profile and would run only if the compliance trigger were active.

Component Handoffs

Component	Data-Mining Responsibility	Contract Handoff
AI consumer	Requests a known governed extract for downstream research.	<code>operation_id</code> , typed parameters, requested fields, and authenticated context to MCP.
MCP and SMR	Resolve the approved operation and dataset contract.	Pinned operation and dataset-definition references to RAPL and SVL.
RAPL and DES	Limit the extract to permitted datasets, fields, portfolios, classifications, and export rights.	Entitlement projection containing row scope, field projection, masks, and export policy.
SVL	Validate the dataset contract, parameters, date range, field compatibility, and stable ordering.	Backend-independent LQP describing the governed dataset selection.
SCL	Evaluate estimated row volume, complexity, classification, compliance purpose, concurrency, and timeout.	Approved execution budget and compliance decision, or structured rejection.
PQP and FQE	Bind approved mappings, execute the bounded selection, apply protections, and paginate the result.	Typed dataset pages plus the extract manifest, source snapshot reference, and execution evidence.
ALS	Correlate the contract, projection, plan, snapshot, page set, membership digest, export decision, and terminal outcome.	Authorized lineage reference for review and repeatability testing.
Structured Response	Return the extract without adding a chart or AI narrative.	Data reference, schema, pagination, extract manifest, lineage, and export status.

Technology-Neutral Contract Example

The application requests daily fixed-income positions for its authorized portfolios during April 2026. The requested fields are a subset of the approved dataset contract:

```

{
  "input": {
    "request_id": "req-20260518-101500",
    "operation_id": "retrieve_position_history",
    "operation_definition_ref": "smr:operation:retrieve_position_history@1.0.0",
    "dataset_definition_ref": "smr:dataset:fixed_income_daily_positions@1.0.0",
    "execution_profile": "data_retrieval",
    "parameters": {
      "portfolio_scope": "caller_authorized_portfolios",
      "asset_class": "FIXED_INCOME",
      "date_from": "2026-04-01",
      "date_to": "2026-04-30",
      "requested_fields": [
        "portfolio_id",
        "instrument_id",
        "position_date",
        "market_value",
        "duration",
        "currency"
      ],
      "page_size": 10000
    },
    "entitlement_projection": {
      "entitlement_projection_ref": "projection:req-20260518-101500",
      "authorized_portfolios": [
        "GLOB_FI_CORE",
        "US_CREDIT_OPP"
      ],
      "approved_fields": [
        "portfolio_id",
        "instrument_id",
        "position_date",
        "market_value",
        "duration",
        "currency"
      ],
      "classification_ceiling": "INTERNAL",
      "export_policy_ref": "export-policy:research-internal@3.1"
    }
  },
  "output": {
    "request_id": "req-20260518-101500",
    "result_id": "dataset-res-20260518-101512",
    "status": "complete",
    "data_ref": "dataset:dataset-res-20260518-101512",
    "dataset_definition_ref": "smr:dataset:fixed_income_daily_positions@1.0.0",
    "source_snapshot_ref": "snapshot:portfolio-source:2026-05-01T00:00:00Z",
    "selection_digest": "digest:<illustrative-selection-value>",
    "dataset_membership_digest": "digest:<illustrative-membership-value>",
    "schema": [
      {
        "field": "portfolio_id",
        "type": "string"
      },
      {
        "field": "instrument_id",
        "type": "string"
      },
      {
        "field": "position_date",

```

```

        "type": "date"
    },
    {
        "field": "market_value",
        "type": "decimal",
        "unit": "reporting_currency"
    },
    {
        "field": "duration",
        "type": "decimal",
        "unit": "years"
    },
    {
        "field": "currency",
        "type": "string"
    }
],
"ordering": [
    "position_date",
    "portfolio_id",
    "instrument_id"
],
"pagination": {
    "page_size": 10000,
    "total_rows": 24837,
    "next_page_token": "page:dataset-res-20260518-101512:2"
},
"extract_manifest_ref": "manifest:dataset-res-20260518-101512",
"lineage_ref": "lineage:req-20260518-101500",
"export": {
    "status": "permitted",
    "policy_ref": "export-policy:research-internal@3.1"
},
"warnings": [],
"errors": []
}
}

```

The extract manifest covers every page and records the dataset contract, logical selection, effective field and row projection, stable ordering, source snapshot, per-page digests, complete membership digest, and export decision. An authorized rerun can therefore distinguish a changed selection rule from changed dataset membership.

The platform's accuracy claim for this workflow is limited to the governed extract: that the returned records and fields conform to the approved contract, permissions, source state, and selection rules. Feature engineering, statistical analysis, model training, model validation, and conclusions produced from the extract remain separate governed activities with their own evidence and approval requirements.

External Components

External components remain outside the AI Analytics Platform boundary and retain their own ownership and controls.

Conversational AI - Chat Front End

Business definition and purpose. The chat front end is a consumer experience for people who request analysis conversationally. It exists to manage dialogue and presentation without owning analytical meaning or controls.

Input and output. It receives a user's question, confirmation choices, and structured platform responses. It outputs authenticated analytical requests, rendered answers, and governed follow-up actions such as drilldown. It does not resolve definitions, enforce entitlements, plan queries, or gain access to backend credentials and schemas.

Semantic Data Repository (SDR)

Business definition and purpose. The SDR is the governed description of the organization's data assets. It exists so analytical definitions can connect to understood data structures, quality expectations, and lineage rather than undocumented source fields.

Input and output. It receives approved data models, critical data elements, quality rules, structural metadata, and data-lineage references. It outputs versioned context and mapping targets used by SMR definitions and physical planning. The proposal does not assume that every organization already has a suitable SDR; readiness assessment establishes its coverage, ownership, freshness, and mapping quality.

Data Entitlements Store (DES)

Business definition and purpose. The DES is the independent policy source for who may use which analytical concepts and data populations. It exists to keep access decisions separate from metric authorship and platform administration.

Input and output. It receives approved policies for metrics, dimensions, row scopes, masks, and classification ceilings. It outputs versioned role and policy definitions to RAPL. Policies reference logical concepts rather than physical table and column names.

External Language Model Service

Business definition and purpose. The external language model service provides bounded language interpretation and synthesis. It exists to support natural interaction while remaining outside calculation, authorization, controls, and source execution.

Input and output. For IRA, it receives approved candidate descriptions and the user's question and returns ranking and parameter suggestions. For NSA, it receives bounded result content and returns a draft narrative. IRA and NSA validate those outputs before the flow continues.

Registered Data Sources

Business definition and purpose. Registered data sources hold the governed business data on which approved analytics operate. They remain systems of record or analytical services rather than becoming part of the AI layer.

Input and output. They receive bounded, authorized source instructions from the FQE and return typed data or explicit failures. They do not receive natural-language questions, model prompts, or consumer credentials.

Optional Rendering Service

Business definition and purpose. The optional rendering service converts a governed display contract into a static visual for consumers that cannot render it directly. It exists as a presentation utility, not as part of analytical computation.

Input and output. It receives a self-contained display specification and outputs SVG or PNG. It has no access to analytical definitions, source credentials, or execution backends. Appendix B identifies an illustrative product and implementation for this role.

Design Decisions Requiring Evaluation

The proposal leaves several choices for implementation and governance teams to validate:

- role combinations and separation of duties;
- interaction between explicit denies and multi-role grants;
- operation contracts that may return partial results;
- evidence and freshness rules for caching;
- source snapshot retention and reconstruction;
- supported formula operations and source instruction formats;
- confidence and compliance-purpose thresholds;
- storage controls for evidence integrity and retention; and
- artifact schemas and approval processes for each intended regulatory use.

These are evaluation questions, not hidden implementation details. The evaluation plan in Appendix B defines how to test the architecture before broader adoption.

Governed AI-Enabled Analytics and Data Mining - Technical White Paper - Draft

3. Glossary

This glossary defines the principal components, contracts, roles, and technical terms used in the AI Analytics Platform white paper. Logical architecture terms follow Section 2. Product-specific terms are identified as elements of the illustrative reference implementation in Appendix B. Terms are sorted alphabetically, and cross-references to other entries are shown in **bold**.

Term	Abbreviation	Definition
Admin API	N/A	An illustrative Appendix B service for governed administration of platform settings and authoring workflows. It is not an analytical execution path and does not grant analytical data access. Data Entitlements Store (DES) policies remain under the Entitlements Manager's control.
AI Analytics Platform	N/A	The proposed architecture for governed AI-enabled analytics and data mining. It combines bounded language interpretation with approved semantic definitions, deterministic planning and computation, entitlement enforcement, controls, structured presentation, and retained evidence.
AI consumer	N/A	A conversational assistant, autonomous agent, data-mining workflow, or custom application that submits an authenticated analytical request and presents the governed response. A consumer does not own analytical definitions, permissions, query planning, or governed computation.
Analytical Lineage Store	ALS	The evidence ledger for accepted analytical requests. It correlates request, intent, entitlement, validation, controls, planning, execution, presentation, artifact, response, and terminal events. Its integrity depends on enforced retention, authorization, signing or content addressing, reconciliation, and recovery controls rather than application convention alone.
Authentication and identity token	N/A	A host-issued credential carried through trusted request transport and validated at platform ingress. Verified identity claims are made available through server-side request context; the bearer token is not an analytical tool argument and is not exposed to a language model. Appendix B illustrates this contract with a JSON Web Token (JWT) .
Compliance trigger	N/A	The deterministic decision that additional provenance evidence is required. The proposed rule requires both a compliance-relevant metric signal and a compliance-purpose signal. A natural-language request receives the purpose signal from the Intent Resolution Agent (IRA) subject to threshold evaluation; a structured caller declares purpose through a validated field. Either signal alone is insufficient.
Data Context Store	DCS	The shared versioned storage and access service that houses the separate Semantic Metrics Repository (SMR) and Semantic Data Repository (SDR) in the illustrative architecture. It does not contain every platform policy or runtime record; the DES and ALS remain distinct responsibilities.
Data Entitlements Store	DES	The independent policy source for analytical access. It stores versioned policy for permitted metrics, dimensions, row scopes, column masks, and classification ceilings. The MCP layer uses a bounded policy view for catalog discovery, while the Role-Aware Projection Layer (RAPL) performs the full request-time entitlement evaluation.
Data mining	N/A	In this paper, the use of governed bulk datasets by an authorized research application, agent, or analytical process to investigate patterns or develop models. The platform controls dataset selection, fields, row scope, source state, pagination, export, and evidence. Downstream feature engineering, statistical analysis, model training, validation, and conclusions remain separately governed activities.

Term	Abbreviation	Definition
Data Visualization Language	DVL	The governed presentation contract produced deterministically from the approved intent, result schema, and registered presentation metadata. It returns a chart or table specification rather than a rendered image. An authorized caller may request a compatible governed override; incompatible presentation choices are rejected.
Dataset contract	N/A	An approved, versioned <code>analytical_dataset</code> definition in the SMR . It declares the permitted fields, classifications, selection rules, source mapping, ordering, pagination, and freshness expectations for governed data retrieval.
Execution profile	N/A	A field on an approved <code>analytical_operation</code> definition that controls optional response assembly after the mandatory governed pipeline has run. <code>data_retrieval</code> returns a governed dataset, <code>metric_query</code> adds a DVL specification, and <code>full_analytical</code> may also add an NSA narrative. The profile never bypasses entitlements, validation, controls, planning, execution, or lineage. PAS invocation is controlled by the compliance trigger , not by the execution profile.
External language model service	N/A	A model service called through bounded adapters by the IRA or NSA . It may rank approved candidates, suggest parameter bindings, or draft a narrative from permitted result content. It does not receive source credentials, authorize access, calculate governed metrics, or generate the executable physical query.
Federated Query Engine	FQE	The controlled execution boundary and the only logical component with registered backend connections and credentials. It admits approved physical execution envelopes, runs bounded source instructions, assembles compatible results, applies any remaining protections, and records execution evidence.
Financial Services Reference Model	FSRM	Illustrative Appendix B bundles of proposed SMR definitions for shared dimensions and selected financial-services domains. The bundles are seeded in <code>proposed</code> state and become resolvable only after governance approval.
Intent confirmation card	N/A	A structured choice returned when intent or compliance purpose is ambiguous. It presents ranked approved operations, bound parameters, material filters, and a presentation preview without executing a backend query or exposing physical schemas.
Intent Resolution Agent	IRA	The bounded AI component that translates natural language into an approved operation and typed parameters. It ranks candidates from an entitlement-scoped SMR catalog view, records confidence and purpose evidence, and requests confirmation when meaning is uncertain. It does not receive credentials, generate executable queries, or approve definitions. Structured requests that already identify an approved operation bypass the IRA.
JSON Web Token	JWT	The token format used for authentication and identity in the illustrative Appendix B implementation. The implementation validates its signature, issuer, audience, expiry, and required claims against trusted keys before analytical processing begins.
Knowledge Store	N/A	An illustrative Appendix B store of versioned, non-user-data guidance exposed read-only through MCP resources. Knowledge resources help consumers use the platform but do not provide an alternate analytical execution path.
Logical Query Plan	LQP	A backend-independent plan produced by the Semantic Validation Layer (SVL) . It carries pinned definition versions, logical calculations, filters, row scope, masks, compliance inputs, output shape, and other governed planning context. It contains no credentials, connection endpoints, physical table names, or executable SQL.

Term	Abbreviation	Definition
MCP Capability Layer	MCP	The single governed platform entry point. It authenticates requests, applies entitlement-aware tool and catalog discovery, accepts natural-language or structured operation requests, correlates the request, and returns the Structured Response . A structured request may bypass the IRA, but it still resolves an approved SMR operation and passes the complete entitlement and controls pipeline.
Metric definition	N/A	An approved, versioned SMR declaration of a metric's business meaning, formula or governed measure, aggregation behavior, units, compatible dimensions, classification, source affinity, mapping reference, owner, and tests. Access policy is maintained independently in the DES .
Model Context Protocol	MCP	The protocol used by the proposed platform to expose governed tools, resources, and prompts to AI consumers. The protocol does not itself provide semantic governance, entitlement enforcement, or analytical correctness.
Narrative	N/A	An optional plain-language explanation produced by the Narrative Synthesis Agent (NSA) from approved result labels, values, units, dimensions, and permitted derivations. Unsupported numbers, causes, recommendations, forecasts, or significance claims are rejected or omitted.
Narrative Synthesis Agent	NSA	The optional post-computation AI component that drafts a result-grounded narrative. A deterministic validator checks numbers, units, labels, comparison direction, and permitted derivations before release. Failure to produce a valid narrative does not change the computed data, DVL specification, lineage, or compliance evidence.
Optional Rendering Service	N/A	An external presentation utility that converts a self-contained DVL specification into SVG or PNG. It has no access to analytical definitions, source credentials, or execution backends and does not participate in governed computation.
Physical Query Planner	PQP	The deterministic boundary that resolves approved mappings and source capabilities for an LQP and produces a bounded physical execution envelope. It binds typed predicates, expands time periods, groups work by source, applies protection directives, and records plan integrity evidence without holding source credentials or executing the plan.
Provenance Artifact	N/A	A sealed, cryptographically signed evidence package produced when the compliance trigger is active. It binds the request and purpose evidence to the applicable definitions, entitlement decision, controls, plans, execution, result, and verification metadata. A valid signature establishes that the sealed bytes have not changed; it does not prove that the calculation is correct or legally sufficient.
Provenance Artifact Service	PAS	The component that assembles and seals a Provenance Artifact from the correlated ALS evidence set. It is invoked only when both compliance signals are active. Export remains blocked until sealing succeeds. PAS is independent of the execution profile and runs after the evidence required by the approved artifact contract is available.
Registered data source	N/A	A governed warehouse, lakehouse, semantic service, approved data API, relationship store, or multidimensional engine available to the FQE through an approved connection and mapping. Registration requires defined credentials, classifications, timeout behavior, capabilities, and failure semantics.
Result Cache	N/A	An optional cache inside the controlled execution boundary. Its canonical key includes the effective plan, tenant, entitlement projection, pinned definition versions, and relevant configuration. Every cache hit still passes current authorization and creates new lineage evidence. Compliance-purpose requests bypass the cache unless approved policy provides equivalent evidence and freshness guarantees.

Term	Abbreviation	Definition
Role-Aware Projection Layer	RAPL	The deterministic component that combines verified identity context with current DES policy to produce the caller's effective entitlement projection. The projection contains permitted concepts, row scope, masks, classification ceiling, policy versions, and decision evidence.
Semantic Controls Layer	SCL	The mandatory release gate between semantic validation and physical planning. It evaluates data scale, complexity, classification, compliance, and concurrency against versioned controls, assigns an execution timeout, records the decision, and rejects requests that may not proceed. When the compliance trigger is active, it marks the provenance artifact as required and export as blocked pending PAS.
Semantic Data Repository	SDR	The governed source of business and physical data metadata, including data models, critical data elements, quality rules, structures, mappings, profiling evidence, and data-lineage references. It is separate from the SMR , although both may use the DCS . Adoption requires a readiness assessment of its coverage, ownership, freshness, and mapping quality; the architecture does not assume that every organization already has a suitable SDR.
Semantic Metrics Repository	SMR	The governed catalog of approved analytical metrics, dimensions, operations, dataset contracts, presentation metadata, and mapping references. Definitions are owned, versioned, reviewed, and approved before governed resolution. The SMR defines analytical meaning; the SDR describes the underlying data context. Search indexes or embeddings may support discovery but do not change approval state or access rights.
Semantic Validation Layer	SVL	The deterministic semantic quality gate that receives a resolved request, entitlement projection, and approved SMR definitions and produces a backend-independent LQP or structured rejection. It validates parameters, pins versions, checks semantic compatibility, applies the entitlement projection, and attaches validated compliance inputs. No language model runs in this component.
Structured Response	N/A	The governed delivery envelope returned to a consumer. It distinguishes status, typed data or a data reference, DVL specification, optional narrative, lineage, compliance state, warnings, and errors. Consumers render this contract without recalculating or reinterpreting governed values.

Governed AI-Enabled Analytics and Data Mining - Technical White Paper - Draft

Appendix A. Text-to-SQL: Appropriate Use and Governance Boundary

Purpose

This appendix explains why direct Text-to-SQL is useful for exploration but is not, by itself, a sustainable execution foundation for regulated analytics. It supports the position stated in the Executive Overview without repeating the full architecture.

The concern is not SQL, natural-language interfaces, or exploratory analysis. It is the delegation of business interpretation and executable-query generation to a probabilistic model without an intervening governed semantic contract.

In this appendix, **direct Text-to-SQL** means a pattern in which a language model receives a question and physical schema context, generates executable SQL, and submits that SQL to a database. Implementations may add database permissions, query validation, cost controls, and human approval. Those controls are valuable, but they do not establish the approved meaning of a business metric or dataset.

Where Text-to-SQL Adds Value

Text-to-SQL can shorten the path from a question to an exploratory result. It is particularly useful when:

- an analyst is investigating unfamiliar data;
- the question is ad hoc and low consequence;
- the result is clearly labeled as exploratory;
- the user is qualified to inspect the generated query and its output; and
- the output does not flow directly into a regulated report, automated decision, or reusable business metric.

These interactions can reveal recurring questions, missing metrics, and useful dataset requirements. They are therefore an input to governance: frequently used exploratory logic can be reviewed, tested, and promoted into an approved metric or dataset contract.

Why It Is Not the Governed Execution Layer

The distinction is not whether both approaches eventually execute SQL. The distinction is where business meaning, authorization, and evidence are established.

Concern	Direct Text-to-SQL	Governed Semantic Execution
Business meaning	Inferred from the question, prompt, and schema context for each request.	Resolved to an approved, versioned metric, operation, or dataset contract.
Executable query	Generated by the language model.	Produced deterministically from a validated logical plan and approved mappings.
Access	Primarily constrained by database permissions and implementation-specific filters.	Expressed as a request-specific projection covering concepts, fields, rows, masks, and classification.
Unsupported concepts	May still produce plausible SQL.	Rejected or returned for clarification when no approved definition applies.
Change control	Prompt, schema context, model behavior, and query output can change independently.	Definitions, policies, mappings, controls, and software are pinned and versioned.
Repeatability	Re-running the same wording can generate a different query.	Re-running against preserved inputs and source state can be compared using recorded versions and digests.
Evidence	Often limited to the prompt, generated SQL, database logs, and result.	Correlates interpretation, definitions, entitlements, controls, plans, execution, result, and terminal status.

Accuracy

A syntactically valid query is not necessarily a correct business calculation. Physical schemas do not fully express approved formulas, fiscal calendars, null treatment, weighting rules, benchmark selection, or the definition version that applies to a reporting period. Prompt instructions can describe those rules, but they remain guidance to a probabilistic generator rather than an enforced analytical contract.

This difference matters most for calculations that are reused or consequential. A governed path can reject an unknown or incompatible concept. A direct Text-to-SQL path is optimized to produce a query, which can turn ambiguity into a plausible but unsupported answer.

Traceability

Generated SQL shows what a database was asked to execute. It does not necessarily show why that query represents the approved business definition, which entitlement decision applied, which definition owner approved it, or whether all required controls completed. Database and model logs can contribute evidence, but they do not replace those missing decisions.

Repeatability

Repeatability requires more than retaining SQL. It requires the request, definition versions, policies, mappings, execution settings, software version, and relevant source state. Direct Text-to-SQL also introduces a probabilistic generation step:

the same question can produce a different query after a context, prompt, or model change. Governed semantic execution records the resolved contract and uses deterministic software after interpretation.

Security and Operational Control

Read-only credentials, parameter binding, statement allowlists, syntax-tree inspection, query budgets, timeouts, row-level security, and approval gates reduce the risk of generated SQL. They should be used where Text-to-SQL is permitted. They address unsafe execution; they do not create approved analytical meaning.

The direct pattern can also expose more physical metadata to the model and user experience than a semantic interface requires. Complex generated queries can create unpredictable cost or load. These are manageable engineering risks for a bounded exploration service, but they strengthen the case for a separate governed execution path for recurring or consequential workloads.

A Complementary Operating Model

Text-to-SQL and governed semantic execution can coexist behind the same consumer experience if their status and boundaries remain explicit.

1. A user submits an analytical question.
2. The consumer first searches the approved operation and dataset catalog.
3. If an approved contract applies, the request follows the governed execution path.
4. If no approved contract applies and policy permits exploration, the user may choose a separately labeled Text-to-SQL path.
5. Repeated or consequential exploratory logic becomes a candidate for definition, testing, ownership, approval, and registration.

The exploratory output must not silently acquire governed status. Promotion occurs only after the organization has defined the business meaning, validated the result, assigned an owner, established entitlements, and approved the applicable mappings and controls.

Use Case	Appropriate Path
Investigating an unfamiliar dataset	Controlled Text-to-SQL exploration
Testing a hypothesis that will be independently reviewed	Controlled Text-to-SQL exploration
Prototyping a new metric definition	Exploration followed by formal promotion
Producing a recurring management metric	Governed semantic execution
Supplying data to an automated or data-mining workflow	Governed dataset contract
Supporting a regulated report or consequential decision	Governed semantic execution with the required evidence controls

Evaluation Implication

The architectural proposition should be tested rather than accepted from diagrams alone. The evaluation plan in Appendix B compares the governed approach with the organization's current process and a controlled Text-to-SQL implementation using the same questions, source state, permissions, and reference answers.

That comparison should measure correctness, unsupported-question handling, repeatability, traceability effort, latency, cost, authoring effort, and failure behavior. The objective is not to prove that one technique wins every category. It is to determine whether governed semantic execution provides sufficient control for the workloads that require it and whether Text-to-SQL remains useful within a clearly bounded exploratory role.

Conclusion

Text-to-SQL is a valuable discovery tool, not the enemy of governed analytics. The unsustainable pattern is allowing generated queries to become the unreviewed source of recurring business truth. The proposed architecture retains AI-assisted exploration while placing approved definitions, deterministic planning, entitlements, controls, and evidence between natural-language interpretation and governed execution.

Appendix B. Illustrative Reference Implementation and Evaluation Plan

This section illustrates how a technology stack could support the proposed architecture. The technology mappings and code fragments explain implementation responsibilities; they are not evidence of a completed or benchmarked system. Alternative stacks can support the same responsibilities. Integration feasibility, correctness, performance, and operating cost require validation against the intended workload.

The proposed behaviors, interfaces, and governance requirements are in Section 2. The design principles define the intended constraints on this implementation.

Readers making an architectural or investment decision can use the Reference Build Summary and Architecture Overview as the implementation summary. The layer-by-layer and infrastructure sections provide companion detail for engineering review, including illustrative pseudocode, schemas, configuration, infrastructure, and seed definitions. The evaluation plan defines the evidence requirements and decision gates that determine whether the proposal should proceed.

Reference Build Summary

This section maps the capabilities defined in Section 2 to a proposed technology stack. Each mapping names the technology, the work it performs, and the application code needed to complete the capability. Teams may substitute technologies that satisfy the same interface contracts and governance requirements.

The table follows the logical flow in Section 2. It accounts for platform components, external dependencies, consumer responsibilities, and optional services so that an omitted row cannot be mistaken for an omitted capability. Python entries require custom application code. The Data Context Store is an existing platform service; this reference implementation does not prescribe its underlying storage product.

Capability (Section 2)	Abbreviation	Technology Stack	Implementation Description
AI Consumers	-	Claude-based chat client, autonomous agent, or custom application	Consumer products provide the user experience, submit natural-language or structured analytical requests, and render the governed response. They do not calculate governed values or bypass platform controls.
MCP Capability Layer	MCP	Python 3.12, FastMCP 2.x, Uvicorn, python-jose	FastMCP exposes tools, resources, and prompts, and Uvicorn serves the application over HTTP. Python middleware uses python-jose to validate identity tokens before routing requests to the analytical pipeline.
Intent Resolution Agent	IRA	Python, Anthropic Claude, Data Context Store (existing service)	Python retrieves candidate operations from the semantic catalog. Claude ranks them, binds request parameters, and scores compliance intent. The application asks the user to confirm ambiguous requests.
Semantic Metrics Repository	SMR	Python, Data Context Store (existing service)	The Data Context Store stores, versions, approves, and indexes metric, dimension, operation, and dataset definitions. Python accesses those definitions through the service API.
Semantic Data Repository	SDR	Data Context Store (existing service)	The existing Data Context Store supplies governed business and physical data metadata. SMR physical mappings reference SDR objects rather than embedding unmanaged source-schema knowledge in AI prompts.
Data Entitlements Store	DES	PostgreSQL	PostgreSQL stores entitlement policies in a dedicated schema with separate credentials. The Entitlements Manager maintains the policies, and the RAPL reads them when evaluating requests.
Role-Aware Projection Layer	RAPL	Python, asyncpg, PostgreSQL	Python uses asyncpg to read entitlement policies from PostgreSQL. It combines the policies with verified identity claims to determine permitted metrics, dimensions, rows, and column protections.
Semantic Validation Layer	SVL	Python, Pydantic v2	Pydantic validates tool input models. Python checks operation parameters against registered JSON schemas, resolves semantic definitions, enforces the entitlement projection, and builds the logical query plan.

Capability (Section 2)	Abbreviation	Technology Stack	Implementation Description
Semantic Controls Layer	SCL	Python, Redis, Data Context Store (existing service)	Python evaluates the five controls checks using configured thresholds and profiling statistics. The Data Context Store stores the configuration, and Redis coordinates concurrent-query admission across application instances.
Physical Query Planner	PQP	Python, Apache Calcite planner adapter	Python binds approved logical concepts to physical catalog references and calls a JVM-based Calcite adapter over a loopback interface. Calcite produces Trino SQL with the required filters and column protections.
Federated Query Engine	FQE	Starburst (Trino), Python, Trino Python client	Starburst acts as the FQE. The Trino Python client submits SQL to its coordinator, and Starburst executes queries across configured source connectors. Python verifies the returned schema, caches results, and records execution details.
Registered Data Sources	-	Snowflake, BigQuery, Databricks, Redshift, dbt Semantic Layer, Cube, REST, OData, Neo4j, Amazon Neptune	Validated Starburst or Trino connectors provide access to supported sources; other systems require an approved adapter or custom connector. Source registration and approved mappings determine which data can participate in governed execution.
Data Visualization Language	DVL	Python, Vega-Lite v5	Python selects the registered chart contract and generates a Vega-Lite display specification. Tabular results use the platform's separate table format.
External Language Model Service	-	Anthropic Claude API	The IRA and NSA call the model service through separate, governed adapters. The model interprets intent or drafts a narrative; it does not receive database credentials or generate the executable query.
Narrative Synthesis Agent	NSA	Python, Anthropic Claude Haiku 4.5, Anthropic Claude Sonnet 4.6	Claude generates narratives from computed result values. Python selects Haiku for simple summaries or Sonnet for complex results, constructs the prompt, and validates the generated text.
Analytical Lineage Store	ALS	Python, Amazon S3, PostgreSQL	S3 stores lineage documents, and PostgreSQL indexes the fields used to find them. Python coordinates writes and retrieval, with retention controls and integrity checks protecting the records.
Provenance Artifact Service	PAS	Python, cryptography, HashiCorp Vault or Kubernetes Secrets, Amazon S3	Python assembles the provenance artifact, and cryptography signs it with ECDSA P-256 and SHA-256. Vault or Kubernetes Secrets supplies the key. The lineage service stores the signed artifact in S3 under configured retention controls.
Structured Response	-	Python, Pydantic v2, FastMCP 2.x	Pydantic assembles and validates the typed response containing data, presentation, narrative, lineage, compliance state, warnings, and errors. FastMCP returns that package without allowing the consumer to recalculate governed values.
Optional Rendering Service	-	Python, FastMCP 2.x, vega-embed, Playwright, Chromium	A separate MCP service converts a self-contained Vega-Lite specification to SVG or PNG. It has no access to analytical sources and is not part of governed computation.

Capability (Section 2)	Abbreviation	Technology Stack	Implementation Description
Result Cache	-	Python, Redis	Redis stores assembled results under keys derived from the complete effective execution context. Python reauthorizes every hit, applies the configured expiry, and bypasses cache reads and writes when the compliance trigger is active.

The two embedded AI components are the **Intent Resolution Agent (IRA)** and the **Narrative Synthesis Agent (NSA)**. Every stage between them — RAPL, SVL, SCL, PQP, and FQE — is deterministic.

Reference Build Profile

This appendix defines one bounded reference build, not a complete product implementation. Its purpose is to prove that the logical contracts in Section 2 can be assembled, secured, and evaluated using the named stack. Productization, multi-region operation, every Financial Services Reference Model domain, and every optional integration remain outside the minimum build.

The reference build uses a **modular monolith** for the governed application pipeline. MCP, IRA adapters, RAPL, SVL, SCL, PQP client, DVL, NSA adapters, ALS coordination, PAS, and cache coordination run as modules in the `analytics-mcp` application. Apache Calcite runs in a JVM sidecar in the same Kubernetes pod because it is not a native Python library. Starburst, the DCS, PostgreSQL, Redis, object storage, the identity provider, and the external language-model service remain external dependencies. This deployment choice reduces distributed-system complexity while preserving component interfaces that could support later separation.

Build Area	Included in the Minimum Reference Build	Deferred or Optional
Governed operations	One metric comparison, one governed dataset retrieval, and one compliance-triggering metric operation.	The full six-domain reference catalog and broad operation coverage.
Request paths	Structured operation calls and natural-language resolution with confirmation for ambiguity.	Additional channels, scheduled execution, alerts, and collaboration.
Data sources	One primary analytical source and one second source sufficient to exercise federation.	Every connector listed in the broader capability matrix.
Governance	Definition approval state, entitlements, validation, all SCL checks, lineage, terminal status, and compliance export gating.	Organization-specific approval user interfaces and enterprise workflow integration.
Responses	Typed data, governed table or chart specification, optional validated narrative, and lineage reference.	Static image rendering, customized themes, and advanced interaction patterns.
Operations	Repeatable deployment, configuration, health checks, structured logs, metrics, traces, failure injection, and evidence reconciliation.	Multi-region disaster recovery and production service-level commitments.

The build is complete enough for formal evaluation only when all three operations traverse the mandatory controls sequence, expected authorization failures are demonstrated, every accepted request has a durable terminal record, preserved inputs can be rerun, and the evaluation corpus can execute without manual changes to application code.

Candidate Bill of Materials

The following products define the candidate stack for the reference build. Exact versions must be pinned in the build manifest and recorded in the evaluation evidence pack. A named product or major version in this paper is a design choice to validate, not a statement that the combination has already been integration-tested.

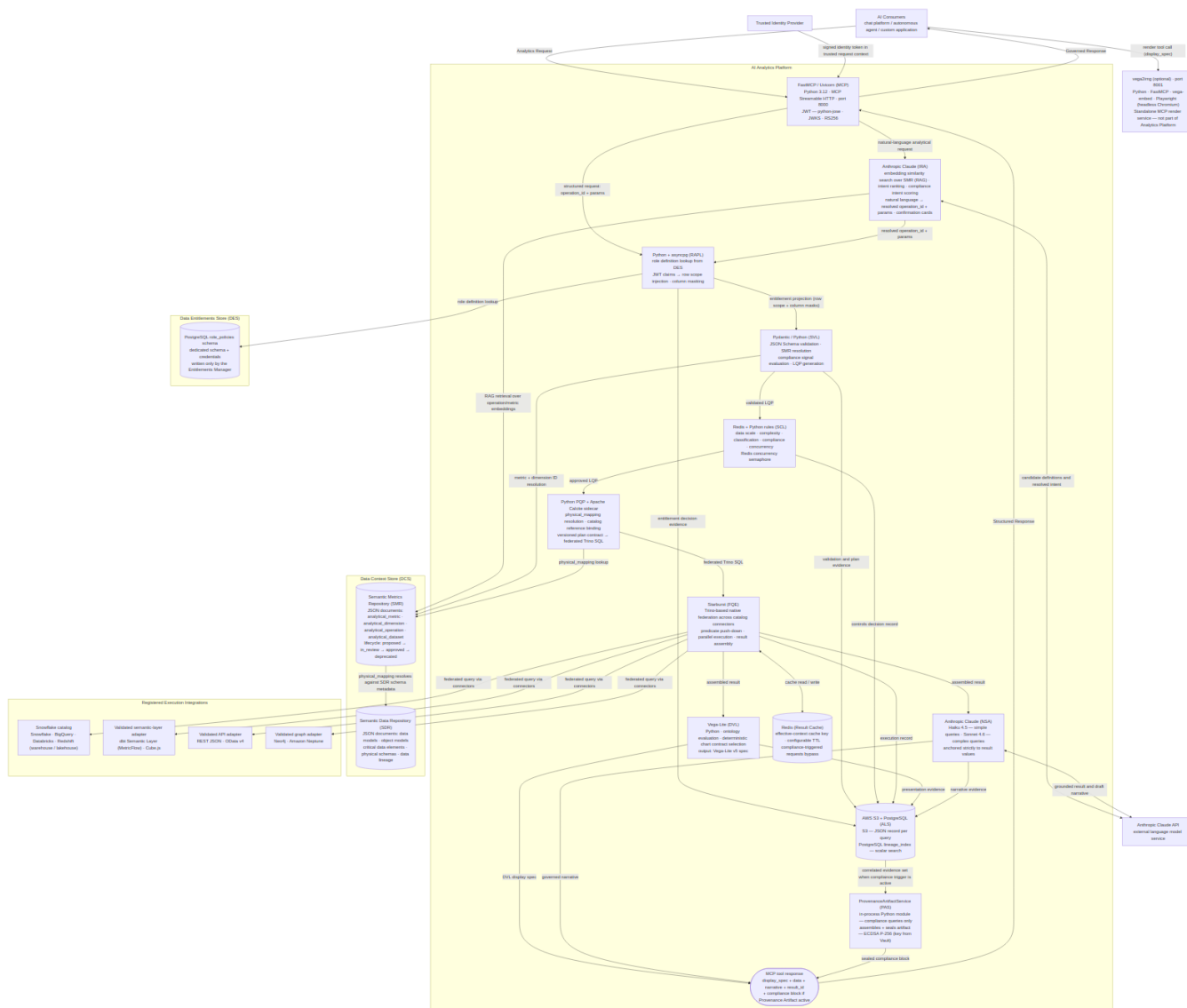
Product or Runtime	Reference-Build Role	Minimum Build	Validation Required
Python 3.12, FastMCP, Uvicorn, Pydantic	Governed application runtime, MCP transport, and typed contracts.	Yes	Confirm supported versions, transport behavior, cancellation, and authentication middleware integration.
Apache Calcite on a JVM sidecar	Relational planning and Trino SQL generation.	Yes	Validate the loopback contract, Trino dialect output, startup dependency, timeout, and failure behavior.
Starburst or compatible Trino distribution	Federated execution boundary.	Yes	Validate every selected connector and required push-down, masking, cancellation, and query-evidence capability.
PostgreSQL	DES policies and ALS search index under separate schemas and credentials.	Yes	Validate migration, backup, recovery, isolation, and reconciliation with object storage.
Redis	Result cache and distributed concurrency state.	Yes	Validate eviction, fail-closed concurrency behavior, key isolation, and recovery after interruption.
S3-compatible object storage	Lineage, artifacts, and knowledge content.	Yes	Validate object lock or equivalent retention controls, versioning, signing, lifecycle, and legal hold where required.
External language-model service	IRA ranking and optional NSA narrative generation.	IRA only	Pin model identifiers in deployment configuration and test ambiguity, unavailability, and output validation.
Vega-Lite and optional rendering service	Governed presentation contract and optional image rendering.	Specification only	Validate consumer compatibility; image rendering is not required for the minimum build.
Kubernetes and a managed secrets service	Deployment, workload identity, configuration, and secret delivery.	Yes	Validate pod security, network policy, autoscaling assumptions, key rotation, and workload identity.

Build Increments and Acceptance Evidence

Increment	Deliverable	Acceptance Evidence
1. Foundation	Deploy the application pod, Calcite sidecar, PostgreSQL, Redis, object storage, DCS client, identity validation, and Starburst connectivity.	Reproducible deployment, pinned manifest, dependency health, secret rotation test, and denied unauthenticated request.
2. Structured metric execution	Implement SMR resolution, RAPL, SVL, SCL, PQP, FQE, ALS, and the structured response for one metric comparison.	Approved reference result, negative entitlement tests, plan and execution evidence, and durable terminal states.

Increment	Deliverable	Acceptance Evidence
3. Governed data retrieval	Add the approved dataset contract, field and row projection, stable ordering, pagination, manifest, and export decision.	Expected membership digest, page continuity, denied-field tests, snapshot reference, and repeatable extract.
4. Natural language and presentation	Add IRA resolution and confirmation, DVL generation, and the optional validated NSA narrative.	Intent corpus results, ambiguity behavior, governed fallback presentation, and narrative omission on validation failure.
5. Compliance evidence	Add the two-signal trigger, PAS signing, verification, retention enforcement, and export gate.	Trigger truth-table results, non-exportability before sealing, signature verification, key-rotation evidence, and failure test.
6. Evaluation readiness	Add observability, failure injection, reconciliation, workload execution, and evidence-pack generation.	Entry-criteria review and successful execution of the corpus defined in the Evaluation Plan.

Architecture Overview



The Semantic Metrics Repository (SMR) and the Semantic Data Repository (SDR) are two independent stores housed within the Data Context Store (DCS). The SDR is a pre-existing organizational component holding the foundational data definitions — data models, physical schemas, and data lineage. The SMR is a separate store holding the four analytical document types (`analytical_metric` , `analytical_dimension` , `analytical_operation` , `analytical_dataset`); both stores are built on the DCS's shared versioned storage, search index, and scoped access control, and both are reached through the DCS API. The `physical_mapping` fields in SMR metric definitions resolve against SDR schema metadata to locate the physical tables and columns behind each metric.

Layer-by-Layer Stack Decisions

The implementation subsections can be read in the same logical sequence as Section 2: [MCP](#), [IRA](#), [SMR](#), [RAPL](#), [SVL request validation](#), [SVL logical planning](#), [SCL](#), [PQP](#), [FQE](#), [DVL](#), [NSA](#), [ALS](#), and [PAS](#). Supporting runtime services follow the primary flow. The subsection placement groups closely related implementation material, while these links preserve traceability to the component sequence.

Unless a block is explicitly described as a complete configuration or contract example, Python blocks in this appendix are **illustrative pseudocode**. Ellipses identify behavior that the build must implement and test; the fragments are not intended to be copied into a deployable service as written. JSON examples are technology-neutral contract instances or illustrative configuration, not generated evidence from a running system.

MCP Capability Layer

Specification: §MCP Capability Layer

FastMCP implements the MCP Capability Layer's tool, resource, and prompt interfaces. Uvicorn serves the Python application over HTTP. Application middleware validates identity and routes analytical requests to the controls pipeline.

Decision	Choice	Rationale
Runtime	Python · FastMCP + Uvicorn	Lightweight ASGI service; minimal dependencies; deploys as a Kubernetes pod or serverless container
Protocol	MCP Streamable HTTP	Standard MCP interoperability; supports request/response and streaming
Auth	JWT validation at request ingress	Stateless; validated before any platform computation begins
Tools	Three tools: <code>run_analytics</code> (SMR-driven execution), <code>list_operations</code> (discovery), <code>drilldown</code> (result navigation)	SMR owns all operation definitions; code is the execution engine
Resources	Knowledge artifacts only — guides, skills definitions, compliance reference	Static; no user data; embedded in AI consumer context before analytical tasks; no controls pipeline
Prompts	Pre-built analytical and regulatory assistant templates	Inject available metrics and governance constraints at session start

FastMCP (`pip install fastmcp`) provides the `@mcp.tool()` , `@mcp.resource()` , and `@mcp.prompt()` decorators and handles MCP Streamable HTTP transport. Each analytical capability is a decorated Python function; the framework serializes schemas and routes calls automatically.

The separation of tools and resources is intentional. All analytical execution goes through `run_analytics` , a single tool that delegates to the SMR for every operation definition. Resources expose static knowledge artifacts from the Knowledge Store; they contain no user data and require no governance evaluation. The SMR owns what operations exist, what parameters they require, and which presentation stages they invoke. The code owns only the execution engine.

Tools

Three tools cover the entire analytical surface. The SMR owns every operation definition: what parameters it needs, what metrics and dimensions it supports, and which presentation stages it invokes. No operation type is hardcoded in the execution layer.

```

from fastmcp import FastMCP
from pydantic import BaseModel, Field, model_validator

mcp = FastMCP(
    name="AI Analytics Platform",
    instructions=(
        "Governed analytical execution engine. All operations are defined in the Semantic Metrics Repository."
        "
        \"Supply a natural-language question to run_analytics, or call list_operations to discover an \"
        \"approved operation and its required parameters before making a structured call.\"
    ),
)

# — Tool input models —————

class RunAnalyticsInput(BaseModel):
    question: str | None = None # natural language; resolved by the IRA
    operation_id: str | None = None # approved SMR operation ID for structured callers
    params: dict = Field(default_factory=dict) # validated against the resolved operation schema

    @model_validator(mode="after")
    def require_one_request_form(self):
        if bool(self.question) == bool(self.operation_id):
            raise ValueError("Supply either question or operation_id, but not both")
        return self

class ListOperationsInput(BaseModel):
    domain: str | None = None # optional filter by analytical domain

class DrilldownInput(BaseModel):
    result_id: str
    hierarchy: str
    selected_value: str | None = None

# — Tools —————

@mcp.tool()
async def run_analytics(input: RunAnalyticsInput) -> dict:
    """Resolve or execute an SMR-registered analytical operation.
    Natural-language questions pass through the IRA. Structured callers may supply an approved
    operation_id and typed parameters directly.
    The presentation depth – raw dataset, display specification, or full analytical response – is
    determined by the operation's execution_profile in the SMR, not by this tool; the full controls
    pipeline runs for every operation."""
    # 1. Validate JWT → claims
    # 2. If question is present, invoke the IRA to resolve operation_id + params
    # 3. Resolve the approved operation from SMR – rejects unknown/unapproved operation IDs
    # 4. Delegate to pipeline_executor.run – returns result shaped by execution_profile
    ...

@mcp.tool()
async def list_operations(input: ListOperationsInput) -> dict:
    """List all SMR-registered operations available to the current user's role.
    Returns operation IDs, display names, required parameters, supported metrics,
    supported dimensions, and execution profiles."""
    # 1. Validate JWT → claims
    # 2. Query SMR for approved operations scoped to claims["org_id"], filtered by domain if supplied
    ...

@mcp.tool()

```

```

async def drilldown(input: DrilldownInput) -> dict:
    """Navigate into a dimension hierarchy from a prior result.
    The parent result's analytical context (operation, filters, hierarchy position) is inherited;
    entitlements and controls are re-evaluated in full for the derived query."""
    # 1. Validate JWT → claims
    # 2. Delegate to drilldown_service.execute – inherits analytical context, never approvals
    ...

```

JWT Validation

The `python-jose[cryptography]` library verifies JWT signatures and claims using keys from the configured JWKS endpoint. Application middleware manages the key cache and enforces the required identity and analytical-role claims listed below. The cache refreshes at startup, after its one-hour TTL expires, and when an unknown key identifier requires a refresh.

```

from jose import jwt, JWTError

JWKS_URI      = config["auth"]["jwks_uri"]          # e.g. https://auth.example.com/.well-known/jwks.json
JWT_AUDIENCE  = config["auth"]["audience"]
JWT_ISSUER    = config["auth"]["issuer"]

jwks_cache = {} # { "keys": [...], "fetched_at": timestamp }

async def validate_jwt(token: str) -> dict:
    # Input: raw Bearer token from MCP request header
    # Output: verified claims dict – { sub, org_id, analytics_roles, managed_portfolios, ... }

    # 1. Refresh JWKS key cache if stale (>1 hour) – avoids a key fetch on every request
    # 2. Decode and verify JWT using RS256 + cached public keys – raises AuthenticationError on failure
    # 3. Assert required claims (sub, org_id) are present – missing claims are rejected immediately
    ...

```

Caller Identity Claims

Every request carries a host-issued JWT in the `Authorization: Bearer <token>` header — the reference implementation uses it as the authentication and identity token. Authentication middleware validates the token and makes verified claims available through server-side request context; bearer tokens are never MCP tool arguments and are never exposed to the model, tool schema, lineage payload, or application logs. Expired tokens and tokens carrying no analytical role claim are denied immediately.

Validation pins the permitted signature algorithm, verifies `iss`, `aud`, `exp`, and `nbfi` when present, and rejects missing required claims. JWKS caching honors key identifiers and supports refresh on an unknown `kid` so planned and emergency rotations do not require a service restart. Downstream services receive audience-specific exchanged tokens or workload credentials rather than the original bearer token unless an explicit, reviewed delegation policy permits forwarding.

Required claims

Claim	Type	Description
<code>sub</code>	string	Caller's unique identifier within the organization
<code>org_id</code>	string	Must match the platform's registered organization identifier

Claim	Type	Description
<code>exp</code>	number	Token expiry timestamp
Field named by <code>roleClaim</code> Field	<code>string[]</code>	Analytical role array — consumed by the RAPL for metric, dimension, and row-level access

Optional analytical claims

Claim	Type	Description
<code>managed_portfolios</code>	<code>string[]</code>	Portfolio IDs managed by the caller — resolved in row scope templates
<code>entity_ids</code>	<code>string[]</code>	Legal entities the caller is associated with
<code>display_name</code>	string	Display name for lineage records and the audit trail
Any claim referenced in a role's row scope template	any	Any value referenced by <code>{{user.claim_name}}</code>

Tokens expire per the consuming application's standard session policy: the consumer issues a refreshed token and includes it on the next request — there is no platform-side re-authentication step. Long-running agentic consumers must supply fresh tokens before each preceding token's `exp`.

Agentic consumers. Autonomous agents — scheduled pipelines, event-triggered monitors, report generators — integrate identically to interactive consumers. The host provisions service-level tokens scoped to the agent's role rather than a user's identity; the RAPL applies identical entitlement enforcement, so an agent cannot access data that a user with the same role cannot access. Every agent-initiated request is recorded in the ALS under the agent's `sub` claim, keeping agent queries distinguishable from user queries in the audit trail.

Request Routing

The MCP layer validates the JWT, then routes by call type. A **natural language** query is sent to the Intent Resolution Agent (IRA), which resolves it to an `operation_id + params` before the deterministic pipeline begins. A **structured** call (an explicit `operation_id + params`) bypasses the IRA and enters the deterministic pipeline directly. From that point the order is the same in both cases: `RAPL → SVL → SCL → PQP → FQE`.

Pipeline Executor

The deterministic pipeline runs in a fixed order: RAPL computes the entitlement projection first, the SVL then validates the request and compiles the Logical Query Plan with that projection enforced, the SCL applies its controls checks, the PQP translates the approved plan into federated Trino SQL, and the FQE submits it to Starburst for execution.

```

class PipelineExecutor:
    def __init__(self, ira, rapl, svl, scl, pqp, fqe, dvl, nsa, als):
        self.ira = ira # IntentResolutionAgent – natural language only
        self.rapl = rapl # RoleAwareProjectionLayer
        self.svl = svl # SemanticValidationLayer
        self.scl = scl # SemanticControlsLayer
        self.pqp = pqp # PhysicalQueryPlanner
        self.fqe = fqe # FederatedQueryEngine
        self.dvl = dvl # DataVisualizationLanguage
        self.nsa = nsa # NarrativeSynthesisAgent
        self.als = als # AnalyticalLineageStore

    async def run(self, operation: dict, params: dict, claims: dict) -> dict:
        # Input: SMR operation definition + resolved call params + verified JWT claims
        #         (params already resolved by the IRA for natural-language requests)
        # Output: result dict – shape varies by execution_profile (see below)

        # 1. RAPL computes the entitlement projection (row scope + column masks) from the caller's roles
        # 2. SVL validates the request, resolves metrics from the SMR, enforces the projection,
        #    and compiles the Logical Query Plan (LQP)
        # 3. SCL approval (five checks – never skipped) → ALS controls write → PQP → FQE
        # 4. Branch on execution_profile – presentation stages only; the controls pipeline above
        #    is identical for every profile:
        #    data_retrieval – { result_id, rows, schema, pagination }
        #    metric_query   – + DVL display_spec
        #    full_analytical – + DVL + NSA in parallel (PAS when the compliance trigger is active)
        #                    – { result_id, rows, schema, display_spec, narrative, export_requires_lineage }
        # Note: DVL is CPU-bound; asyncio.to_thread prevents it blocking the NSA API call
        ...

```

Drilldown Service

A consumer holding a `result_id` traverses hierarchies without re-specifying the original query:

```

{
  "tool": "drilldown",
  "input": {
    "result_id": "res-20260518-093247",
    "hierarchy": "asset_class_hierarchy",
    "selected_value": "EQUITY"
  }
}

```

The service recovers the parent result's analytical context — operation, filters, hierarchy position — from the ALS; the consumer re-specifies nothing. Entitlements and controls are re-evaluated in full for the derived query (fresh RAPL projection, SVL enforcement, and the SCL's five checks), so a revoked entitlement takes effect immediately, even mid-drill-chain.

```

class DrilldownService:
    def __init__(self, als, rapl, svl, scl, pqp, fqe, dvl, smr):
        self.als = als # AnalyticalLineageStore – fetch original lineage records
        self.rapl = rapl # RoleAwareProjectionLayer – fresh entitlement projection at drilldown time
        self.svl = svl # SemanticValidationLayer – re-enforce the projection on the derived LQP
        self.scl = scl # SemanticControlsLayer – re-run the five checks on the derived query
        self.pqp = pqp # PhysicalQueryPlanner – re-plan the refined sub-queries
        self.fqe = fqe # FederatedQueryEngine – execute the refined federated query via Starburst
        self.dvl = dvl # DataVisualizationLanguage – generate updated display_spec
        self.smr = smr # SemanticMetricsRepository – resolve drill-target metric definitions

    async def execute(self, input: DrilldownInput, claims: dict) -> dict:
        # Input: drilldown request – parent result_id + hierarchy dimension + selected value
        # Output: { result_id, parent_id, rows, display_spec }

        # 1. Fetch original lineage record – recovers the parent LQP and analytical context
        # 2. Clone the parent LQP and append a filter node for the selected hierarchy value
        # 3. Re-run the full pipeline on the derived query: fresh RAPL projection → SVL enforcement
        #    → SCL five checks (hierarchy descent can grow scan volume) → PQP → FQE
        # 4. DVL produces the updated display spec; lineage record written with parent_id linkage
        ...

```

Execution profiles

Each SMR operation carries an `execution_profile` that tells the pipeline executor which presentation stages to invoke after the full deterministic pipeline has run. Profile definitions are in \$MCP Capability Layer.

Resources

Resources are used for **knowledge artifacts** — static reference material, skills definitions, and workflow guides that AI consumers embed in their context to understand how to use the platform effectively. Dynamic data lookups (metric definitions, lineage records, dimension catalogs) are handled by tools, not resources.

```

@mcp.resource("guide://analytics/platform-overview")
async def guide_platform_overview() -> str:
    """What the Analytics Platform does, how it governs queries, and when to use
    each analytical capability. Load this before helping a user with any analytical task."""
    return knowledge_store.get("guide/platform-overview")

@mcp.resource("guide://analytics/analytical-domains")
async def guide_analytical_domains() -> str:
    """Reference guide to the six analytical domains (portfolio, performance, risk,
    regulatory, counterparty, benchmarks) – what each covers, which metrics belong
    to it, and the governance constraints that apply."""
    return knowledge_store.get("guide/analytical-domains")

@mcp.resource("guide://analytics/query-patterns")
async def guide_query_patterns() -> str:
    """Common analytical query patterns with worked examples: time-period comparisons,
    benchmark-relative performance, risk attribution, regulatory ratio queries.
    Use this to choose the right tool and parameter structure for a given user request."""
    return knowledge_store.get("guide/query-patterns")

@mcp.resource("skills://analytics/portfolio-performance-review")
async def skills_portfolio_performance() -> str:
    """Step-by-step skills definition for conducting a portfolio performance review:
    which metrics to query, which dimensions to apply, how to interpret the results,
    and when to drilldown. Designed for AI assistants supporting portfolio managers."""
    return knowledge_store.get("skills/portfolio-performance-review")

@mcp.resource("skills://analytics/risk-analysis")
async def skills_risk_analysis() -> str:
    """Skills definition for risk metric analysis: VaR, tracking error, expected
    shortfall, issuer concentration. Covers query construction, result interpretation,
    threshold context, and escalation patterns."""
    return knowledge_store.get("skills/risk-analysis")

@mcp.resource("skills://analytics/regulatory-reporting")
async def skills_regulatory_reporting() -> str:
    """Skills definition for regulatory metric queries: required dimensions,
    compliance provenance requirements, business justification requirements,
    and how to surface lineage references in regulatory responses. The specific
    regulatory frameworks in force are carried as attributes on the metric
    definitions, not hard-coded into the platform."""
    return knowledge_store.get("skills/regulatory-reporting")

@mcp.resource("guide://compliance/regulatory-overview")
async def guide_regulatory_overview() -> str:
    """Compliance provenance reference: which query types require a business
    justification, what the provenance artifact captures, how the two-signal
    compliance trigger works, and how to explain compliance constraints to users.
    Framework-specific requirements derive from the metric definitions' regulatory
    attributes in the SMR."""
    return knowledge_store.get("guide/compliance-regulatory-overview")

```

Knowledge artifacts are stored in a versioned content store (`knowledge_store`) managed via the Admin API. Administrators can extend or override the default guides and skills definitions. Resources do not require JWT authentication (they contain no user data), but are scoped to the platform's public knowledge surface.

Prompts

Prompts provide pre-built instruction templates that AI consumers can load to anchor their analytical behavior before making tool calls.

```
@mcp.prompt()
async def analytical_assistant() -> str:
    """System prompt for an AI assistant using the Analytics Platform.
    Injects the organization's available metrics and governance constraints."""
    # 1. Read verified claims from authenticated server-side request context
    # 2. Fetch slim metric summary from SMR (id + label + description only – prompt size matters)
    # 3. Return system prompt string – instructs the assistant to use tool results only, never estimate
    ...

@mcp.prompt()
async def regulatory_reporting_assistant() -> str:
    """System prompt for a compliance-focused assistant operating on regulatory metrics.
    Adds regulatory framing and prohibits investment recommendations. The frameworks in
    force are derived from the regulatory attributes on the queried metric definitions."""
    # 1. Read verified claims from authenticated server-side request context
    # 2. Fetch regulatory-domain metric summary from SMR
    # 3. Return system prompt – extends analytical_assistant rules with compliance constraints:
    #    no investment recommendations, cite result_id in every response, explain compliance errors
    ...

if __name__ == "__main__":
    mcp.run(transport="streamable-http", host="0.0.0.0", port=8000)
```

Response Envelope

A successful call returns a JSON object containing the result data, the display specification, the optional narrative, and execution metadata for consumer telemetry:

```
{
  "result_id": "res-20260518-093247",
  "lineage_ref": "lineage/acme-wealth/2026/05/18/res-20260518-093247.json",
  "data": { "schema": [ "..."], "rows": [ "..."] },
  "display_spec": { "type": "chart", "...": "..."},
  "narrative": { "lead": "...", "detail": "...", "anchoredTo": [ "..."] },
  "meta": { "latencyMs": 1187, "cacheHit": false, "rowCount": 4 }
}
```

Consumers branch on `display_spec.type` (`"chart"` → Vega-Lite rendering; `"table"` → data grid). The `narrative` field is present when `features.narrativeSynthesis` is enabled. The platform governs which contract is selected and how it is parameterised; the consumer governs how it is rendered.

Error Handling

All tool call failures return a structured error envelope. The MCP framework serializes Python exceptions as tool error responses; the platform maps exception classes to stable error codes before they leave the service boundary.

```

class AnalyticsError(Exception):
    code: str # stable string identifier consumed by AI clients
    message: str # human-readable; safe to surface to end users

class AuthenticationError(AnalyticsError): code = "AUTH_FAILED"
class AccessDeniedError(AnalyticsError): code = "ACCESS_DENIED"
class OperationNotAvailableError(AnalyticsError): code = "OPERATION_NOT_FOUND"
class MetricNotFoundError(AnalyticsError): code = "METRIC_NOT_FOUND"
class ControlsCeilingExceeded(AnalyticsError): code = "CONTROLS_REJECTED" # data scale or complexity ceiling
class ConcurrentQueryLimitExceeded(AnalyticsError): code = "CAPACITY_LIMIT"
class NarrativeValidationError(AnalyticsError): code = "NARRATIVE_FAILED"
class ClassificationGateError(AnalyticsError): code = "CLASSIFICATION_BLOCKED"

def handle_tool_error(exc: Exception) -> dict:
    # Input: any exception raised inside a tool handler
    # Output: { error: { code, message, result_id } } – safe to return to the AI consumer
    # result_id is always present, even for blocked or failed requests

    # Known AnalyticsError subclasses map to stable error codes (see table above)
    # All other exceptions collapse to INTERNAL_ERROR – detail is never leaked to the consumer
    ...

```

Error code reference table:

Code	Raised by	Consumer action
AUTH_FAILED	JWT validation	Re-authenticate; do not retry with same token
ACCESS_DENIED	RAPL	Inform user; do not retry
OPERATION_NOT_FOUND	SMR	Call <code>list_operations</code> to discover valid operation IDs
METRIC_NOT_FOUND	SMR	Call <code>list_operations</code> to check available metrics
CONTROLS_REJECTED	SCL	Reduce metric count, scan scope, or time range; inform user
CAPACITY_LIMIT	SCL	Retry with exponential back-off
NARRATIVE_FAILED	NSA	Return result without narrative; log for operator review
CLASSIFICATION_BLOCKED	SCL	Inform user the requested metric requires elevated access
INTERNAL_ERROR	Any	Log <code>result_id</code> if available; operator investigation required

Every error response carries a `result_id`, so every request — successful, blocked, or denied — appears in the audit trail and is reachable through the lineage API. The `message` field is human-readable and suitable for surfacing directly to end users or agentic decision loops.

Intent Resolution Agent

Specification: §Intent Resolution Agent

Custom Python code implements the IRA's retrieval and confirmation flow. The DCS search index supplies candidate SMR operations, and Anthropic Claude ranks those candidates, binds request parameters, and scores compliance intent. The application uses the model response to return a resolved request or ask for confirmation.

Decision	Choice	Rationale
Candidate retrieval	Embedding similarity search over SMR operation/metric embeddings	RAG retrieval narrows the full catalog to a handful of candidates before the model ranks them
Intent ranking	Anthropic Claude	Ranks candidate operations and binds parameters from the natural-language query
Confirmation	Candidate cards when intent is ambiguous	The user selects or refines before any query executes
Compliance intent	<code>compliance_purpose_score</code> produced within the same ranking call	Signal 2 of the SCL's two-signal compliance gate; an ambiguous purpose triggers a clarification card rather than a guess
Scope	Natural-language requests only	Structured <code>operation_id</code> + <code>params</code> calls bypass the IRA entirely and declare compliance purpose explicitly

The Intent Resolution Agent (IRA) is the only AI step in the pre-computation pipeline. It receives a natural-language query and the caller's JWT from the MCP layer, retrieves candidate operations from the SMR catalog using embedding similarity search, ranks them with a language model, binds parameters to the leading candidate, and derives a presentation preview. Within the same ranking call it scores whether the stated purpose of the query is compliance-driven — the `compliance_purpose_score` forwarded with the resolved intent and consumed by the SCL's two-signal compliance gate — and clarifies with the user through the confirmation card flow when the purpose is ambiguous. When the top candidate is confident and unambiguous, the resolved intent is forwarded directly to the RAPL; when it is ambiguous, ranked candidate cards are returned to the consumer for selection or conversational refinement. The IRA produces no output visible to the end user once intent is resolved, and makes no governance or execution decisions — those belong to the deterministic pipeline that follows.

```

import anthropic

class IntentResolutionAgent:
    def __init__(self, smr: "SemanticMetricsRepository", client: anthropic.AsyncAnthropic):
        self.smr = smr
        self.client = client

    RANKING_MODEL = "claude-sonnet-4-6"

    async def resolve(self, query: str, claims: dict) -> dict:
        # Input: natural-language query + verified JWT claims
        # Output: resolved intent – { operation_id, params, presentation_hint, compliance_purpose_score }
        #         or a candidate-card set when intent or compliance purpose is ambiguous

        # 1. Embed the query and retrieve top-k candidate operations from the SMR by vector similarity
        # 2. Rank candidates with the language model; bind params to the leading operation;
        #     score compliance intent of the stated purpose in the same call
        # 3. If the top candidate is confident and clear of the runner-up, return the resolved intent → RAPL
        # 4. Otherwise return ranked candidate cards for the consumer to select or refine
        ...

    async def _retrieve_candidates(self, query: str, claims: dict) -> list[dict]:
        # Input: query string + claims (for org scoping)
        # Output: top-k approved operations from the SMR catalog by embedding similarity
        ...

    def _rank_and_bind(self, query: str, candidates: list[dict]) -> dict:
        # Input: query + candidate operations
        # Output: ranked candidates with bound params + confidence scores
        # The model only ranks and binds; it never selects chart types or makes governance decisions
        ...

    def _score_compliance_intent(self, query: str, ranked: dict) -> float:
        # Input: natural-language query + ranked leading candidate
        # Output: float 0.0–1.0 – compliance purpose probability (Signal 2 of the two-signal gate)
        # Scored by the language model within the same ranking call – no separate API call
        # A score near complianceIntentThreshold triggers a clarification card instead of a guess
        ...

```

Structured API consumers that already know the `operation_id` skip the IRA entirely: the MCP layer routes their call straight into the deterministic pipeline at the RAPL.

Request Validation (SVL)

Specification: §Semantic Validation Layer

Pydantic validates the tool input models, and JSON Schema validation checks operation parameters against their registered definitions. Custom Python code performs SMR lookups, enforces the entitlement projection, and builds the LQP. Together, these checks implement the SVL.

Decision	Choice	Rationale
Parameter validation	JSON Schema + Pydantic	Strict schema enforcement against MCP tool input models; structured error responses

Decision	Choice	Rationale
SMR resolution	Direct SMR service call	Synchronous lookup against the metric registry; rejects unregistered IDs before LQP generation
LQP generation	Custom Python	Backend-agnostic DAG construction from validated parameters; deterministic for any given input

`SemanticValidationLayer` runs after the RAPL. It receives the entitlement projection (row scope and column masks) from the RAPL together with the resolved request, validates params, resolves metrics from the SMR, enforces the projection, attaches the IRA's compliance intent score, and delegates DAG construction to `LQPGenerator`. The SVL performs no compliance classification of its own — the `compliance_purpose_score` is produced by the IRA at intent resolution (structured calls, which bypass the IRA, declare compliance purpose explicitly via a `compliance_purpose` parameter); the SVL attaches it to the LQP so the SCL can apply its two-signal gate. The SVL also attaches a Tier-1 `preliminary_impact_estimate` — the sum of the resolved metrics' `performance_impact_weight` values — as a coarse indicator of query weight; the SCL replaces this with a precise `estimated_scan_rows` figure at its data-scale check.

```
class SemanticValidationLayer:
    def __init__(self, smr: "SemanticMetricsRepository"):
        self.smr = smr

    async def resolve(self, operation: dict, params: dict, projection: dict, claims: dict) -> dict:
        # Input:  SMR operation definition + resolved call params + RAPL entitlement projection + JWT claims
        # Output: LQP with compliance_purpose_score, resolved_metrics, and preliminary_impact_estimate
        attached

        # 1. Validate params against operation's required_params – fail fast before any SMR calls
        # 2. Resolve each metric ID from SMR – rejects unknown or non-approved metrics
        # 3. Enforce the RAPL projection – inject row scope filter nodes; embed column masks on the LQP
        # 4. Delegate DAG construction to LQPGenerator (see Logical Query Plan Generation)
        # 5. Attach compliance_purpose_score from the resolved request – scored by the IRA for
        #     natural-language queries; explicit compliance_purpose param for structured calls
        # 6. Attach preliminary_impact_estimate (Σ performance_impact_weight) – Tier-1 coarse estimate
        # 7. Retain resolved_metrics on LQP – SCL needs them for classification and compliance checks
        ...

    def _validate_params(self, params: dict, required: list[str]) -> None:
        # Input:  raw params dict + required field list from SMR operation definition
        # Raises: ValueError listing all missing keys – fast rejection before SMR resolution
        ...
```

Narrative Synthesis Agent

Specification: `SNarrative Synthesis Agent`

Anthropic Claude generates the NSA's narrative text. Custom Python code selects the model, builds the prompt from result values, and validates the returned narrative. Claude Haiku handles simple summaries, and Claude Sonnet handles the more complex cases listed below.

Decision	Choice	Rationale
Provider	Anthropic Claude	Reliable instruction-following for constrained summarization tasks
Standard queries	Claude Haiku	Generates simple metric summaries; latency requires measurement against the intended workload.
Complex queries	Claude Sonnet	Attribution decompositions and multi-portfolio results require richer prose
Prompt construction	Result-only context	Metric labels + row values + units injected; no user query, no physical schema
Post-generation validation	Custom Python	Every numeric value in narrative matched against result set; reject and retry once on failure
Feature flag	<code>features.narrativeSynthesis</code>	Platform-level on/off; disabled means NSA is never invoked

```

import anthropic

class NarrativeSynthesisAgent:
    def __init__(self, client: anthropic.AsyncAnthropic):
        self.client = client

    # Update model IDs on deprecation – or read from platform config models.narrativeSynthesisModel
    FAST_MODEL = "claude-haiku-4-5-20251001"
    STANDARD_MODEL = "claude-sonnet-4-6"

    async def synthesise(self, result: dict, operation: dict) -> str:
        # Input: assembled FQE result + SMR operation definition
        # Output: governed narrative string – all numeric values verified against result rows

        # 1. Select model – Haiku for simple results (≤5 metrics, ≤3 dimensions); Sonnet otherwise
        # 2. Build prompt – injects metric labels, row values, and units; never includes user query or schema
        # 3. Call Anthropic API and extract narrative text
        # 4. Validate numbers in narrative against result rows – retry once on failure
        # 5. Raise NarrativeValidationError if validation fails on both attempts
        ...

    def _is_simple(self, result: dict) -> bool:
        # Input: assembled result with schema field list
        # Output: True if ≤5 metrics and ≤3 dimensions – routes to Haiku
        # Attribution, multi-portfolio, and regulatory queries always route to Sonnet regardless of
count
        ...

    def _build_prompt(self, result: dict, operation: dict) -> str:
        # Input: result rows + operation display_name
        # Output: prompt string – metric labels + values injected; no user query, no physical schema
        # Keeping the prompt free of schema details prevents the model from leaking internal structure
        ...

    def _validate_numbers(self, narrative: str, rows: list[dict]) -> None:
        # Input: generated narrative + result rows
        # Raises: NarrativeValidationError if any numeric value in the narrative is neither present
        # verbatim in the result rows nor a derived count recomputed from them –
        # entity counts ("2 of 4 portfolios") are verified against result cardinalities
        # Purpose: prevents hallucinated figures reaching the consumer
        ...

```

Provenance Artifact Service (PAS)

Specification: \$Provenance Artifact Service

An in-process Python module implements the PAS. The `cryptography` library signs the assembled artifact with ECDSA P-256 and SHA-256. Vault or Kubernetes Secrets supplies the signing key, and the ALS stores the signed artifact in S3 under the configured retention controls.

Decision	Choice	Rationale
Deployment	In-process module within the <code>analytics-mcp</code> service	Invoked only for compliance-purpose queries — low volume; shares the S3 lineage bucket the service already writes to; no extra deployable

Decision	Choice	Rationale
Signing	ECDSA P-256 (SHA-256) via the <code>cryptography</code> library	The concrete algorithm behind the design documents' cryptographic seal; any holder of the published public key can verify independently
Key management	Private key injected from Vault / Kubernetes Secrets; public key published	Key never baked into container images; rotation via <code>key_id</code>
Sealing	Artifact written to S3 as the <code>{result_id}_provenance.json</code> sibling document	Immutable from the moment of writing — same write-once semantics as lineage records
Export gate	<code>export_requires_lineage: true</code> until the S3 write is confirmed	Consumer withholds export affordances until sealing is confirmed

The PAS runs in the parallel presentation-assembly step alongside the DVL and NSA, but only when the SCL's two-signal compliance check is active. It reads the projection, controls decision, and execution records for the current query from the ALS, assembles the Provenance Artifact document, signs it, writes it back to the ALS as an immutable sibling record, and returns the sealed compliance block for inclusion in the MCP tool response.

Hardening note: in this reference implementation the signing key is held in the `analytics-mcp` process. High-assurance deployments can isolate it by splitting the PAS into its own service, or by delegating signing to a cloud KMS/HSM so the private key is never exportable — the interface contract is unchanged either way.

```
class ProvenanceArtifactService:
    def __init__(self, als: "AnalyticalLineageStore", signing_key, key_id: str):
        self.als = als # reads lineage records; writes the sealed sibling document
        self.signing_key = signing_key # ECDSA P-256 private key – injected from Vault, never logged
        self.key_id = key_id # published with the artifact for verification and rotation

    async def seal(self, result_id: str, lqp: dict, compliance: dict) -> dict:
        # Input: result_id + approved LQP + compliance context (signals, intent score, frameworks)
        # Output: sealed compliance block – { compliance_purpose, intent_score, triggered_by_metrics,
        #         triggered_by_frameworks, regulatory_trace_id, artifact_set_version,
        #         export_requires_lineage, classification_ceiling_applied }

        # 1. Fetch the projection, controls decision, and execution records from the ALS
        # 2. Assemble the Provenance Artifact – intent, escalation signals, metric versions,
        #     logical field spec, physical execution detail, entitlement snapshot
        # 3. Sign the canonical serialized artifact – ECDSA-P256-SHA256, signed_fields listed
        # 4. Write {result_id}_provenance.json to the ALS bucket – immutable sibling record
        # 5. Return the compliance block; the export gate stays locked until the write is confirmed
        ...

    def _sign(self, artifact: dict) -> dict:
        # Input: assembled artifact dict
        # Output: signature block – { algorithm, key_id, signed_fields, value, sealed_at }
        ...
```

Semantic Metrics Repository (SMR)

Specification: §Semantic Metrics Repository

The DCS stores and versions SMR definitions, manages their approval workflow, and indexes them for discovery. The Python `SemanticMetricsRepository` class accesses these functions through the DCS API.

Decision	Choice	Rationale
Definition storage	DCS store, sibling to the SDR	Metric and operation definitions held in the SMR — a separate store from the SDR, both within the Data Context Store; no duplicate semantic infrastructure
Authoring and approval	DCS native capabilities	Document creation, versioning, and approval workflow are handled by the existing DCS tooling shared with the SDR — no new write layer needed
Runtime reads	Direct DCS API query by Semantic Validation Layer	Definitions read from the authoritative source at resolution time
Search	DCS native search index	<code>list_operations</code> queries the DCS index directly — no separate search infrastructure

The SMR and the SDR are two independent stores within the Data Context Store (DCS). The SMR holds four document types — `analytical_metric`, `analytical_dimension`, `analytical_operation`, and `analytical_dataset` — while the SDR holds the foundational data definitions. The DCS manages the full document lifecycle (draft → in review → approved → deprecated) for all four SMR types using the same versioned storage, search, and approval capabilities the SDR relies on.

SMR document type: `analytical_metric`

The core metric definition. One document per approved metric version. The `status` field follows the DCS approval lifecycle; the Semantic Validation Layer only resolves documents with `"status": "approved"`.

```
{
  "type": "analytical_metric",
  "org_id": "acme-wealth",
  "metric_id": "var_95",
  "version": 2,
  "status": "approved",
  "source": "platform",
  "display_name": "Value at Risk (95%)",
  "description": "Maximum expected portfolio loss over a 1-day horizon at 95% confidence.",
  "domain": "risk",
  "category": "market_risk",
  "unit": "percentage",
  "decimals": 2,
  "aggregation": "value_weighted_average",
  "weight_metric_id": "market_value",
  "performance_impact_weight": 3,
  "classification_level": "internal",
  "data_affinity": "risk_metrics",
  "physical_mapping": {
    "source": "risk-semantic-layer",
    "cube": "risk_cube",
    "measure": "var_95_daily"
  },
  "formula": "MAX(losses) WHERE confidence_level = 0.95",
  "compliance_relevant": false,
  "regulatory_framework": [],
  "required_dimensions": ["portfolio_id", "as_of_date"],
  "optional_dimensions": ["asset_class", "geography", "sector", "currency"],
  "approved_by": "cdo@acme.com",
  "approved_at": "2026-05-14T09:00:00Z",
  "created_at": "2026-05-13T14:32:00Z"
}
```

`status` is one of `"proposed"` | `"in_review"` | `"approved"` | `"deprecated"` | `"retired"`. The DCS enforces a uniqueness constraint: at most one document per `(org_id, metric_id)` may carry `"status": "approved"` at any point in time. All prior versions are retained as `"deprecated"` for lineage reconstruction. `source` is `"platform"` for Financial Services Reference Model entries and `"custom"` for organization-customized definitions.

`weight_metric_id` is required when `aggregation` is `"value_weighted_average"` (or any other weighted aggregation variant) and must reference the `metric_id` of an approved `analytical_metric` in the SMR. The SVL resolves and validates this reference at query time. If the weight metric is missing or unapproved, the query is rejected. The field is absent for non-weighted aggregations (`"sum"`, `"last"`, `"count"`, `"min"`, `"max"`, `"mean"`). The LQP generator emits a `weight_metric_id` key on the `metric_scan` node so that the execution backend can fetch the weighting values alongside the primary metric.

`formula` stores the business-logic expression defined in the SMR formula language. It is the human-readable and audit-visible definition of what the metric computes. At query time the FQE resolves the formula against the `physical_mapping` to generate the backend-specific query; the formula itself is never executed directly. Metrics backed entirely by a pre-computed measure in a semantic layer (e.g. a Cube.js measure) may leave `formula` as an empty string and rely solely on `physical_mapping`.

SMR document type: `analytical_dimension`

Dimension definitions are the third SMR document type. They define the valid slicing axes referenced in `supported_dimensions` and `required_dimensions` on metrics and operations. The SVL validates dimension IDs against this catalog at resolution time.

```
{
  "type": "analytical_dimension",
  "org_id": "acme-wealth",
  "dimension_id": "asset_class",
  "version": 1,
  "status": "approved",
  "source": "platform",
  "display_name": "Asset Class",
  "description": "Top-level asset class classification applied to holdings.",
  "data_affinity": "portfolio",
  "physical_mapping": { "source": "primary-warehouse", "table": "dim_asset_classification", "column":
"asset_class" },
  "values": ["EQUITY", "FIXED_INCOME", "ALTERNATIVES", "CASH", "DERIVATIVES"],
  "hierarchical": false,
  "parent_dimension": null
}
```

SMR document type: `analytical_operation`

The operation catalog. One document per approved operation. The `execution_profile` field tells the pipeline executor which stages to invoke. The `supported_metrics` and `supported_dimensions` lists are enforced by the Semantic Validation Layer. A `run_analytics` call referencing an out-of-catalog value is rejected before LQP generation.

```
{
  "type": "analytical_operation",
  "org_id": "acme-wealth",
  "operation_id": "retrieve_position_history",
  "version": 1,
  "status": "approved",
  "source": "platform",
  "display_name": "Position History Retrieval",
  "description": "Retrieve governed current or historical position records for authorized
portfolios.",
  "execution_profile": "data_retrieval",
  "dataset_id": "fixed_income_daily_positions",
  "required_params": ["portfolio_scope", "date_from", "date_to"],
  "optional_params": ["asset_class", "requested_fields", "page_size", "page_token"],
  "supported_metrics": [],
  "supported_dimensions": ["portfolio_id", "asset_class", "currency", "instrument_id", "position_date"]
}
```

SMR document type: `analytical_dataset`

The governed dataset contract for bulk retrieval. The `approved_fields` set — with per-field classification — defines exactly which columns a `data_retrieval` operation may return for this dataset; fields above the caller's classification ceiling are excluded at projection time.

```

{
  "type": "analytical_dataset",
  "org_id": "acme-wealth",
  "dataset_id": "fixed_income_daily_positions",
  "version": 1,
  "status": "approved",
  "source": "platform",
  "display_name": "Fixed Income Daily Positions",
  "description": "Daily position and PnL records for fixed income portfolios.",
  "domain": "portfolio",
  "data_affinity": "portfolio",
  "physical_mapping": { "source": "primary-warehouse", "table": "positions_fact" },
  "approved_fields": [
    { "field": "portfolio_id", "classification_level": "internal" },
    { "field": "instrument_id", "classification_level": "internal" },
    { "field": "asset_class", "classification_level": "internal" },
    { "field": "daily_pnl", "classification_level": "internal" },
    { "field": "market_value", "classification_level": "internal" },
    { "field": "duration", "classification_level": "internal" },
    { "field": "currency", "classification_level": "internal" },
    { "field": "position_date", "classification_level": "internal" }
  ],
  "required_dimensions": ["portfolio_id", "position_date"],
  "pagination": { "default_page_size": 10000, "max_page_size": 50000 },
  "refresh_cadence": "daily",
  "approved_by": "cdo@acme.com",
  "approved_at": "2026-05-14T09:00:00Z",
  "created_at": "2026-05-13T14:32:00Z"
}

```

```

class SemanticMetricsRepository:
    def __init__(self, dcs_client):
        self.dcs = dcs_client  # SMR documents are read through the DCS API

    async def get_operation(self, operation_id: str, claims: dict) -> dict:
        # Input:  operation_id string + JWT claims (for org_id scoping)
        # Output: approved analytical_operation document from the SMR (via the DCS API)
        # Raises: OperationNotAvailableError if not found or status != "approved"
        ...

    async def list_operations(self, claims: dict, domain: str | None = None) -> list[dict]:
        # Input:  JWT claims + optional domain filter
        # Output: list of approved analytical_operation documents for the caller's org
        ...

    async def get_metric(self, metric_id: str, claims: dict) -> dict:
        # Input:  metric_id string + JWT claims
        # Output: approved analytical_metric document – includes physical_mapping and compliance fields
        # Raises: MetricNotFoundError if not found or not approved
        ...

    async def list_metrics(self, claims: dict, **filters) -> list[dict]:
        # Input:  JWT claims + arbitrary SDR filter kwargs (status, domain, etc.)
        # Output: list of matching analytical_metric documents for the caller's org
        ...

    async def list_approved_summary(self, claims: dict, domain: str | None = None) -> list[dict]:
        # Input:  JWT claims + optional domain filter
        # Output: slim list – [{ id, label, description }] – injected into prompt context
        # Intentionally minimal: prompt context size matters; full definitions are available via get_metric
        ...

```

Logical Query Plan Generation (SVL)

The MCP tool call JSON (metric IDs, dimension IDs, time period, filters) is the analytical intent representation. The SVL validates these parameters, resolves metrics from the SMR, applies the RAPL entitlement projection, and constructs the LQP DAG.

MCP input example

```

{
  "tool": "run_analytics",
  "input": {
    "operation_id": "compare_portfolio_to_benchmark",
    "params": {
      "portfolio_ids": ["GLOB_EQ_OPP", "UK_CORE_INC"],
      "metrics":      ["portfolio_return", "tracking_error"],
      "time_period":  "quarter_to_date"
    }
  }
}

```

The Semantic Validation Layer resolves metric IDs against the SMR, enforces the RAPL entitlement projection, and emits a platform-agnostic LQP. The LQP carries pinned metric definition versions, expanded time ranges, row scope filters, and

a Tier-1 `preliminary_impact_estimate` for governance validation; physical mappings are resolved later by the PQP from the SMR, keyed on the pinned versions.

LQP output example

```
{
  "lqp_id": "lqp-20260518-093243",
  "org_id": "acme-wealth",
  "nodes": [
    {
      "id": "node-1",
      "op": "metric_scan",
      "metric_id": "portfolio_return",
      "metric_version": "2.1.0",
      "aggregation": "value_weighted_average",
      "data_affinity": "portfolio"
    },
    {
      "id": "node-2",
      "op": "metric_scan",
      "metric_id": "tracking_error",
      "metric_version": "1.3.0",
      "aggregation": "value_weighted_average",
      "data_affinity": "risk_metrics"
    },
    {
      "id": "node-3",
      "op": "join",
      "inputs": ["node-1", "node-2"],
      "join_keys": ["portfolio_id", "date"]
    },
    {
      "id": "node-4",
      "op": "filter",
      "input": "node-3",
      "predicates": [
        "portfolio_id IN ('GLOB_EQ_OPP', 'UK_CORE_INC')",
        "asset_class = 'EQUITY'"
      ]
    },
    {
      "id": "node-5",
      "op": "time_expand",
      "input": "node-4",
      "period": "quarter_to_date",
      "as_of_date": "2026-05-14",
      "resolved_range": { "from": "2026-04-01", "to": "2026-05-14" }
    },
    {
      "id": "node-6",
      "op": "sort",
      "input": "node-5",
      "by": [{ "field": "portfolio_return", "direction": "desc" }]
    }
  ],
  "preliminary_impact_estimate": 850,
  "column_masks": [],
  "row_scope_applied": true
}
```

```

from datetime import date, timedelta

class LQPGenerator:
    def build(self, operation: dict, params: dict, metrics: list[dict], claims: dict) -> dict:
        # Input:  SMR operation + validated params + resolved metric documents + JWT claims
        # Output: LQP dict – { lqp_id, org_id, nodes: [...], output: terminal_node_id }

        # 1. Emit one metric_scan node per resolved metric – carries metric version, data affinity, and
        aggregation
        # 2. If metrics span >1 node, emit a join node – join keys inferred from shared required_dimensions
        # 3. Emit filter node from params["filters"] if present
        # 4. Emit time_expand node if time_period or as_of_date in params – resolves symbolic period to date
        range
        # 5. Emit sort node from params["sort"] or operation["default_sort"] if present
        # Note: "output" points to the terminal node – PQP reads this to find the plan entry point
        ...

    def _infer_join_keys(self, metrics: list[dict]) -> list[str]:
        # Input:  list of resolved metric documents
        # Output: intersection of required_dimensions across all metrics – safe shared join keys
        # Raises: ValueError if no shared dimensions exist – co-queried metrics must share at least one
        ...

    def _render_predicate(self, f: dict) -> str:
        # Input:  filter dict – { dimension, operator, value }
        # Output: SQL predicate string – e.g. "asset_class IN ('EQUITY', 'FIXED_INCOME')"
        ...

    def _resolve_time(self, params: dict) -> dict:
        # Input:  params with time_period and/or as_of_date
        # Output: { from: ISO date, to: ISO date } – concrete range for time_expand node
        # Handles: month_to_date, quarter_to_date, year_to_date, trailing_12m; defaults to point-in-time
        ...

```

Role-Aware Projection Layer

Specification: §Role-Aware Projection Layer

Custom Python middleware implements the RAPL. It uses `asyncpg` to read policies from the PostgreSQL DES, combines those policies with verified identity claims, and produces the entitlement projection for the SVL.

Decision	Choice	Rationale
Implementation	Custom middleware (Python)	Thin, stateless; computes the entitlement projection before the LQP is compiled
Role resolution	JWT claim extraction + DES role definition lookup	Role claim field name is configurable
Policy store (DES)	The PostgreSQL <code>role_policies</code> schema acts as the Data Entitlements Store in the reference implementation	Dedicated schema and credentials, logically separate from platform data; written only by the Entitlements Manager — not writable via the platform Admin API
Row scope	Typed predicate binding in Python	Binds verified claim values to allowlisted logical fields for the SVL to include in the plan.

Decision	Choice	Rationale
Column protection	Exclusions and deterministic masks are compiled into the PQP and executed before protected values leave the governed query boundary	Prevents unmasked values from entering the application process, cache, telemetry, or lineage store
Default policy	Deny-by-default — fixed, not configurable	No access unless a matching role definition is found; an architectural property (P5), not a setting

Role policies schema

In the reference implementation, the PostgreSQL `role_policies` schema acts as the **Data Entitlements Store (DES)**. The schema carries its own credentials and is logically separate from platform data even when physically co-located; it is written only by the Entitlements Manager and is not writable through the platform Admin API. Organizations with an existing entitlement system substitute it behind the same role-definition read interface. Each document maps directly to the following JSON shape:

```
{
  "role_id": "regional_analyst",
  "org_id": "acme-wealth",
  "data_access_domains": ["portfolio", "risk"],
  "classification_ceiling": "internal",
  "allowed_metrics": null,
  "denied_metrics": ["var_99", "expected_shortfall"],
  "allowed_dimensions": null,
  "denied_dimensions": ["issuer"],
  "row_scope": {
    "portfolio": { "operator": "in", "field": "portfolio_id", "claim": "managed_portfolios" }
  },
  "column_masks": {
    "aum": {
      "condition": "{user.roles} NOT CONTAINS 'senior_analyst'",
      "action": "null_replacement",
      "replacement": null
    },
    "client_id": {
      "condition": "always",
      "action": "hash_replacement",
      "replacement": null
    }
  },
  "created_at": "2026-05-14T09:00:00Z",
  "updated_at": "2026-05-14T09:00:00Z"
}
```

Field reference:

Field	Type	Description
<code>role_id</code>	string	Matches the role name in the JWT <code>analytics_roles</code> claim
<code>org_id</code>	string	Organization scope — one policy per org per role
<code>data_access_domains</code>	array	Data domains the role may access; requests outside them are DENIED (<code>DATA_NOT_ENTITLED</code>)

Field	Type	Description
<code>classification_ceiling</code>	string	Highest classification level the role may touch; metrics and fields above it are denied or excluded
<code>allowed_metrics</code>	array null	Null = all metrics permitted; array = explicit allowlist
<code>denied_metrics</code>	array	Metric IDs denied regardless of <code>allowed_metrics</code> (<code>METRIC_NOT_ENTITLED</code>)
<code>allowed_dimensions</code>	array null	Null = all dimensions permitted; array = explicit allowlist
<code>denied_dimensions</code>	array	Dimension IDs denied regardless of <code>allowed_dimensions</code> (<code>DIMENSION_NOT_ENTITLED</code>)
<code>row_scope</code>	object	Key = dimension name; value = a typed predicate referencing an allowlisted logical field and a required verified claim
<code>column_masks</code>	object	Key = field name; value = mask rule with <code>action</code> : one of <code>null_replacement</code> , <code>redacted_label</code> , <code>excluded</code> , <code>hash_replacement</code>

```

class RoleAwareProjectionLayer:
    def __init__(self, pg_pool, config: dict = None):
        self.pg = pg_pool
        self.config = config or {} # platform config – used for roleClaimField lookup

    async def project(self, request: dict, claims: dict) -> dict:
        # Input: resolved request (operation + params) + JWT claims
        # Output: entitlement projection – { data_access_domains, classification_ceiling,
        #         metric_access, dimension_access, row_scope, column_masks }
        #         consumed by the SVL, which enforces it while compiling the LQP

        # 1. Extract analytics_roles from claims – roleClaimField is configurable
        # 2. Load a role policy for each role – raises AccessDeniedError if none found (deny-by-default – not
        configurable)
        # 3. Merge policies – row scope intersected; column masks unioned
        # 4. Bind verified claim values to typed predicates; missing claims deny the request
        # 5. Return the projection – the SVL injects the row scope nodes and embeds the column masks
        ...

    def _merge_policies(self, policies: list[dict]) -> dict:
        # Input: list of role policy documents for all of the user's roles
        # Output: merged policy – { data_access_domains, classification_ceiling,
        #         metric_access, dimension_access, row_scope, column_masks }

        # Data domains, metric access, dimension access: union – entitlement via any role suffices
        # Classification ceiling: highest ceiling across the user's roles
        # Row scope: strict AND – every condition from every role is applied; where two roles
        # constrain the same dimension, their value sets intersect (most restrictive wins,
        # independent of role order)
        # Column masks: union – masked by any role means masked for the user
        ...

    def _resolve_row_scope(self, policy: dict, claims: dict) -> list[dict]:
        # Input: merged policy + claims (for template interpolation)
        # Output: list of resolved row scope conditions for the SVL to inject as filter nodes
        ...

    def _bind_predicate(self, predicate: dict, claims: dict) -> dict:
        # Input: typed predicate referencing an allowlisted logical field and verified claim
        # Output: typed bound predicate compiled later with query parameters – never SQL text
        # Missing claims, unsupported operators, and unknown fields fail closed
        ...

    async def _load_policy(self, org_id: str, role: str) -> dict | None:
        # Input: org_id + role name
        # Output: role_policies row as dict, or None if no policy exists for this role
        ...

```

Semantic Controls Layer

Specification: §Semantic Controls Layer

Custom Python rules implement the SCL's five controls checks. The DCS stores control thresholds, the SDR supplies profiling statistics for scan estimates, and Redis coordinates concurrent-query admission across application instances.

Decision	Choice	Rationale
Implementation	Custom rules engine (Python)	Deterministic; config-driven; no ML inference
Five checks	Data scale · complexity · classification gate · compliance · concurrency	Every query passes all five before release to the PQP
Data-scale estimation	Tier-2 <code>estimated_scan_rows</code> from SDR profiling statistics	Precise scan volume computed from row counts, partition sizes, and time-series distributions; replaces the SVL's Tier-1 <code>preliminary_impact_estimate</code>
Config store	DCS document store — <code>controls_config</code> document type	Platform-level thresholds stored as a JSON document alongside SMR documents

Concurrent query enforcement uses a Redis-backed semaphore rather than an in-process counter, ensuring the limit applies across all running pods. The implementation acquires the concurrency slot first, as a cheap admission control before computing scan estimates — check evaluation order is an implementation detail; all five outcomes are recorded in the controls decision record regardless.

The platform has one controls config document. The Semantic Controls Layer reads it at startup and refreshes it on change events from the DCS:

```
{
  "type": "controls_config",
  "org_id": "acme-wealth",
  "maxScanRows": 50000000,
  "maxMetricsPerQuery": 10,
  "maxDimensions": 5,
  "maxJoinDepth": 4,
  "classificationGate": true,
  "blockedClassifications": ["TOP_SECRET", "RESTRICTED"],
  "maxConcurrentQueries": 20,
  "queryTimeoutSeconds": 60,
  "requireLineageForExport": true,
  "auditAllQueries": true,
  "complianceIntentThreshold": 0.8
}
```

Compliance is always active and evaluated per request — there is no platform on/off switch. The `complianceIntentThreshold` only tunes the sensitivity of the second signal.

```

class SemanticControlsLayer:
    def __init__(self, sdr_client, pg_pool, redis_client):
        self.sdr = sdr_client # DCS/SDR client – reads profiling statistics for the data-scale check
        self.pg = pg_pool
        self.redis = redis_client

    async def _acquire_query_slot(self, org_id: str, config: dict) -> None:
        # Input: org_id + controls config (for maxConcurrentQueries and timeout)
        # Raises: ConcurrentQueryLimitExceeded if the org is at its concurrent query ceiling
        # Uses Redis INCR as a cross-pod atomic counter – safe under horizontal scaling
        ...

    async def _release_query_slot(self, org_id: str) -> None:
        # Decrements the Redis counter – called in the except block of approve() to release on failure
        ...

    async def approve(self, lqp: dict, claims: dict) -> dict:
        # Input: SVL-validated LQP (carries preliminary_impact_estimate) + JWT claims
        # Output: LQP with controls_approved: true and estimated_scan_rows attached
        # Raises: one of ControlsCeilingExceeded, ClassificationGateError, ConcurrentQueryLimitExceeded

        # The five sequential checks:
        # 1. Load controls config for the org
        # 2. Concurrency – acquire Redis query slot; ConcurrentQueryLimitExceeded if at ceiling
        # 3. Data scale – compute estimated_scan_rows from SDR profiling; ControlsCeilingExceeded if >
maxScanRows
        # 4. Complexity – node count and join depth vs limits; ControlsCeilingExceeded if exceeded
        # 5. Classification gate – ClassificationGateError if any metric is in blocked_classifications
        # 6. Compliance – two-signal trigger; escalates to Provenance Artifact if both signals active
        # Note: all checks are inside try/except so the semaphore slot is always released on failure
        ...

    def _estimate_scan_rows(self, lqp: dict, config: dict) -> int:
        # Input: LQP with resolved_metrics and nodes
        # Output: Tier-2 estimated_scan_rows – compared against config["maxScanRows"]

        # Computed from SDR profiling statistics rather than a static weight:
        # - base table row counts for each metric_scan node's physical_mapping
        # - partition sizes and the resolved time range from the time_expand node
        # - time-series volume distributions for the requested period
        # This precise figure replaces the SVL's Tier-1 preliminary_impact_estimate on the LQP.
        ...

    def _check_complexity(self, lqp: dict, config: dict) -> None:
        # Input: LQP nodes + controls config
        # Raises: ControlsCeilingExceeded if node count, join depth, or number of federated catalogs exceeds
limits
        ...

    def _check_classification(self, lqp: dict, config: dict) -> None:
        # Input: LQP with resolved_metrics + controls config with blocked_classifications list
        # Raises: ClassificationGateError if any metric's classification_level is in blocked_classifications
        ...

    def _check_compliance(self, lqp: dict, claims: dict, config: dict) -> dict:
        # Input: LQP (with compliance_purpose_score – scored by the IRA, attached by the SVL) + controls
config
        # Output: LQP with a compliance block attached

        # Two-signal gate (always evaluated – no platform on/off switch):

```

```

# Signal 1 – any resolved metric has compliance_relevant: true
# Signal 2 – compliance_purpose_score >= complianceIntentThreshold (default 0.8)
# Both signals active → compliance_purpose = true: Provenance Artifact required,
# triggered_by_frameworks derived from metric regulatory_framework tags,
# cache bypassed, export gated until lineage sealed
# Either signal absent → compliance_purpose = false (normal query path)
...

async def _load_config(self, org_id: str) -> dict:
    # Input: org_id
    # Output: controls_config document from the DCS – thresholds, classification gate, compliance
threshold
    ...

```

Physical Query Planner (PQP)

Specification: \$Physical Query Planner

Apache Calcite provides the relational planning functions used by the PQP. Because Calcite is JVM-based and the governed application is Python, the reference build runs a small `calcite-planner` sidecar in the same Kubernetes pod as `analytics-mcp`. The Python PQP resolves approved mappings and sends a typed relational-plan request over a loopback-only HTTP interface. The adapter returns Trino SQL and a digest of the canonical planner input. It has no source credentials and cannot execute the query.

Decision	Choice	Rationale
Implementation	Python PQP client plus JVM-based Apache Calcite sidecar	Makes the cross-runtime boundary explicit while keeping the planner inside the reference build's deployment unit.
Planner interface	Versioned JSON over loopback HTTP	Accepts only the approved relational operators and logical references defined by the PQP contract; inaccessible outside the pod.
Catalog binding	SMR <code>physical_mapping</code> lookup → Starburst catalog name	The PQP resolves each node's <code>physical_mapping</code> from the SMR, keyed on the pinned metric version, then binds <code>source</code> → catalog
Output	A single federated Trino SQL statement	Starburst performs the cross-source join natively; no per-backend decomposition needed
Execution	None	The PQP has no backend connectivity; it hands the federated SQL to the FQE (Starburst)

For each `metric_scan` node, the PQP reads the `physical_mapping` from the pinned SMR metric version and binds it to a Starburst catalog reference (`catalog.schema.table`). The relational tree includes scans, joins, filters, time expansion, and sorting. The PQP adds row-scope filters, dimension filters, and column-mask directives before emitting Trino SQL.

In this reference implementation, Starburst divides the federated query into source-specific execution work and performs cross-source joins. The application submits a single SQL statement. This allocation places the source-specific planning described in Section 2 within Starburst. The PQP supplies SQL to the FQE and does not execute it.

PQP input — approved LQP

The PQP resolves each metric node's `physical_mapping` from the SMR (keyed on the pinned metric version) and binds its `source` to a Starburst catalog:

```
{
  "lqp_id": "lqp-20260518-093243",
  "org_id": "acme-wealth",
  "nodes": [
    {
      "id": "node-1", "op": "metric_scan",
      "metric_id": "portfolio_return", "metric_version": "2.1.0",
      "aggregation": "value_weighted_average", "weight_metric_id": "market_value",
      "data_affinity": "portfolio"
    },
    {
      "id": "node-2", "op": "metric_scan",
      "metric_id": "tracking_error", "metric_version": "1.3.0",
      "aggregation": "value_weighted_average", "weight_metric_id": "market_value",
      "data_affinity": "risk_metrics"
    },
    { "id": "node-3", "op": "join", "inputs": ["node-1", "node-2"], "join_keys": ["portfolio_id",
"date"] },
    { "id": "node-4", "op": "filter", "input": "node-3",
      "predicates": ["portfolio_id IN ('GLOB_EQ_OPP', 'UK_CORE_INC')", "asset_class = 'EQUITY'"] },
    { "id": "node-5", "op": "time_expand", "input": "node-4",
      "period": "quarter_to_date", "resolved_range": { "from": "2026-04-01", "to": "2026-05-14" } },
    { "id": "node-6", "op": "sort", "input": "node-5",
      "by": [{ "field": "portfolio_return", "direction": "desc" }] }
  ],
  "estimated_scan_rows": 412000,
  "governance_approved": true,
  "row_scope_applied": true,
  "column_masks": []
}
```

The two metrics resolve to different sources — `portfolio_return`'s `primary-warehouse` source maps to the `snowflake` catalog, `tracking_error`'s `risk-semantic-layer` source maps to the `risk` catalog (the `physical_mapping.source` → catalog map is configured at deployment) — so the emitted statement references both catalogs and lets Starburst perform the join.

```

class PhysicalQueryPlanner:
    def __init__(self, smr: "SemanticMetricsRepository", catalog_map: dict):
        self.smr = smr
        self.catalog_map = catalog_map # physical_mapping.source → Starburst catalog name

    def plan(self, lqp: dict) -> dict:
        # Input: SCL-approved LQP
        # Output: federated Trino query – { federated_sql, catalogs_referenced, column_masks,
        query_timeout_seconds }

        # 1. Resolve each metric_scan node's physical_mapping from the SMR (pinned metric version),
        # then map physical_mapping.source → Starburst catalog
        # 2. Build the versioned planner request from approved operators and bound catalog references
        # 3. Call the loopback Calcite adapter with a deadline and verify its input digest
        # 4. Receive one Trino-dialect SQL statement – Starburst performs the cross-catalog join
        ...

    def _catalog_for(self, physical_mapping: dict) -> str:
        # Maps physical_mapping.source → configured Starburst catalog name
        ...

    def _emit_trino_sql(self, planner_request: dict) -> str:
        # Versioned request → loopback Calcite adapter → Trino SQL
        ...

```

PQP output — federated Trino SQL

```

{
  "lqp_id": "lqp-20260518-093243",
  "engine": "starburst",
  "catalogs_referenced": ["snowflake", "risk"],
  "column_masks": [],
  "query_timeout_seconds": 60,
  "federated_sql": "SELECT p.portfolio_id, p.portfolio_return, r.tracking_error FROM
snowflake.analytics.fact_portfolio_daily p JOIN risk.metricflow.tracking_error r ON p.portfolio_id =
r.portfolio_id AND p.date = r.date WHERE p.portfolio_id IN ('GLOB_EQ_OPP', 'UK_CORE_INC') AND p.asset_class =
'EQUITY' AND p.date BETWEEN DATE '2026-04-01' AND DATE '2026-05-14' GROUP BY p.portfolio_id ORDER BY
p.portfolio_return DESC"
}

```

The PQP passes the federated Trino SQL to the FQE.

Federated Query Engine (FQE)

Specification: §Federated Query Engine

Decision	Choice	Rationale
Engine	Starburst (Trino)	A mature federation engine with an ANSI-SQL surface and native connectors; performs cross-source joins and predicate/aggregate push-down without bespoke code
Federation	One federated Trino SQL statement over multiple catalogs	Starburst plans and executes the cross-source join — no application-level fan-out or per-backend adapters to maintain

Decision	Choice	Rationale
Client	Python Trino client	Submits the PQP's federated SQL to the Starburst coordinator and streams typed rows
Result handling	Custom (Python)	Verifies the protected result schema, caches by LQP signature, and writes the lineage record

Starburst acts as the FQE in the reference implementation. The Python Trino client submits SQL from the PQP to the Starburst coordinator. Starburst plans and executes the query across configured catalog connectors. Custom Python code verifies the returned schema against the entitlement projection, caches the result, and writes the execution record.

The PQP compiles row predicates, column exclusions, and supported deterministic masks into the physical query so protected raw values do not enter the application result assembler. Unsupported masks fail closed during planning rather than being deferred to post-execution processing.

FQE input — federated Trino SQL

The FQE receives the federated Trino SQL produced by the PQP (see *PQP output* above) — a single statement referencing every catalog the query touches. It submits the statement to Starburst, which plans and executes the cross-catalog join.

FQE output — assembled result

After Starburst executes the federated query, the FQE returns a typed result envelope in parallel to the Data Visualization Language (DVL) and Narrative Synthesis Agent:

```
{
  "result_id": "res-20260518-093247",
  "lqp_id": "lqp-20260518-093243",
  "org_id": "acme-wealth",
  "cache_hit": false,
  "latency_ms": 1243,
  "scan_rows": 408517,
  "engine": "starburst",
  "catalogs_used": ["snowflake", "risk"],
  "schema": [
    { "field": "portfolio_id", "type": "string" },
    { "field": "portfolio_return", "type": "number", "unit": "percentage", "decimals": 2 },
    { "field": "tracking_error", "type": "number", "unit": "percentage", "decimals": 2 }
  ],
  "rows": [
    { "portfolio_id": "GLOB_EQ_OPP", "portfolio_return": 4.21, "tracking_error": 3.18 },
    { "portfolio_id": "UK_CORE_INC", "portfolio_return": 2.87, "tracking_error": 1.94 }
  ],
  "executed_sql": "SELECT p.portfolio_id, p.portfolio_return, r.tracking_error FROM
snowflake.analytics.fact_portfolio_daily p JOIN risk.metricflow.tracking_error r ON p.portfolio_id =
r.portfolio_id AND p.date = r.date WHERE p.portfolio_id IN ('GLOB_EQ_OPP','UK_CORE_INC') AND p.asset_class =
'EQUITY' AND p.date BETWEEN DATE '2026-04-01' AND DATE '2026-05-14' GROUP BY p.portfolio_id ORDER BY
p.portfolio_return DESC"
}
```

Data Source Registration

Execution backends are registered through the Admin API by the Integration Engineer. The registration record is the logical input — type, endpoint, authentication mode, capabilities, and the logical data affinities the backend serves; the platform materializes it as a Starburst catalog properties file plus an entry in the `physical_mapping.source` → catalog map.

```
{
  "executionBackends": [
    {
      "id": "primary-warehouse", "name": "Primary Data Warehouse",
      "type": "sql_warehouse",
      "endpoint": "https://internal.api/analytics/backends/warehouse",
      "authType": "service-account",
      "capabilities": ["aggregate", "filter", "join", "window", "timeseries"],
      "dataAffinity": ["portfolio", "positions", "transactions", "benchmarks"],
      "priority": 1, "enabled": true
    },
    {
      "id": "risk-semantic-layer", "name": "Risk Metrics Semantic Layer",
      "type": "semantic_layer",
      "endpoint": "https://internal.api/analytics/backends/risk",
      "authType": "service-account",
      "capabilities": ["metric", "dimension", "filter"],
      "dataAffinity": ["risk_metrics", "performance_attribution"],
      "priority": 1, "enabled": true
    }
  ]
}
```

Field	Required	Description
<code>type</code>	Yes	Source type — <code>sql_warehouse</code> , <code>opendata_api</code> , <code>graph_api</code> , <code>semantic_layer</code> , <code>olap_engine</code> , <code>custom</code> — selects the Starburst connector
<code>authType</code>	Yes	<code>service-account</code> (platform-held credential), <code>api-key</code> , or <code>bearer</code> (forward the caller's token to the backend)
<code>dataAffinity</code>	Yes	Logical data domains this backend serves — sub-plans whose metric <code>data_affinity</code> matches are routed here
<code>capabilities</code>	Yes	Operations that may be routed to this backend: <code>aggregate</code> , <code>filter</code> , <code>join</code> , <code>window</code> , <code>timeseries</code> , <code>metric</code> , <code>traverse</code>
<code>priority</code>	No	Selection order when multiple backends share a <code>dataAffinity</code>

Multiple backends may share a `dataAffinity` ; selection uses `priority` , `capabilities` , and availability. With `authType: "bearer"` the caller's token is forwarded and backend-layer enforcement becomes the source system's responsibility — the platform's row-scope injection still applies upstream.

Starburst and Trino integration routes

The following table is a candidate capability matrix, not a claim that every combination is available through an unmodified connector. Before a source is admitted to the reference build, the team must verify the selected Starburst or Trino distribution, connector edition, authentication method, data-type mapping, predicate and aggregation push-down,

cancellation behavior, and licensing terms. Sources without a validated connector require an approved adapter or remain out of scope.

Each registered source is exposed to Starburst as a catalog. The reference deployment configures at least the following connector types; any Trino-compatible connector may be added:

Catalog type	Starburst connector	Sources
SQL warehouse / lakehouse	Snowflake · BigQuery · Databricks/Delta · Redshift · Iceberg · Hive	Primary performance and position data
Semantic layer	Approved JDBC, REST, or custom Trino adapter for dbt Semantic Layer or Cube	Pre-modeled governed metrics; not assumed to be a built-in connector.
Relational	PostgreSQL · MySQL	Reference and governance data
Graph	Validated commercial or custom connector for Neo4j or Amazon Neptune	Relationship and counterparty data; admitted only after capability testing.
REST / OpenData	Validated commercial or custom connector for REST or OData v4	Reference data and third-party feeds; admitted only after schema, paging, and push-down testing.
Custom	Any Trino-compatible connector	Proprietary or specialized sources

A catalog is registered with a standard Starburst catalog properties file — one per source — and becomes addressable as `catalog.schema.table` in the federated SQL the PQP emits:

```
# etc/catalog/snowflake.properties — one catalog per registered source
connector.name=snowflake
connection-url=jdbc:snowflake://acme.snowflakecomputing.com
connection-user=${ENV:SNOWFLAKE_USER}
connection-password=${ENV:SNOWFLAKE_PASSWORD}
```

FQE implementation

```
from trino.dbapi import connect

class FederatedQueryEngine:
    def __init__(self, starburst_dsn: dict, lineage_store: "AnalyticalLineageStore", cache: "ResultCache"):
        self.dsn = starburst_dsn # Starburst coordinator host/port/user + default catalog
        self.lineage = lineage_store
        self.cache = cache

    async def execute(self, plan: dict, lqp: dict, claims: dict) -> dict:
        # Input: PQP federated Trino query (plan["federated_sql"]) + LQP metadata + JWT claims
        # Output: assembled result — { result_id, rows, schema, cache_hit, catalogs_used, ... }

        # 1. Cache read — return cached result if available; compliance queries always bypass
        # 2. Submit plan["federated_sql"] to the Starburst coordinator via the Trino client
        #    Starburst federates across catalogs, pushes down predicates, performs cross-source joins
        # 3. Stream typed rows; verify the returned schema against the entitlement projection
        # 4. Cache write — store the assembled result with TTL
        # 5. Write execution record to ALS — engine, catalogs_used, executed_sql, latency, scan_rows
        ...

    def _verify_protected_schema(self, schema: list[dict], lqp: dict) -> None:
        # Fail closed if a denied or unmasked protected field is returned
        ...
```

Data Visualization Language (DVL)

Specification: §Data Visualization Language (DVL)

Custom Python code selects the DVL chart contract and generates the display specification. Vega-Lite v5 supplies the chart grammar; the platform defines a separate table format. Chart selection remains the responsibility of the Python contract evaluator.

Decision	Choice	Rationale
Chart spec	Vega-Lite v5 JSON	Industry-standard chart grammar; wide ecosystem for web, server-side, and image rendering
Table spec	Platform-defined <code>type: "table"</code> extension	Vega-Lite has no native table mark; same <code>data + columns</code> convention
Color palette	Platform theme configuration	Not part of the chart contract — deterministic within a deployment, brandable across deployments

Data Visualization Language (DVL) input

The ontology evaluator receives the FQE assembled result alongside the resolved intent pattern. It uses the result schema and intent pattern to select the best-matching chart contract:

```
{
  "intent_pattern": "COMPARISON",
  "schema": [
    { "field": "portfolio_id", "type": "string" },
    { "field": "portfolio_return", "type": "number", "unit": "percentage" },
    { "field": "tracking_error", "type": "number", "unit": "percentage" }
  ],
  "rows": [
    { "portfolio_id": "GLOB_EQ_OPP", "portfolio_return": 4.21, "tracking_error": 3.18 },
    { "portfolio_id": "UK_CORE_INC", "portfolio_return": 2.87, "tracking_error": 1.94 }
  ],
  "sort": { "field": "portfolio_return", "direction": "desc" },
  "allowed_chart_types": ["bar", "line", "scatter", "heatmap", "treemap", "table"]
}
```

Data Visualization Language (DVL) output — DVL display spec

The evaluator matches the `COMPARISON` intent pattern and two-metric schema to the `BAR_MULTI_SERIES_COMPARISON` contract and emits a Vega-Lite v5 DVL display spec:

```
{
  "type": "chart",
  "contract": "BAR_MULTI_SERIES_COMPARISON",
  "mark": "bar",
  "data": {
    "values": [
      { "portfolio_id": "GLOB_EQ_OPP", "metric": "Portfolio Return", "value": 4.21 },
      { "portfolio_id": "GLOB_EQ_OPP", "metric": "Tracking Error", "value": 3.18 },
      { "portfolio_id": "UK_CORE_INC", "metric": "Portfolio Return", "value": 2.87 },
      { "portfolio_id": "UK_CORE_INC", "metric": "Tracking Error", "value": 1.94 }
    ]
  },
  "encoding": {
    "x": { "field": "portfolio_id", "type": "nominal", "title": "Portfolio", "sort": "-y" },
    "y": { "field": "value", "type": "quantitative", "title": "Value (%)", "axis": {
      "format": ".2f" } },
    "color": { "field": "metric", "type": "nominal", "title": "Metric" }
  },
  "colorScheme": ["#003f5c", "#bc5090"],
  "formatHints": {
    "portfolio_return": { "format": ".2%", "unit": "%" },
    "tracking_error": { "format": ".2%", "unit": "%" }
  },
  "interactions": ["click:drilldown", "hover:tooltip", "select:multi-point"]
}
```

Full DVL examples including the `type: "table"` spec are in Structured Response. Full chart contract definitions are in Data Visualization Language (DVL).

```

INTENT_CONTRACTS = {
    ("ATTRIBUTION", 1): "WATERFALL_ATTRIBUTION",
    ("COMPARISON", 2): "BAR_MULTI_SERIES_COMPARISON",
    ("TREND", 1): "LINE_TIME_SERIES_TREND",
    ("DISTRIBUTION", 1): "HISTOGRAM_DISTRIBUTION",
}

class DataVisualizationLanguage:
    def evaluate(self, result: dict, operation: dict) -> dict:
        # Input: assembled FQE result + SMR operation definition
        # Output: DVL display spec – Vega-Lite v5 JSON for charts, platform table spec for tabular results

        # 1. Infer intent pattern from operation's default_visualization field
        # 2. Match intent + metric count to a named chart contract
        # 3. Build display spec for the matched contract – TABLE_GOVERNED is the safe fallback for any
        # unmatched case
        ...

    def _infer_intent(self, operation: dict) -> str:
        # Input: SMR operation definition
        # Output: intent pattern string – ATTRIBUTION, COMPARISON, TREND, or DISTRIBUTION
        # (no match → the TABLE_GOVERNED fallback applies downstream)
        # Derived from operation["default_visualization"] – set at metric authoring time
        ...

    def _match_contract(self, intent: str, schema: list) -> str:
        # Input: intent pattern + result schema field list
        # Output: named chart contract – e.g. BAR_MULTI_SERIES_COMPARISON, LINE_TIME_SERIES_TREND,
        # TABLE_GOVERNED
        # Looks up (intent, metric_count) in INTENT_CONTRACTS map; defaults to TABLE_GOVERNED
        ...

    def _build_display_spec(self, contract: str, result: dict, operation: dict) -> dict:
        # Input: contract name + assembled result + operation definition
        # Output: Vega-Lite v5 spec (for charts) or { type: "table", columns, data } (for tables)

        # Each contract has a fixed encoding shape – no runtime chart-type decisions
        # Multi-metric charts pivot to long form (one row per dimension × metric)
        # TABLE_GOVERNED is always the safe fallback if no contract matches
        ...

```

Static Image Rendering (vega2img)

The vega2img service uses vega-embed to display Vega-Lite charts in a Vite application. Playwright controls headless Chromium to capture the rendered output. A custom HTML template supplies the table layout for table screenshots.

vega2img is a **standalone MCP render service**, not part of the Analytics Platform. Consumers that need static image output register it as a peer MCP server alongside the Analytics Platform.

Decision	Choice	Rationale
Integration	Standalone MCP server registered directly with consumers	Keeps rendering outside the Analytics Platform; consumers decide when to call it
Implementation	Vite + vega-embed + headless Chromium (Playwright)	Pixel-accurate SVG/PNG from Vega-Lite specs

Decision	Choice	Rationale
Table rendering	Custom HTML template + Playwright screenshot	Styled table rendering

Consumer	Chart	Table	Static image
AI Chat Platform	vega-embed	Native data table	vega2img (direct MCP call)
Custom UI	vega-embed (recommended)	Host's own grid	vega2img
Agentic consumers	vega2img	vega2img	vega2img

```

import json
import base64
from playwright.async_api import async_playwright
from fastmcp import FastMCP
from pydantic import BaseModel
from typing import Literal

mcp = FastMCP(
    name="vega2img",
    instructions="Render Vega-Lite display specs and tables to static PNG or SVG images.",
)

class RenderChartInput(BaseModel):
    display_spec: dict  # Vega-Lite v5 DVL spec from Analytics Platform
    format: Literal["png", "svg"] = "png"
    width: int = 800
    height: int = 400

class RenderTableInput(BaseModel):
    display_spec: dict  # type: "table" DVL spec from Analytics Platform
    format: Literal["png", "svg"] = "png"
    width: int = 900

@mcp.tool()
async def render_chart(input: RenderChartInput) -> dict:
    """Render a Vega-Lite display spec from the Analytics Platform to a static image.
    Returns a base64-encoded image and MIME type."""
    # 1. Build an HTML page embedding the Vega-Lite spec via vega-embed
    # 2. Screenshot the rendered canvas via Playwright – returns base64 PNG or SVG
    ...

@mcp.tool()
async def render_table(input: RenderTableInput) -> dict:
    """Render a table display spec from the Analytics Platform to a static image.
    Returns a base64-encoded image and MIME type."""
    # 1. Build a plain HTML table from the DVL table spec
    # 2. Screenshot via Playwright – same path as render_chart
    ...

def _vega_embed_html(spec: dict, width: int, height: int) -> str:
    # Input: Vega-Lite v5 spec dict + dimensions
    # Output: self-contained HTML page that renders the spec via vega-embed
    ...

def _table_html(spec: dict, width: int) -> str:
    # Input: DVL table spec – { columns: [{field, label, type}], data: { values: [...] } }
    # Output: styled HTML table string – col["field"] used for row lookup, col["label"] for headers
    ...

async def _screenshot(html: str, fmt: str, width: int, height: int) -> bytes:
    # Input: rendered HTML + format + viewport dimensions
    # Output: raw image bytes – PNG or SVG
    # Launches headless Chromium via Playwright; waits for Vega canvas render before capturing
    ...

if __name__ == "__main__":
    mcp.run(transport="streamable-http", host="0.0.0.0", port=8001)

```

Analytical Lineage Store

Specification: §Analytical Lineage Store (ALS)

S3-compatible object storage stores the ALS records, and PostgreSQL indexes the fields needed to find them. The Python `AnalyticalLineageStore` class coordinates writes and retrieval across both stores. Storage retention controls and the application integrity checks described below protect the records.

Decision	Choice	Rationale
Lineage records	S3-compatible object store — one JSON document per query	Write-once; append-only; cheap at scale; no schema migration required; natural fit for immutable audit records
Object key	<code>lineage/{org_id}/{yyyy}/{mm}/{dd}/{result_id}.json</code>	Date-partitioned; enables prefix-based listing by time window
Search index	Thin PostgreSQL table (scalar fields only, no JSON blobs)	Used by the Lineage Query REST API for filtered search; full records are always fetched from the object store.
Retention	Object lifecycle policy — sample default 7 years (configurable)	Long-horizon regulatory retention; enforced at the storage layer, not application code. Periods are deployment choices — the design documents deliberately prescribe none

The lineage store is a high-sensitivity security boundary because records can contain identity, entitlement, query, and result information. Deployments must encrypt records in transit and at rest with tenant-scoped access controls and auditable key rotation. Result payloads are excluded by default; the record stores a digest, schema, row count, and bounded summary unless an approved retention policy explicitly requires full result preservation. Requests and SQL are redacted for secrets and unnecessary personal data before persistence.

Claims of write-once retention require a storage-enforced WORM control such as object lock in compliance mode (or an equivalent control), versioning, retention policy, and legal-hold support. Application convention alone is not immutability. Each record is signed or content-addressed, and periodic reconciliation detects missing objects, orphaned index rows, and signature failures. Object creation and index updates use a durable outbox/reconciliation workflow because S3 and PostgreSQL do not share a transaction.

Lineage document schema

Each completed query writes a single JSON document to the object store at `lineage/{org_id}/{yyyy}/{mm}/{dd}/{result_id}.json`:

```
{
  "result_id": "res-20260518-093247",
  "org_id": "acme-wealth",
  "user_sub": "auth0|user_xyz",
  "lqp_id": "lqp-20260518-093243",
  "cache_hit": false,
  "request_payload": { "tool": "run_analytics", "input": { "operation_id":
"compare_portfolio_to_benchmark", "params": { "portfolio_scope": "caller_authorized_portfolios",
"asset_class": "EQUITY", "time_period": "current_quarter" } } },
  "resolved_metrics": [{ "metric_id": "portfolio_return", "version": "2.1.0" }],
  "controls_decision": { "approved": true, "estimated_scan_rows": 408517, "checks_passed":
["data_scale_check", "complexity_check", "classification_gate", "compliance_check", "concurrency_check"] },
  "execution": { "engine": "starburst", "catalogs_used": ["snowflake", "risk"], "executed_sql":
"..", "latency_ms": 1243 },
  "result_summary": { "row_count": 2, "schema": [".."], "result_digest": "sha256:.." },
  "display_spec": { "type": "chart", "contract": "BAR_MULTI_SERIES_COMPARISON",
"additional_properties": ".." },
  "error_code": null,
  "regulatory_frameworks": [<framework_id>],
  "compliance_meta": { "justification": "Quarterly review", "trace_id": "trace-20260518-093247-
<framework_id>" },
  "created_at": "2026-05-18T09:32:47Z",
  "expires_at": "2033-05-18T09:32:47Z"
}
```

Records are written once and never mutated. Post-hoc compliance annotations are written as separate sibling documents (`{result_id}_amendment_{n}.json`) referencing the original `result_id` .

Search index schema

A lightweight PostgreSQL table (`analytics.lineage_index`) holds only the scalar fields required for the Lineage Query REST API. Full records are always retrieved from the S3 object store; this table is never the source of truth for record content. Each row corresponds to the following JSON shape:

```
{
  "result_id": "res-20260518-093247",
  "org_id": "acme-wealth",
  "user_sub": "auth0|user_xyz",
  "regulatory_frameworks": "<framework_id>",
  "estimated_scan_rows": 408517,
  "error_code": null,
  "cache_hit": false,
  "created_at": "2026-05-14T09:32:47Z",
  "expires_at": "2033-05-14T09:32:47Z"
}
```

Field reference:

Field	Type	Notes
<code>result_id</code>	string	Primary key; matches S3 object path
<code>org_id</code>	string	Organization scope
<code>user_sub</code>	string	JWT <code>sub</code> claim of the requesting user

Field	Type	Notes
regulatory_frameworks	string null	Comma-separated framework tags from triggered metrics; null for non-compliance queries
estimated_scan_rows	integer null	Tier-2 scan estimate recorded by the data-scale check
error_code	string null	Null for successful queries
cache_hit	boolean	True if result was served from Redis cache
created_at	ISO 8601	Query execution timestamp
expires_at	ISO 8601	Computed from retention policy; enforced at S3 lifecycle layer

Indexed on (org_id, user_sub, created_at DESC) , (org_id, created_at DESC) , and (org_id, regulatory_frameworks) for the compliance-filtered query pattern.

```

import json

def utc_now() -> str:
    # Output: current UTC time as ISO 8601 string with Z suffix
    ...

def compute_expiry(lqp: dict) -> str:
    # Input: LQP – checks compliance_purpose to select retention period
    # Output: ISO 8601 expiry timestamp – sample defaults: 7 years; 10 years for compliance-purpose queries
    ...

class AnalyticalLineageStore:
    def __init__(self, s3_client, pg_pool, bucket: str):
        self.s3 = s3_client
        self.pg = pg_pool
        self.bucket = bucket # S3 bucket name for lineage records

    async def write(self, record: dict) -> str:
        # Input: lineage record dict
        # Output: result_id string
        # Writes JSON to S3 at date-partitioned key, then indexes scalar fields in PostgreSQL
        ...

    async def fetch(self, result_id: str, org_id: str) -> dict:
        # Input: result_id + org_id (organization scope)
        # Output: full lineage record from S3
        # PostgreSQL index used only to resolve the S3 key – full record always read from S3
        ...

    async def write_projection(self, projection: dict, claims: dict) -> None:
        # Input: RAPL entitlement projection + JWT claims
        # Writes the projection record to S3 keyed by intent_id (before lqp_id exists)
        # First of the three ALS writes – entitlement decisions (including denials) are
        # recorded even when the request stops before the SCL runs
        ...

    async def write_controls_decision(self, lqp: dict, claims: dict) -> None:
        # Input: SCL-approved LQP + JWT claims
        # Writes a controls_decision record to S3 keyed by lqp_id (before result_id exists)
        # Second of the three ALS writes – captures governance decision before FQE runs
        ...

    async def write_execution(self, lqp: dict, result: dict) -> None:
        # Input: approved LQP + assembled FQE result
        # Builds a full execution lineage record – includes the federated SQL + catalogs used,
        # regulatory_frameworks, result summary
        # Third of the three ALS writes – called by FQE after assembly
        # regulatory_frameworks aggregated from resolved_metrics with compliance_relevant: true
        ...

    def _object_key(self, org_id: str, result_id: str, created_at: str) -> str:
        # Input: org_id + result_id + ISO timestamp
        # Output: S3 key – lineage/{org_id}/{yyyy}/{mm}/{dd}/{result_id}.json
        ...

    async def _index(self, record: dict) -> None:
        # Input: full lineage record dict
        # Inserts scalar fields only into analytics.lineage_index – never stores JSON blobs
        # Index supports the Lineage Query API – full records are always fetched from S3
        ...

```

Knowledge Store

Used by: MCP Resource handlers

S3-compatible object storage stores the Knowledge Store's versioned Markdown or MDX documents. Python resource handlers retrieve the active versions for MCP consumers, and the Admin API manages document updates.

Decision	Choice	Rationale
Storage	S3-compatible object store (versioned Markdown or MDX files)	Human-readable; diffable; straightforward Admin API management
Access	Read-only at runtime via MCP resource handlers	No user data; no controls pipeline required
Management	Admin API — create, update, version knowledge artifacts	Administrators can extend or override default content
Defaults	Bundled at installation alongside the Financial Services Reference Model	Covers platform overview, all six analytical domains, core skills definitions, and a regulatory compliance provenance guide

Each knowledge artifact is a versioned Markdown document identified by a URI path that maps directly to its MCP resource address (`guide://analytics/platform-overview` → `guide/analytics/platform-overview.md`). The active version for each artifact is controlled via the Admin API; previous versions are retained for audit purposes. Administrators may add custom skills definitions and workflow guides without modifying the platform defaults.

```
class KnowledgeStore:
    def __init__(self, s3_client, bucket: str):
        self.s3 = s3_client
        self.bucket = bucket

    def get(self, artifact_path: str) -> str:
        # Input: artifact path – maps directly to MCP resource URI (e.g. "guide/platform-overview")
        # Output: Markdown content string
        ...

    async def put(self, artifact_path: str, content: str, author: str) -> str:
        # Input: artifact path + Markdown content + author identifier
        # Output: version ID string
        # Called via Admin API only – previous version retained in S3 with version suffix for audit
        ...
```

Result Cache

Redis acts as the Result Cache. The Python `ResultCache` class derives keys from the complete effective execution context, applies the configured TTL, and bypasses cache reads and writes when the compliance trigger is active. A cache hit is usable only after current authorization has been evaluated and produces fresh lineage evidence.

Decision	Choice	Rationale
Store	Redis (cluster mode)	Provides shared cache storage and key expiry; read latency requires measurement in the deployed environment.

Decision	Choice	Rationale
Cache key	SHA-256 of the canonical effective execution context	Includes the LQP, tenant, entitlement projection, pinned definition and policy versions, relevant control and mapping versions, output contract, and source-freshness token.
TTL	5 minutes default; configurable per operation via <code>cache_ttl_seconds</code> on <code>analytical_operation</code>	Short TTL balances freshness against backend load
Compliance bypass	Requests for which the SCL activates the two-signal compliance trigger skip read and write	The Provenance Artifact requires fresh execution evidence.
Cache-aside pattern	FQE checks before execution; writes after assembly	Cache is never on the critical governance path

```
import hashlib, json

class ResultCache:
    def __init__(self, redis_client):
        self.redis = redis_client

    def _key(self, execution_context: dict) -> str:
        # Input: LQP + tenant + entitlement projection + pinned definition, policy,
        # mapping, control, output-contract, and source-freshness versions
        # Output: Redis key string — SHA-256 of the canonical serialized context
        ...

    async def get(self, lqp: dict, claims: dict) -> dict | None:
        # Input: LQP + claims
        # Output: cached result dict, or None on miss
        # Reauthorize the request before reuse. Return None when the SCL compliance trigger is active.
        ...

    async def set(self, lqp: dict, claims: dict, result: dict, ttl: int = 300) -> None:
        # Input: LQP + claims + assembled result + TTL seconds (default 5 min)
        # No-op when the SCL compliance trigger is active.
        ...
```

The FQE uses a cache-aside pattern: check before execution, write after assembly. See `FederatedQueryEngine.execute()` in §Federated Query Engine above for the full implementation.

Admin API

The Admin API is a REST service authenticated with a platform service token (Bearer). It is the only write path for DCS documents (metric definitions, controls config, knowledge artifacts) outside of the normal SDR authoring workflow.

Method	Path	Description
POST	<code>/v1/admin/smr/seed</code>	Import a Financial Services Reference Model bundle. Body: JSON array of <code>analytical_metric</code> , <code>analytical_dimension</code> , or <code>analytical_operation</code> documents. Idempotent — existing approved documents are skipped.
GET	<code>/v1/admin/smr/metrics</code>	List all metrics for an org with status filter. Query: <code>?org_id=&status=proposed\ in_review\ approved\ deprecated</code>

Method	Path	Description
POST	<code>/v1/admin/smr/metrics/{metric_id}/approve</code>	Transition a metric from <code>in_review</code> to <code>approved</code> . Body: <code>{ "approved_by": "user@example.com" }</code>
PUT	<code>/v1/admin/controls/config</code>	Replace the controls config document for an org. Body: <code>controls_config</code> JSON document.
GET	<code>/v1/admin/controls/config</code>	Fetch the current controls config for an org.
POST	<code>/v1/admin/knowledge/{artifact_path}</code>	Create or update a knowledge artifact. Body: Markdown text. Path maps to MCP resource URI.
GET	<code>/v1/admin/knowledge/{artifact_path}</code>	Fetch the current active version of a knowledge artifact.
GET	<code>/v1/admin/lineage/{result_id}</code>	Fetch a full lineage record from the object store by result ID.
GET	<code>/v1/admin/health</code>	Platform health check — returns status of all registered backends and DCS connectivity.

Admin Console. A web-based Admin Console provides a visual interface over the same API surface; changes made in the console are identical in effect to the corresponding API calls and appear in the platform audit log.

Registration validation. On submission of a backend registration or controls configuration change, the Admin API validates before accepting the record:

Validation	Description
Backend reachability	Connectivity probe to the registered <code>endpoint</code> , confirmed within the configured timeout
Data affinity consistency	Each <code>dataAffinity</code> value checked against the known logical domain registry — unrecognized labels generate a warning
Role claim field	<code>roleClaimField</code> verified to be a non-empty string; a warning is surfaced if no token issued in the last 30 days contained that claim
Metric resolution	Metrics whose <code>data_affinity</code> has no matching registered backend generate a resolution warning

Warnings do not block registration; errors (unreachable endpoint, invalid schema) block it until resolved.

```

async def handle_seed_bundle(request: Request) -> Response:
    # Input: POST body – JSON array of analytical_metric / analytical_dimension / analytical_operation docs
    # Output: 207 Multi-Status – { seeded: [...], skipped: [...], errors: [...] }

    # 1. Verify platform service token – this endpoint requires admin credentials, not user JWT
    # 2. For each document: skip if an approved version already exists (idempotent import)
    # 3. Write to DCS; collect seeded/skipped/error counts
    ...

```

Platform Settings

Alongside `controls_config`, a platform settings document configures analytical scope, model tiers, and feature flags:

```
{
  "type": "platform_settings",
  "org_id": "acme-wealth",
  "scope": {
    "analyticalDomain": "wealth_management",
    "fiscalYearStartMonth": 1
  },
  "models": {
    "narrativeSynthesisModel": "fast"
  },
  "features": {
    "naturalLanguageQuery": true, "governedDrilldown": true, "narrativeSynthesis": true,
    "resultDownload": true, "benchmarkComparison": true, "regulatoryReporting": false
  }
}
```

`analyticalDomain` selects the Financial Services Reference Model bundles imported at setup. `fiscalYearStartMonth` anchors the fiscal calendar for time-relative metric expressions such as `YTD` and `FY`. The `models` block selects the NSA inference tier — `"fast"` (Haiku) or `"standard"` (Sonnet); see §Narrative Synthesis Agent. Feature flags toggle presentation capabilities without affecting the controls pipeline: disabling `narrativeSynthesis` removes the narrative from responses but has no effect on lineage, entitlement enforcement, or computation. Metric approval, ownership, and versioning are architectural behaviors of the SMR governance workflow, not settings — there are no toggles for them.

Service Startup and Dependency Wiring

```
# app.py – dependency construction and service startup
import asyncio, asyncpg, boto3, redis.asyncio as aioredis
from anthropic import AsyncAnthropic
from fastmcp import FastMCP

async def build_app() -> FastMCP:
    # Output: fully wired FastMCP application ready to serve on port 8000

    # 1. Load config from env vars
    # 2. Construct infrastructure clients – asyncpg pool, S3, Redis, DCS, Anthropic
    # 3. Construct platform services – ALS, ResultCache, SMR, IRA, RAPL, SVL, SCL, PQP, DVL, NSA, PAS
    # 4. Configure Starburst catalogs (one per registered source) + the physical_mapping.source → catalog map
    # 5. Assemble FederatedQueryEngine with the Starburst coordinator DSN + ALS + cache
    # 6. Assemble PipelineExecutor with all services injected (IRA → RAPL → SVL → SCL → PQP → FQE)
    # 7. Wire everything into the FastMCP app and return
    ...

if __name__ == "__main__":
    app = asyncio.run(build_app())
    app.run(transport="streamable-http", host="0.0.0.0", port=8000)
```

Configuration is read from environment variables at startup. Required variables:

Variable	Description
POSTGRES_DSN	PostgreSQL connection string (lineage index + role policies)
REDIS_URL	Redis connection URL (cache + concurrency semaphore)
DCS_URL	Data Context Store base URL
DCS_API_KEY	DCS service-to-service API key
S3_LINEAGE_BUCKET	S3 bucket name for lineage records
STARBURST_DSN	Starburst (Trino) coordinator connection — host, port, user, default catalog
CALCITE_PLANNER_URL	Loopback URL for the Calcite planner sidecar; not routable outside the application pod.
ANTHROPIC_API_KEY	Anthropic API key for the IRA (intent ranking) and NSA (narrative synthesis)
JWT_JWKS_URI	JWKS endpoint for JWT public key retrieval
JWT_AUDIENCE	Expected JWT audience claim
JWT_ISSUER	Expected JWT issuer claim
PAS_SIGNING_KEY_PATH	Path to the ECDSA P-256 private key mounted from Vault / Kubernetes Secrets (PAS artifact sealing)
PAS_SIGNING_KEY_ID	Published key identifier included in artifact signature blocks (verification and rotation)

Infrastructure

Kubernetes runs the reference build as a small number of deployment units. The `analytics-mcp` pod contains the Python modular monolith and its Calcite planner sidecar. The separately deployed Admin API cannot execute analytical requests. Starburst executes federated queries, PostgreSQL stores policies and indexes, Redis stores cached results and concurrency state, and S3-compatible storage stores lineage and knowledge documents. Vault or the selected cloud secrets service supplies credentials to these components.

Component	Choice	Rationale
Governed application	Python, FastMCP, and Uvicorn in <code>analytics-mcp</code>	MCP, IRA and NSA adapters, RAPL, SVL, SCL, PQP client, DVL, ALS coordination, PAS, and cache coordination run as modules in one application process.
Planner adapter	Apache Calcite JVM sidecar in the <code>analytics-mcp</code> pod	Exposes the versioned planning contract only on loopback; has no source credentials and cannot execute queries.
Federated Query Engine	Starburst (Trino) — Enterprise or Galaxy	Coordinator + workers; one catalog per registered source; performs all cross-source federation
Primary database	PostgreSQL (managed service)	Hosts the ALS search index and the DES <code>role_policies</code> schema under separate schemas, credentials, migration histories, and access paths.
Data Context Store (DCS)	Pre-existing platform component	SMR metric definitions, controls config, SMR search — reuses SDR versioned storage and native search
Knowledge Store	S3-compatible object store (versioned Markdown)	MCP resource content — guides, skills definitions, compliance reference
Object storage	S3-compatible	Lineage records (one JSON document per query), result artifacts, large cached result sets
Secrets	HashiCorp Vault or cloud-native	Starburst catalog credentials, platform service keys

Kubernetes Deployment Summary

The following values are bootstrap assumptions for an evaluation environment, not production sizing recommendations or service-level commitments. The evaluation establishes measured resource baselines and scaling thresholds before a pilot decision.

Deployment Unit	Containers or Service	Network Surface	Evaluation Starting Point
Governed application pod	<code>analytics-mcp</code> plus <code>calcite-planner</code> sidecar	MCP on port 8000; planner available only on pod loopback	Two replicas to exercise shared concurrency and cache behavior; resource requests established by an initial load baseline.
Admin API	<code>analytics-admin</code>	Authenticated administrative API on port 9000	One replica; no route to analytical execution or DES writes.
Optional rendering	<code>vega2img</code>	Separate MCP service on port 8001	Excluded from the minimum build; add only for image-output evaluation.

Deployment Unit	Containers or Service	Network Surface	Evaluation Starting Point
Federated Query Engine	Managed or self-hosted Starburst or compatible Trino distribution	Private coordinator endpoint	Sized for the selected two-source workload and configured with query cancellation and resource limits.
Persistence	Managed PostgreSQL, Redis, and S3-compatible object storage	Private service endpoints	Separate credentials and network policies; backup, restore, retention, and reconciliation tested before formal evaluation.

Each service exposes a minimal unauthenticated liveness endpoint that returns only `{"status": "ok"}`. A separate authenticated readiness endpoint reports dependency state to authorized operators; it does not expose catalog names or infrastructure topology to ordinary consumers.

All platform services run in a dedicated Kubernetes namespace (`analytics`). Workload identity is preferred for cloud services. Credentials that cannot use workload identity are delivered through a managed secrets provider and mounted as files or short-lived runtime material; they are never baked into container images. The PAS signing key should be non-exportable through KMS or HSM for a high-assurance evaluation. A file-mounted key is permitted only as an explicitly documented reference-build simplification.

Financial Services Reference Model

The reference model below is extended sample content rather than minimum-build scope. Formal evaluation needs only the definitions required by the three operations in the Reference Build Profile. The remaining domain bundles demonstrate how the catalog can expand after the build proves the core contracts.

Decision	Choice	Rationale
Packaging	Versioned JSON document bundles (one per domain)	Conforms directly to the SMR <code>analytical_metric</code> schema; idempotently importable via <code>POST /v1/admin/smr/seed</code> ; selective per-domain activation
Distribution	Bundled at installation; updatable from Semantic Registry Service	Air-gapped deployments supported
Activation	<code>analyticalDomain</code> config triggers SMR import at initial platform setup	Bundle documents are written to the SMR in <code>proposed</code> state; Analytics Governance approves before metrics become resolvable
Customization	Full edit/override via Admin API after import	Customized definitions marked <code>source: "custom"</code> in the SMR document

The full reference model covers six primary domains, activated per domain via `analyticalDomain` / seed configuration:

Domain	Key metrics	Typical consumers
Wealth management	Portfolio return, tracking error, Sharpe ratio, benchmark return, active return, information ratio	Portfolio managers, private bankers
Risk management	VaR 95/99, expected shortfall, beta, duration, convexity, issuer concentration	Risk officers, investment committees

Domain	Key metrics	Typical consumers
Performance attribution	Brinson attribution, factor attribution, sector attribution, active weight	Portfolio managers, analysts
Regulatory reporting	LCR, NSFR, leverage ratio, capital ratios, RWA, concentration limits	Compliance teams, regulatory reporting
Banking	NIM, RWA density, provision coverage, cost-to-income, deposit beta	Finance, treasury, credit risk
ESG	Carbon intensity, ESG score, engagement coverage, exclusion exposure	Sustainability analysts, client reporting

The hierarchies shipped with the reference model — including the asset class, geography, and time hierarchies — become available for governed drilldown immediately upon approval of the associated dimension definitions. The bundles below illustrate the shared dimensions and three of the domains in full.

Each bundle is a JSON array of SMR documents conforming to the schemas defined in §Semantic Metrics Repository. Bundles are seeded into the SMR in `"proposed"` state at initial platform setup; the Analytics Governance approves each document before it becomes resolvable by the Semantic Validation Layer.

Version format note: Seed bundle documents use an integer `version` field (starting at 1) as the bootstrap state. On first platform activation the platform converts these to semantic versioning (`"1.0.0"`). Subsequent versions follow semver — the `"2.1.0"` form used in the worked examples reflects a metric that has been through two major revisions post-activation.

Dimensions bundle (shared across all domains)

One bundle covers all analytical dimensions. Every domain bundle's metrics and operations reference these dimension IDs.

```
[
  {
    "type": "analytical_dimension",
    "org_id": "acme-wealth",
    "dimension_id": "asset_class",
    "version": 1,
    "status": "approved",
    "source": "platform",
    "display_name": "Asset Class",
    "description": "Top-level asset class classification applied to holdings.",
    "data_affinity": "portfolio",
    "physical_mapping": { "source": "primary-warehouse", "table": "dim_asset_classification", "column":
"asset_class" },
    "values": ["EQUITY", "FIXED_INCOME", "ALTERNATIVES", "CASH", "DERIVATIVES"],
    "hierarchical": false,
    "parent_dimension": null
  },
  {
    "type": "analytical_dimension",
    "org_id": "acme-wealth",
    "dimension_id": "geography",
    "version": 1,
    "status": "approved",
    "source": "platform",
    "display_name": "Geography",
    "description": "Country of domicile of the issuer. Rolls up to region and continent.",
    "data_affinity": "portfolio",
    "physical_mapping": { "source": "primary-warehouse", "table": "dim_geography", "column":
"country_iso2" },
    "values": null,
    "hierarchical": true,
    "parent_dimension": null,
    "hierarchy_levels": ["continent", "region", "country"]
  },
  {
    "type": "analytical_dimension",
    "org_id": "acme-wealth",
    "dimension_id": "sector",
    "version": 1,
    "status": "approved",
    "source": "platform",
    "display_name": "Sector (GICS)",
    "description": "GICS Level 1 sector classification.",
    "data_affinity": "portfolio",
    "physical_mapping": { "source": "primary-warehouse", "table": "dim_sector", "column": "gics_sector" },
    "values": ["ENERGY", "MATERIALS", "INDUSTRIALS", "CONSUMER_DISCRETIONARY", "CONSUMER_STAPLES",
"HEALTH_CARE", "FINANCIALS", "INFORMATION_TECHNOLOGY", "COMMUNICATION_SERVICES",
"UTILITIES", "REAL_ESTATE"],
    "hierarchical": true,
    "parent_dimension": null,
    "hierarchy_levels": ["sector", "industry_group", "industry", "sub_industry"]
  },
  {
    "type": "analytical_dimension",
    "org_id": "acme-wealth",
    "dimension_id": "currency",
    "version": 1,
    "status": "approved",
    "source": "platform",
    "display_name": "Currency",

```

```

    "description": "Denomination currency of the holding (ISO 4217).",
    "data_affinity": "portfolio",
    "physical_mapping": { "source": "primary-warehouse", "table": "fact_portfolio_daily", "column":
"currency_code" },
    "values": null,
    "hierarchical": false,
    "parent_dimension": null
  },
  {
    "type": "analytical_dimension",
    "org_id": "acme-wealth",
    "dimension_id": "issuer",
    "version": 1,
    "status": "approved",
    "source": "platform",
    "display_name": "Issuer",
    "description": "Legal entity name of the security issuer. High cardinality – use with limit.",
    "data_affinity": "portfolio",
    "physical_mapping": { "source": "primary-warehouse", "table": "dim_issuer", "column": "issuer_name" },
    "values": null,
    "hierarchical": false,
    "parent_dimension": null
  }
]

```

Performance domain bundle (domain: "performance")

```
[
  {
    "type": "analytical_metric",
    "org_id": "acme-wealth",
    "metric_id": "market_value",
    "version": 1,
    "status": "approved",
    "source": "platform",
    "display_name": "Market Value",
    "description": "End-of-period market value of a portfolio or position in the portfolio base
currency.",
    "domain": "performance",
    "category": "valuation",
    "unit": "currency",
    "decimals": 2,
    "aggregation": "sum",
    "performance_impact_weight": 1,
    "classification_level": "internal",
    "data_affinity": "portfolio",
    "physical_mapping": { "source": "primary-warehouse", "table": "fact_portfolio_daily", "measure":
"market_value_base_ccy" },
    "compliance_relevant": false,
    "regulatory_framework": [],
    "required_dimensions": ["portfolio_id", "as_of_date"],
    "optional_dimensions": ["asset_class", "currency", "sector"],
    "approved_by": "cdo@acme.com",
    "approved_at": "2026-05-14T09:00:00Z",
    "created_at": "2026-05-13T14:32:00Z"
  },
  {
    "type": "analytical_metric",
    "org_id": "acme-wealth",
    "metric_id": "portfolio_return",
    "version": 1,
    "status": "approved",
    "source": "platform",
    "display_name": "Portfolio Return",
    "description": "Total return of a portfolio over the specified period, net of fees.",
    "domain": "performance",
    "category": "total_return",
    "unit": "percentage",
    "decimals": 2,
    "aggregation": "value_weighted_average",
    "weight_metric_id": "market_value",
    "performance_impact_weight": 1,
    "classification_level": "internal",
    "data_affinity": "portfolio",
    "physical_mapping": { "source": "primary-warehouse", "table": "fact_portfolio_daily", "measure":
"total_return_net" },
    "compliance_relevant": false,
    "regulatory_framework": [],
    "required_dimensions": ["portfolio_id", "time_period"],
    "optional_dimensions": ["benchmark_id", "asset_class"],
    "approved_by": "cdo@acme.com",
    "approved_at": "2026-05-14T09:00:00Z",
    "created_at": "2026-05-13T14:32:00Z"
  },
  {
    "type": "analytical_metric",
    "org_id": "acme-wealth",
```

```

    "metric_id": "sharpe_ratio",
    "version": 1,
    "status": "approved",
    "source": "platform",
    "display_name": "Sharpe Ratio",
    "description": "Annualised excess return divided by annualised standard deviation.",
    "domain": "performance",
    "category": "risk_adjusted_return",
    "unit": "ratio",
    "decimals": 2,
    "aggregation": "last",
    "performance_impact_weight": 2,
    "classification_level": "internal",
    "data_affinity": "portfolio",
    "physical_mapping": { "source": "primary-warehouse", "table": "fact_portfolio_analytics", "measure":
"sharpe_ratio_annualised" },
    "compliance_relevant": false,
    "regulatory_framework": [],
    "required_dimensions": ["portfolio_id", "time_period"],
    "optional_dimensions": [],
    "approved_by": "cdo@acme.com",
    "approved_at": "2026-05-14T09:00:00Z",
    "created_at": "2026-05-13T14:32:00Z"
  },
  {
    "type": "analytical_metric",
    "org_id": "acme-wealth",
    "metric_id": "volatility",
    "version": 1,
    "status": "approved",
    "source": "platform",
    "display_name": "Volatility",
    "description": "Annualised standard deviation of portfolio returns.",
    "domain": "performance",
    "category": "risk_adjusted_return",
    "unit": "percentage",
    "decimals": 2,
    "aggregation": "value_weighted_average",
    "weight_metric_id": "market_value",
    "performance_impact_weight": 2,
    "classification_level": "internal",
    "data_affinity": "portfolio",
    "physical_mapping": { "source": "primary-warehouse", "table": "fact_portfolio_analytics", "measure":
"volatility_annualised" },
    "compliance_relevant": false,
    "regulatory_framework": [],
    "required_dimensions": ["portfolio_id", "time_period"],
    "optional_dimensions": ["asset_class"],
    "approved_by": "cdo@acme.com",
    "approved_at": "2026-05-14T09:00:00Z",
    "created_at": "2026-05-13T14:32:00Z"
  },
  {
    "type": "analytical_operation",
    "org_id": "acme-wealth",
    "operation_id": "portfolio_return",
    "version": 1,
    "status": "approved",
    "source": "platform",
    "display_name": "Portfolio Return",
    "description": "Total return for a portfolio over a specified period.",

```

```

    "execution_profile": "metric_query",
    "required_params": ["portfolio_id", "time_period"],
    "supported_metrics": ["portfolio_return"]
  },
  {
    "type": "analytical_operation",
    "org_id": "acme-wealth",
    "operation_id": "compare_portfolio_to_benchmark",
    "version": 1,
    "status": "approved",
    "source": "platform",
    "display_name": "Portfolio Comparison",
    "description": "Compare one or more metrics across two or more portfolios, optionally against a benchmark.",
    "execution_profile": "full_analytical",
    "required_params": ["portfolio_ids", "metrics", "time_period"],
    "optional_params": ["benchmark_id"],
    "supported_metrics": ["portfolio_return", "tracking_error", "sharpe_ratio", "volatility", "beta"],
    "default_visualization": "BAR_MULTI_SERIES_COMPARISON"
  },
  {
    "type": "analytical_operation",
    "org_id": "acme-wealth",
    "operation_id": "performance_attribution",
    "version": 1,
    "status": "approved",
    "source": "platform",
    "display_name": "Performance Attribution",
    "description": "BHB or Brinson-Fachler attribution decomposition for a portfolio versus its benchmark.",
    "execution_profile": "full_analytical",
    "required_params": ["portfolio_id", "benchmark_id", "attribution_by", "time_period"],
    "supported_dimensions": ["asset_class", "sector", "geography", "currency"],
    "default_visualization": "WATERFALL_ATTRIBUTION"
  }
]

```

Risk domain bundle (domain: "risk")

```
[
  {
    "type": "analytical_metric",
    "org_id": "acme-wealth",
    "metric_id": "var_95",
    "version": 2,
    "status": "approved",
    "source": "platform",
    "display_name": "Value at Risk (95%)",
    "description": "Maximum expected portfolio loss over a 1-day horizon at 95% confidence.",
    "domain": "risk",
    "category": "market_risk",
    "unit": "percentage",
    "decimals": 2,
    "aggregation": "value_weighted_average",
    "weight_metric_id": "market_value",
    "performance_impact_weight": 3,
    "classification_level": "internal",
    "data_affinity": "risk_metrics",
    "physical_mapping": { "source": "risk-semantic-layer", "cube": "risk_cube", "measure":
"var_95_daily" },
    "compliance_relevant": false,
    "regulatory_framework": [],
    "required_dimensions": ["portfolio_id", "as_of_date"],
    "optional_dimensions": ["asset_class", "geography", "sector", "currency"],
    "approved_by": "cdo@acme.com",
    "approved_at": "2026-05-14T09:00:00Z",
    "created_at": "2026-05-13T14:32:00Z"
  },
  {
    "type": "analytical_metric",
    "org_id": "acme-wealth",
    "metric_id": "var_99",
    "version": 1,
    "status": "approved",
    "source": "platform",
    "display_name": "Value at Risk (99%)",
    "description": "Maximum expected portfolio loss over a 1-day horizon at 99% confidence.",
    "domain": "risk",
    "category": "market_risk",
    "unit": "percentage",
    "decimals": 2,
    "aggregation": "value_weighted_average",
    "weight_metric_id": "market_value",
    "performance_impact_weight": 4,
    "classification_level": "internal",
    "data_affinity": "risk_metrics",
    "physical_mapping": { "source": "risk-semantic-layer", "cube": "risk_cube", "measure":
"var_99_daily" },
    "compliance_relevant": false,
    "regulatory_framework": [],
    "required_dimensions": ["portfolio_id", "as_of_date"],
    "optional_dimensions": ["asset_class", "geography", "sector", "currency"],
    "approved_by": "cdo@acme.com",
    "approved_at": "2026-05-14T09:00:00Z",
    "created_at": "2026-05-13T14:32:00Z"
  },
  {
    "type": "analytical_metric",
    "org_id": "acme-wealth",
```

```

    "metric_id": "tracking_error",
    "version": 2,
    "status": "approved",
    "source": "platform",
    "display_name": "Tracking Error",
    "description": "Annualised standard deviation of the difference between portfolio and benchmark
returns.",
    "domain": "risk",
    "category": "relative_risk",
    "unit": "percentage",
    "decimals": 2,
    "aggregation": "value_weighted_average",
    "weight_metric_id": "market_value",
    "performance_impact_weight": 2,
    "classification_level": "internal",
    "data_affinity": "risk_metrics",
    "physical_mapping": { "source": "risk-semantic-layer", "cube": "risk_cube", "measure":
"tracking_error_annualised" },
    "compliance_relevant": false,
    "regulatory_framework": [],
    "required_dimensions": ["portfolio_id", "benchmark_id", "as_of_date"],
    "optional_dimensions": ["asset_class", "sector"],
    "approved_by": "cdo@acme.com",
    "approved_at": "2026-05-14T09:00:00Z",
    "created_at": "2026-05-13T14:32:00Z"
  },
  {
    "type": "analytical_metric",
    "org_id": "acme-wealth",
    "metric_id": "expected_shortfall",
    "version": 1,
    "status": "approved",
    "source": "platform",
    "display_name": "Expected Shortfall (CVaR 95%)",
    "description": "Average loss in the worst 5% of outcomes over a 1-day horizon.",
    "domain": "risk",
    "category": "tail_risk",
    "unit": "percentage",
    "decimals": 2,
    "aggregation": "value_weighted_average",
    "weight_metric_id": "market_value",
    "performance_impact_weight": 4,
    "classification_level": "internal",
    "data_affinity": "risk_metrics",
    "physical_mapping": { "source": "risk-semantic-layer", "cube": "risk_cube", "measure":
"cvar_95_daily" },
    "compliance_relevant": false,
    "regulatory_framework": [],
    "required_dimensions": ["portfolio_id", "as_of_date"],
    "optional_dimensions": ["asset_class", "geography"],
    "approved_by": "cdo@acme.com",
    "approved_at": "2026-05-14T09:00:00Z",
    "created_at": "2026-05-13T14:32:00Z"
  },
  {
    "type": "analytical_metric",
    "org_id": "acme-wealth",
    "metric_id": "beta",
    "version": 1,
    "status": "approved",
    "source": "platform",

```

```

    "display_name":      "Beta",
    "description":       "Market beta – sensitivity of portfolio returns to benchmark returns.",
    "domain":           "risk",
    "category":          "market_risk",
    "unit":              "ratio",
    "decimals":          2,
    "aggregation":       "value_weighted_average",
    "weight_metric_id":  "market_value",
    "performance_impact_weight": 2,
    "classification_level": "internal",
    "data_affinity":     "risk_metrics",
    "physical_mapping":  { "source": "risk-semantic-layer", "cube": "risk_cube", "measure":
"portfolio_beta" },
    "compliance_relevant": false,
    "regulatory_framework": [],
    "required_dimensions": ["portfolio_id", "benchmark_id", "as_of_date"],
    "optional_dimensions": ["asset_class", "sector"],
    "approved_by":       "cdo@acme.com",
    "approved_at":       "2026-05-14T09:00:00Z",
    "created_at":        "2026-05-13T14:32:00Z"
  },
  {
    "type":              "analytical_metric",
    "org_id":            "acme-wealth",
    "metric_id":         "duration",
    "version":           1,
    "status":            "approved",
    "source":            "platform",
    "display_name":      "Modified Duration",
    "description":       "Modified duration of the portfolio – sensitivity of price to yield changes.",
    "domain":           "risk",
    "category":          "interest_rate_risk",
    "unit":              "years",
    "decimals":          2,
    "aggregation":       "value_weighted_average",
    "weight_metric_id":  "market_value",
    "performance_impact_weight": 2,
    "classification_level": "internal",
    "data_affinity":     "risk_metrics",
    "physical_mapping":  { "source": "risk-semantic-layer", "cube": "risk_cube", "measure":
"modified_duration" },
    "compliance_relevant": false,
    "regulatory_framework": [],
    "required_dimensions": ["portfolio_id", "as_of_date"],
    "optional_dimensions": ["asset_class", "currency"],
    "approved_by":       "cdo@acme.com",
    "approved_at":       "2026-05-14T09:00:00Z",
    "created_at":        "2026-05-13T14:32:00Z"
  },
  {
    "type":              "analytical_operation",
    "org_id":            "acme-wealth",
    "operation_id":      "retrieve_position_history",
    "version":           1,
    "status":            "approved",
    "source":            "platform",
    "display_name":      "Position History Retrieval",
    "description":       "Retrieve governed current or historical position records for authorized
portfolios.",
    "execution_profile": "data_retrieval",
    "dataset_id":        "fixed_income_daily_positions",

```

```

    "required_params": ["portfolio_scope", "date_from", "date_to"],
    "optional_params": ["asset_class", "requested_fields", "page_size", "page_token"]
  },
  {
    "type": "analytical_operation",
    "org_id": "acme-wealth",
    "operation_id": "risk_breakdown",
    "version": 1,
    "status": "approved",
    "source": "platform",
    "display_name": "Risk Breakdown",
    "description": "Decompose a risk metric into factor contributions by the specified dimension.",
    "execution_profile": "full_analytical",
    "required_params": ["portfolio_id", "metrics", "attribution_by", "as_of_date"],
    "supported_metrics": ["var_95", "var_99", "tracking_error", "beta", "duration",
      "expected_shortfall"],
    "supported_dimensions": ["asset_class", "geography", "sector", "currency", "issuer"],
    "default_visualization": "WATERFALL_ATTRIBUTION"
  }
]

```

Regulatory domain bundle (domain: "regulatory")

All regulatory metrics carry "classification_level": "restricted" , "compliance_relevant": true , and a populated "regulatory_framework" tag. The specific framework identifiers live only on the metric definitions — the platform itself names no framework. The SCL compliance check is triggered for every query against these metrics.

```
[
  {
    "type": "analytical_metric",
    "org_id": "acme-wealth",
    "metric_id": "lcr",
    "version": 1,
    "status": "approved",
    "source": "platform",
    "display_name": "Liquidity Coverage Ratio",
    "description": "High-quality liquid assets as a percentage of net cash outflows over a 30-day
stress period. A regulatory minimum threshold applies.",
    "domain": "regulatory",
    "category": "liquidity",
    "unit": "percentage",
    "decimals": 1,
    "aggregation": "last",
    "performance_impact_weight": 5,
    "classification_level": "restricted",
    "data_affinity": "regulatory",
    "physical_mapping": { "source": "regulatory-data-store", "table": "fact_regulatory_ratios",
"measure": "lcr_ratio" },
    "compliance_relevant": true,
    "regulatory_framework": ["<framework_id>"],
    "required_dimensions": ["entity_id", "reporting_date", "jurisdiction"],
    "optional_dimensions": [],
    "approved_by": "cdo@acme.com",
    "approved_at": "2026-05-14T09:00:00Z",
    "created_at": "2026-05-13T14:32:00Z"
  },
  {
    "type": "analytical_metric",
    "org_id": "acme-wealth",
    "metric_id": "leverage_ratio",
    "version": 1,
    "status": "approved",
    "source": "platform",
    "display_name": "Leverage Ratio",
    "description": "Tier 1 capital as a percentage of total exposure. A regulatory minimum threshold
applies.",
    "domain": "regulatory",
    "category": "capital_adequacy",
    "unit": "percentage",
    "decimals": 1,
    "aggregation": "last",
    "performance_impact_weight": 5,
    "classification_level": "restricted",
    "data_affinity": "regulatory",
    "physical_mapping": { "source": "regulatory-data-store", "table": "fact_regulatory_ratios",
"measure": "leverage_ratio" },
    "compliance_relevant": true,
    "regulatory_framework": ["<framework_id>"],
    "required_dimensions": ["entity_id", "reporting_date", "jurisdiction"],
    "optional_dimensions": [],
    "approved_by": "cdo@acme.com",
    "approved_at": "2026-05-14T09:00:00Z",
    "created_at": "2026-05-13T14:32:00Z"
  },
  {
    "type": "analytical_metric",
    "org_id": "acme-wealth",
```

```

    "metric_id": "nsfr",
    "version": 1,
    "status": "approved",
    "source": "platform",
    "display_name": "Net Stable Funding Ratio",
    "description": "Available stable funding as a percentage of required stable funding. A
regulatory minimum threshold applies.",
    "domain": "regulatory",
    "category": "liquidity",
    "unit": "percentage",
    "decimals": 1,
    "aggregation": "last",
    "performance_impact_weight": 5,
    "classification_level": "restricted",
    "data_affinity": "regulatory",
    "physical_mapping": { "source": "regulatory-data-store", "table": "fact_regulatory_ratios",
"measure": "nsfr_ratio" },
    "compliance_relevant": true,
    "regulatory_framework": [<framework_id>],
    "required_dimensions": ["entity_id", "reporting_date", "jurisdiction"],
    "optional_dimensions": [],
    "approved_by": "cdo@acme.com",
    "approved_at": "2026-05-14T09:00:00Z",
    "created_at": "2026-05-13T14:32:00Z"
  },
  {
    "type": "analytical_operation",
    "org_id": "acme-wealth",
    "operation_id": "regulatory_report",
    "version": 1,
    "status": "approved",
    "source": "platform",
    "display_name": "Regulatory Compliance Report",
    "description": "Entity-level regulatory compliance metric report under the applicable
regulatory framework.",
    "execution_profile": "full_analytical",
    "required_params": ["metric_id", "entity_id", "reporting_date", "jurisdiction"],
    "supported_metrics": ["lcr", "leverage_ratio", "nsfr"],
    "regulatory_framework": [<framework_id>],
    "required_feature_flag": "regulatoryReporting",
    "default_visualization": "TABLE_GOVERNED"
  }
]

```

Complementary Ecosystem Services

Three optional ecosystem services extend the platform for financial services deployments. None is a hard dependency — the platform functions with any combination present or absent.

Service	Type	Activation	Primary benefit
Semantic Registry Service	Config-time resource	Package import via the Admin API (<code>POST /v1/admin/smr/seed</code>)	Accelerated SMR setup from curated, version-controlled metric definition packages across the reference model domains
Regulatory Reference Service	Runtime execution backend	Registered as a data source with <code>dataAffinity: ["regulatory"]</code>	Authoritative, always-current regulatory metric values and thresholds without internal maintenance

Service	Type	Activation	Primary benefit
Benchmark Data Service	Runtime execution backend	Registered as a data source with <code>dataAffinity: ["benchmarks"]</code>	Licensed benchmark data for comparison queries across equity, fixed income, multi-asset, factor, and custom blend indices

Semantic Registry Service — a curated, version-controlled library of pre-built metric definitions, dimension schemas, hierarchy definitions, measure group collections, formula documentation, and regulatory mappings, organized into domain packages. Administrators import packages through the Admin API rather than authoring definitions from scratch. Imported definitions enter the SMR as `proposed` and follow the normal approval workflow; modifications are tracked under `source: "custom"` and are never overwritten by package updates, which are notified and may be imported selectively as deltas. The Financial Services Reference Model bundled at installation is a snapshot of the relevant packages; the live service provides the most current definitions and packages beyond the core seed set.

Regulatory Reference Service — a runtime execution backend serving the `regulatory` data affinity. Once registered, the FQE routes regulatory-domain sub-plans to it, ensuring threshold values (LCR, NSFR, leverage and capital ratios) are sourced from the authoritative service rather than host-maintained tables that may lag regulatory publication schedules. The service publishes update notifications when thresholds change. If it is unavailable, the FQE falls back to the next registered backend with `regulatory` affinity; with no fallback configured, regulatory sub-plans fail with a structured error — the platform never fabricates regulatory threshold values.

Benchmark Data Service — a runtime execution backend serving the `benchmarks` data affinity: equity, fixed income, multi-asset, and factor indices, plus administrator-configured custom benchmark blends (component identifiers and weights, registered via the service's Admin API). The service operates under data licensing agreements with index providers and enforces per-index licensing entitlement checks; blended benchmarks are subject to the same enforcement as their component indices.

Evaluation Plan and Decision Gates

The reference implementation should be evaluated as a bounded architectural proposition, not treated as a production platform merely because its components can be deployed. The evaluation determines whether governed semantic execution produces sufficiently accurate, traceable, repeatable, and access-controlled results for named workloads, and whether its additional governance effort is justified when compared with the organization's current analytical process and a controlled Text-to-SQL implementation.

The evaluation does not establish regulatory compliance. It produces evidence that accountable business, governance, security, and technology owners can use to decide whether to stop, revise, proceed to a limited pilot, or consider broader adoption.

Evaluation Questions

The evaluation must answer six questions:

1. **Accuracy:** Does each result or dataset extract match an independently verified reference result within an approved tolerance?
2. **Traceability:** Can an authorized reviewer reconstruct how a successful, rejected, failed, or timed-out request was interpreted, authorized, planned, executed, and presented?
3. **Repeatability:** Can an authorized rerun reproduce the result or dataset membership when the relevant request, source snapshot, definitions, permissions, configuration, and software versions are preserved?
4. **Governed access:** Do discovery, execution, caching, evidence retrieval, and export enforce the caller's current permissions without cross-role or cross-tenant leakage?
5. **Operability:** Can the implementation meet approved workload expectations for latency, capacity, failure recovery, cost, and administrative effort?
6. **Comparative value:** Does the approach improve governed analytical outcomes enough to justify the cost of authoring definitions, maintaining mappings and policies, and operating the evidence controls?

Scope and Entry Criteria

Before testing begins, the evaluation sponsor and accountable owners approve a versioned evaluation charter containing:

- the business domains, personas, operations, metrics, dataset contracts, and source systems in scope;
- at least one metric comparison, one governed dataset-retrieval operation, and one compliance-triggering operation;
- the approved metric formulas, expected dataset membership, numerical tolerances, ordering rules, and treatment of missing data;
- a preserved or reproducibly addressable source-data snapshot for every repeatability test;
- representative grants, denials, row scopes, masks, classification ceilings, and tenant boundaries;
- the versions of semantic definitions, entitlement policies, controls, mappings, software, model adapters, and prompts under test;
- the workload mix, concurrency range, result sizes, source-failure cases, and recovery scenarios;
- the comparator design, including the existing analytical process and any controlled Text-to-SQL implementation;

- named owners for reference results, access policy, security review, platform operation, evidence review, and the final decision; and
- workload-specific operating thresholds or an explicit statement that the evaluation will establish a baseline before thresholds are approved.

Testing does not begin until reference results and access expectations have been approved independently of the implementation team. A generated answer, an existing dashboard, or the output of another unverified query is not sufficient as a reference result.

Evaluation Corpus

The evaluation corpus must cover the intended operating range rather than a collection of successful demonstrations.

Corpus Area	Required Coverage
Valid analytical requests	Natural-language variants and structured calls for every in-scope operation, including boundary values and permitted drilldowns.
Ambiguous requests	Questions with competing operations, missing parameters, unclear time periods, and ambiguous compliance purpose.
Invalid semantic requests	Unknown, retired, incompatible, or unapproved metrics, dimensions, operations, and datasets.
Access tests	Permitted and denied metrics, dimensions, rows, fields, evidence records, exports, and catalog-discovery results across roles and tenants.
Data-mining tests	Approved dataset selection, field projection, pagination, ordering, row scope, snapshot reference, membership digest, and export behavior.
Control boundaries	Requests immediately below, at, and above scale, complexity, classification, concurrency, and timeout limits.
Failure and recovery	Source unavailability, partial-source failure, stale statistics, cache hit and miss, model unavailability, ALS write failure, PAS signing failure, timeout, retry, and cancellation.
Adversarial cases	Prompt injection, catalog poisoning, parameter manipulation, identifier guessing, cache-isolation attempts, lineage-access probing, and attempts to bypass the governed path.

The charter records the number of cases in each category and explains why the sample represents the intended workload. Results are reported by category and operation; an aggregate percentage must not hide a material failure in a regulated or high-risk scenario.

Evaluation Workstreams

Workstream	Method	Required Evidence	Accountable Owner
Semantic accuracy	Compare computed values, labels, units, dimensions, filters, and dataset membership with independently verified references. Review both natural-language resolution and structured execution.	Expected and actual results, approved tolerance, difference analysis, interpretation outcome, and reviewer sign-off.	Analytics Governance and metric or dataset owner.

Workstream	Method	Required Evidence	Accountable Owner
Entitlements and isolation	Execute positive and negative tests across roles, row scopes, masks, classifications, tenants, cache paths, evidence lookup, and export.	Policy versions, entitlement projections, test identities, returned schemas and rows, denial records, cache evidence, and security findings.	Entitlements Manager and Security.
Controls and compliance	Exercise every SCL boundary and both states of the two-signal compliance trigger. Verify that no execution bypasses SCL and no triggered result is exportable before PAS sealing.	Control inputs, limits, decisions, terminal states, artifact verification, and export-gate evidence.	Analytics Governance and Compliance.
Repeatability	Rerun the same requests against preserved source state and pinned definitions, policies, mappings, configuration, and software.	Run manifests, source snapshot references, result and dataset-membership digests, numerical comparisons, and explanation of every difference.	Evaluation Lead and Data Owner.
Traceability	Give an independent reviewer only the authorized evidence entry point and require reconstruction of sampled complete, rejected, failed, and timed-out requests.	Reconstruction worksheet, retrieved event chain, missing evidence, elapsed review time, and reviewer conclusion.	Independent Audit or Assurance Reviewer.
Resilience and recovery	Inject approved failures at each external dependency and persistence boundary. Verify fail-closed behavior, durable queuing where permitted, reconciliation, recovery, and explicit consumer status.	Failure timeline, alerts, terminal records, recovery record, reconciliation result, and unresolved data or evidence loss.	Platform Admin and Security.
Performance and cost	Run the approved workload mix at the stated concurrency and result sizes. Separate cache hits, executions, source types, presentation profiles, and compliance paths.	Latency distribution, throughput, resource use, model calls, storage growth, unit cost, bottlenecks, and capacity assumptions.	Platform Admin and Finance or FinOps.
Comparative evaluation	Run the same questions and source snapshot through the current process and controlled Text-to-SQL comparator. Do not grant the comparator broader data access or a different reference answer.	Correctness, unsupported questions, repeatability, traceability effort, latency, cost, authoring effort, review effort, and failure analysis for each approach.	Evaluation Lead and Analytics Governance.

Mandatory Invariants

The following are pass-or-fail architectural invariants rather than aspirational operating targets:

Invariant	Acceptance Condition
Reference-result correctness	No unresolved material discrepancy between a governed result and its approved reference result. Numerical tolerances and dataset comparison rules are approved before execution.
Execution control	No request, retry, drilldown, cache path, or administrator action reaches a registered data source without the required entitlement, validation, and SCL decisions.
Tenant and entitlement isolation	No unauthorized metric, dimension, row, field, cached result, catalog metadata, lineage record, or artifact is disclosed.

Invariant	Acceptance Condition
Durable terminal evidence	Every accepted request has a durable terminal record for completion, rejection, cancellation, timeout, or failure. If evidence cannot be written or durably queued under approved policy, execution fails closed.
Compliance export gate	A request with both compliance signals active cannot be exported until the required artifact is complete, sealed, and verifiable.
Repeatability evidence	Every reported repeatability result identifies the request, snapshot or source reference, definition and policy versions, configuration, software version, and comparison outcome.
Failure transparency	Missing sources, partial semantics, stale data, validation failure, timeout, and narrative omission are never presented as a complete governed answer.

A failure of an isolation or export-gate invariant stops affected testing immediately and triggers security review. Other invariant failures require documented remediation and a clean rerun of the affected corpus before an adoption decision.

Operating Measures and Baselines

Availability, latency, throughput, cache effectiveness, backend error rate, cost per successful request, definition-authoring effort, approval time, catalog coverage, user adoption, drilldown use, and narrative use are operating measures. They are not universal proof of architectural correctness.

The evaluation report must distinguish:

Measure Class	Treatment
Architectural invariant	Mandatory acceptance condition; not averaged against other results.
Workload acceptance threshold	Approved in the evaluation charter for a named workload and environment.
Baseline measure	Recorded during evaluation when no defensible target exists yet.
Adoption hypothesis	Used to assess utility during a limited pilot; not used as evidence that calculations are correct.

Targets such as availability, percentile latency, cache-hit rate, or weekly adoption are approved only after the workload, denominator, observation period, exclusions, and business consequence are defined. A high cache-hit or adoption rate cannot compensate for a correctness, isolation, or lineage failure.

Evidence Pack

The Evaluation Lead maintains a versioned evidence pack containing:

- the approved charter and change history;
- corpus definitions and expected outcomes;
- reference-result construction and independent approval;
- environment, software, model, prompt, definition, policy, mapping, and configuration versions;
- source snapshot identifiers and freshness evidence;
- raw test outcomes, structured differences, lineage references, artifact verification, and failure records;
- performance, capacity, cost, and administrative-effort measurements;
- comparator configuration and results;

- defects, remediation, reruns, exceptions, and unresolved risks; and
- signed conclusions from the accountable business, governance, security, and technology owners.

Failed initial runs remain in the evidence pack. A successful rerun supplements rather than replaces the original result.

Decision Gates

Gate	Required Decision
Charter approval	Confirm that scope, owners, reference results, test corpus, invariants, comparator, and operating thresholds or baseline objectives are sufficient to begin.
Readiness review	Confirm that the environment, source snapshots, definitions, policies, observability, evidence capture, and failure-injection controls are ready for formal testing.
Midpoint review	Examine failures and corpus coverage. Continue unchanged, expand a weak area, pause for remediation, or stop if the proposition cannot be tested credibly.
Exit review	Choose proceed , revise and retest , or stop using the complete evidence pack. No weighted score may offset a failed mandatory invariant.

Proceed to a limited pilot only when all mandatory invariants pass, reference-result discrepancies are resolved, workload thresholds are met or explicitly accepted, the comparator supports the claimed benefit, and accountable owners accept the documented residual risks.

Revise and retest when the architecture remains credible but implementation defects, catalog gaps, policy errors, operating limits, or insufficient evidence prevent a decision. The review names the failed criteria, owner, remediation, and required rerun.

Stop when the evaluation cannot establish trustworthy reference results, isolation, complete controls, usable evidence, or a defensible advantage for the intended workloads, or when the governance and operating burden is not justified by the measured benefit.